Tema 10

# El Nivel de Transporte en Internet

Rogelio Montañana

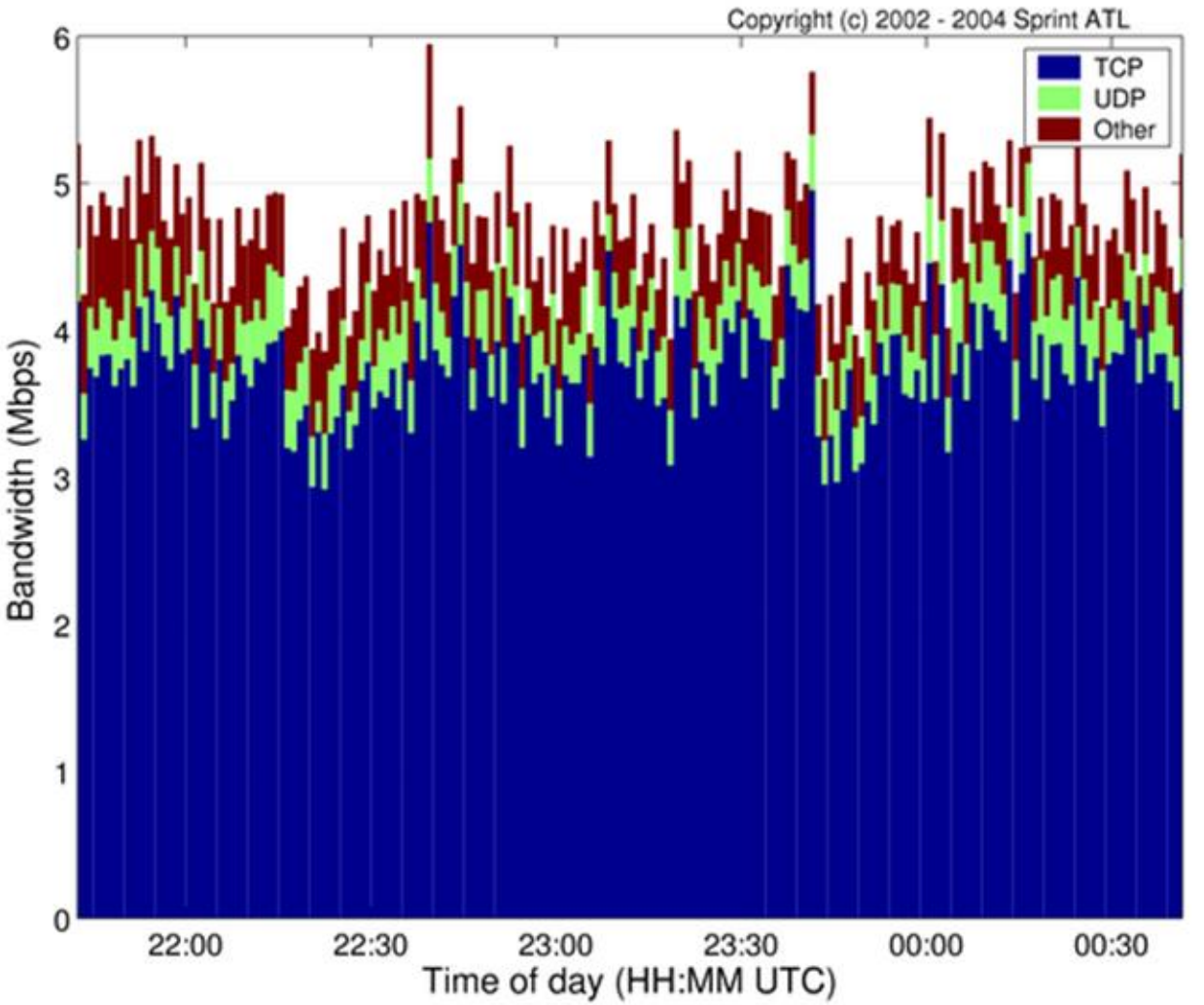
# Sumario

- **Introducción**
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

# Funciones del Nivel de Transporte

- Transporta los datos extremo a extremo (host a host).
- Realiza la comunicación de forma transparente al medio físico. Usa los servicios del nivel de red
- Multiplexa tráfico de diversas instancias (procesos) del nivel de aplicación. El nivel de transporte (como el de red) tiene una (y sólo una) instancia en el host para TCP y otra para UDP
- El servicio que ofrece puede ser:
  - Garantizado: TCP (orientado a conexión)
  - No garantizado: UDP (no orientado a conexión)

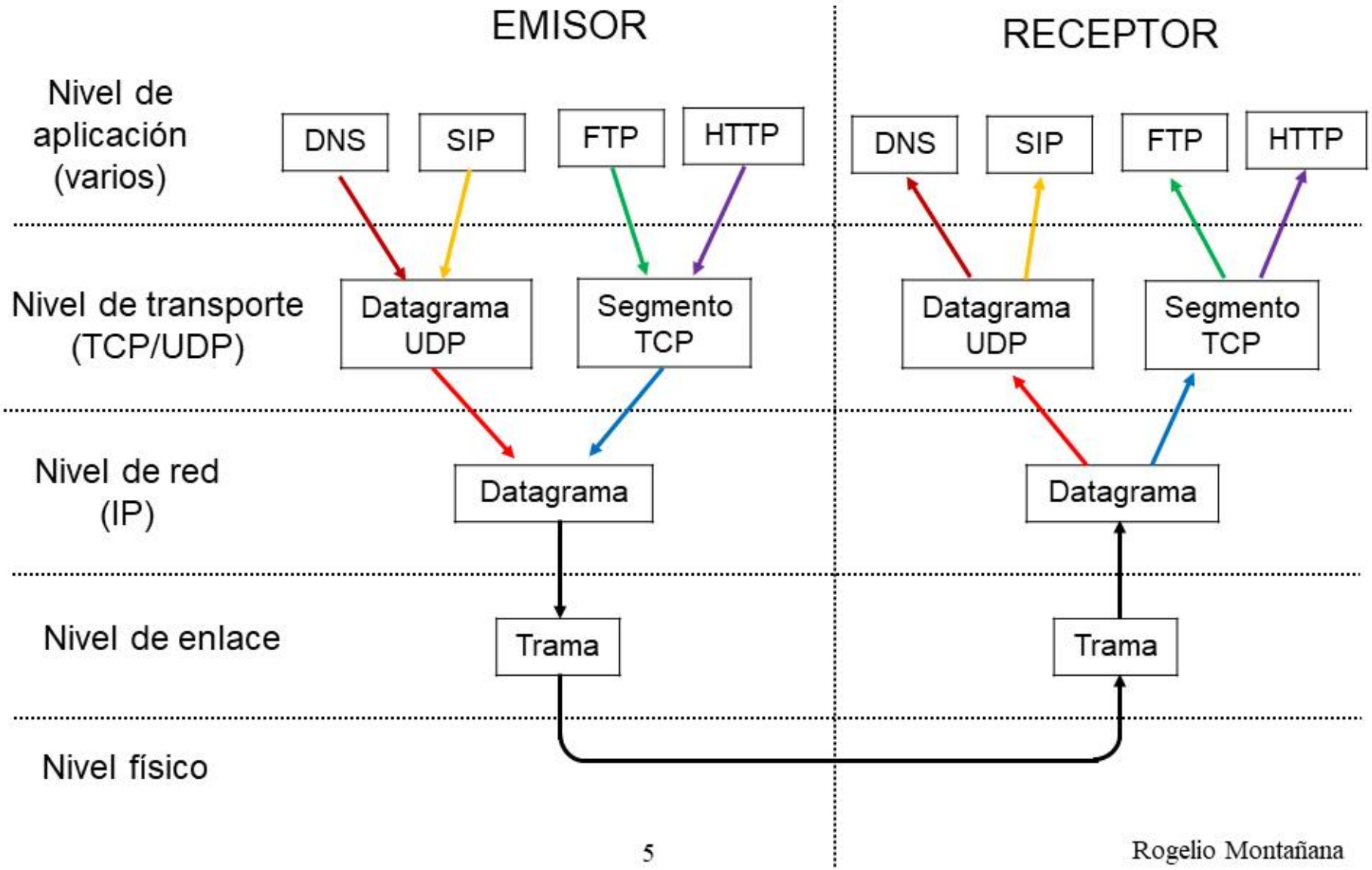
# TCP vs UDP en Internet




Otros: 10%
UDP: 10%
TCP: 80%

Otros: ICMP, OSPF, etc.

# Nivel de transporte en Internet

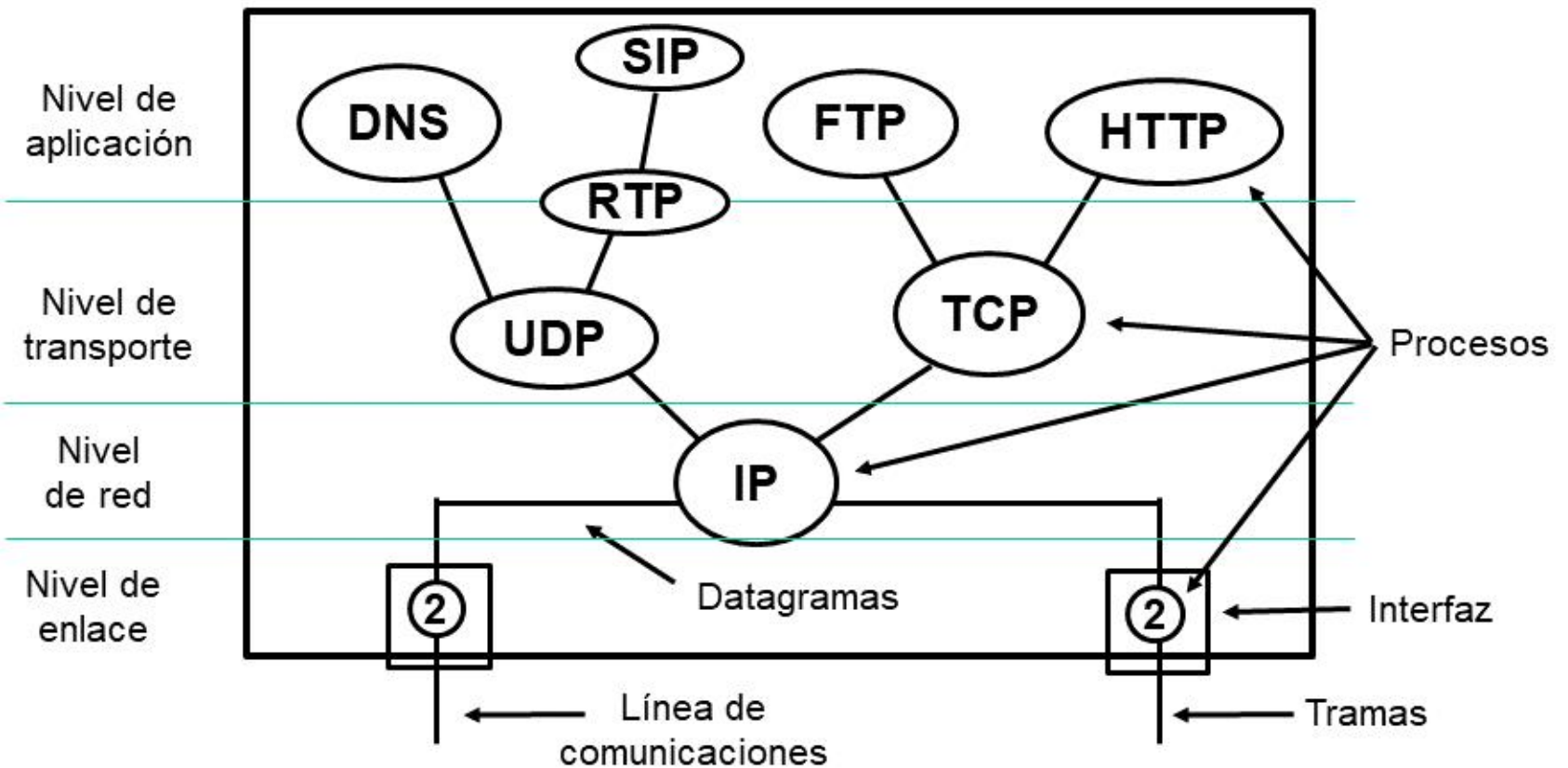


# Desarrollo típico del nivel de transporte

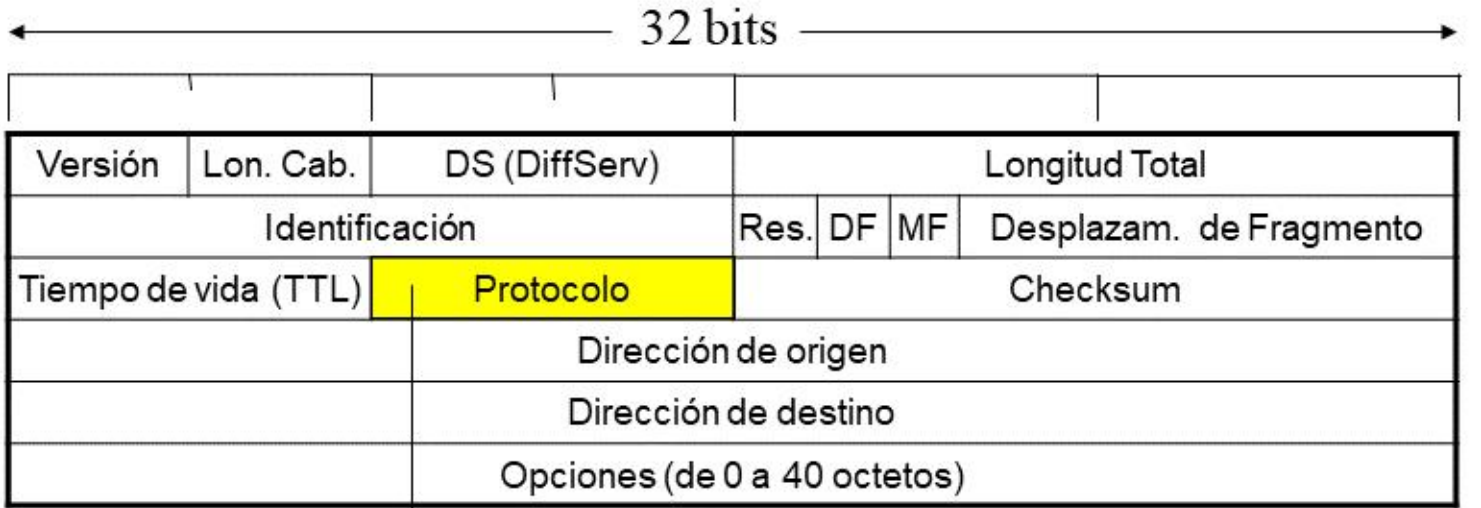
| | | | |
|---|---|---|---|
| Aplicación | DHCP, RIP, DNS, NTP, SNMP, NFS, TFTP | SIP, H.323 (tiempo real) | BGP, HTTP, FTP, IMP, POP, SMTP, SSH, TelNet, DNS |
| | | RTP | |
| Transporte | UDP | | TCP |
| Red | IP | | |

RTP (Real-time Transport Protocol) complementa las funciones que desempeña UDP sin llegar al control que ejerce (y al overhead que supone) TCP

# Arquitectura de red en un host multihomed



# Especificación del protocolo de transporte en la cabecera IP




| Valor | Protocolo |
|---|---|
| 1 | ICMP |
| 4 | IP |
| 6 | TCP |
| 17 | UDP |
| 89 | OSPF |

Esto son solo algunos ejemplos de los valores que puede tener el campo 'protocolo'

# Sumario

- Introducción
- **Protocolo UDP**
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

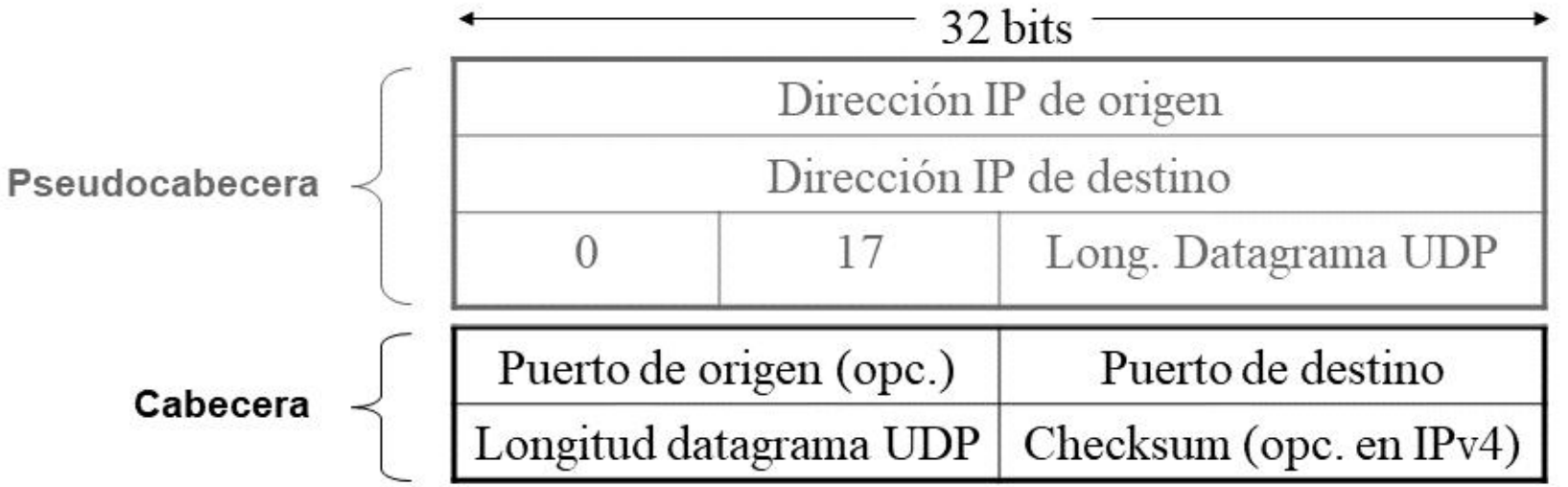
# Protocolo UDP

- Servicio básico 'connectionless' (no fiable)
- Se utiliza cuando:
  - El intercambio de mensajes es muy escaso, ej.: consultas al DNS (servidor de nombres), DHCP
  - Se requiere tiempo real y no se pueden esperar confirmaciones. Ej.: videoconferencia, telefonía.
  - Los mensajes se producen regularmente y no importa si se pierde alguno. Ej: NTP, SNMP
  - Se trata de tráfico broadcast/multicast (este no puede enviarse nunca por TCP)
  - Se quiere hacer una implementación muy sencilla, ej.: TFTP

# Protocolo UDP

- Los paquetes que envía UDP se denominan **mensajes** o **datagramas UDP** (datagrama=connectionless)
- UDP multiplexa los datos de las aplicaciones y opcionalmente comprueba errores, pero no realiza:
  - Conexión/desconexión
  - Retransmisión de datos perdidos
  - Control de flujo
  - Control de congestión

# La cabecera UDP




La pseudocabecera se antepone a la cabecera UDP, pero no se envía realmente (de ahí su nombre). Solo se utiliza para calcular el checksum. Permite al UDP receptor comprobar que el nivel IP no le ha pasado, por error, un datagrama que era para otra dirección.

El checksum comprueba **todo** el datagrama (incluidos los datos), En IPv4 su uso es opcional, si no se calcula se pone todo a ceros y no se comprueba.

# Multiplexación

- UDP asigna a cada proceso del nivel de aplicación un no. de puerto.
- El puerto es una dirección local que se utiliza para la multiplexación.
- Al ser un entero de 16 bits su valor siempre está comprendido entre 0 y 65535
- La combinación dirección_IP-puerto identifica un 'socket' (enchufe o conector) que es el origen o destino de los datagramas UDP. Ej:



# Rangos de puertos

- Los puertos se dividen en tres rangos:
  - Del 0 al 1023 son los puertos **bien conocidos** ('well known ports'). Se reservan para servicios de protocolos estándar (ej.: HTTP, puerto 80). Sólo procesos con acceso superusuario pueden utilizarlos.
  - Del 1024 al 49151: puertos **registrados**. Se usan para clientes y para servicios que no requieren acceso superusuario (ej.: SIP (telefonía IP) puerto 5060).
  - Del 49152 al 65535: puertos **dinámicos** o **privados**. Sólo se usan para clientes.
- La correspondencia puertos-protocolos se puede consultar en: http://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

# Tipos de puertos

- Los puertos se dividen en tres tipos:

| Tipo de puerto | Rango valores | Uso |
|---|---|---|
| Bien conocido | 0-1023 | Servicios de protocolos estándar (ej.: HTTP, puerto 80). Solo procesos con acceso superusuario pueden utilizarlos |
| Registrado | 1024-49151 | Clientes y servicios que no requieren acceso superusuario (ej.: SIP (telefonía IP) puerto 5060) |
| Dinámico o privado | 49152-65535 | Clientes. |

- La correspondencia oficial puertos-protocolos se puede consultar en: http://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

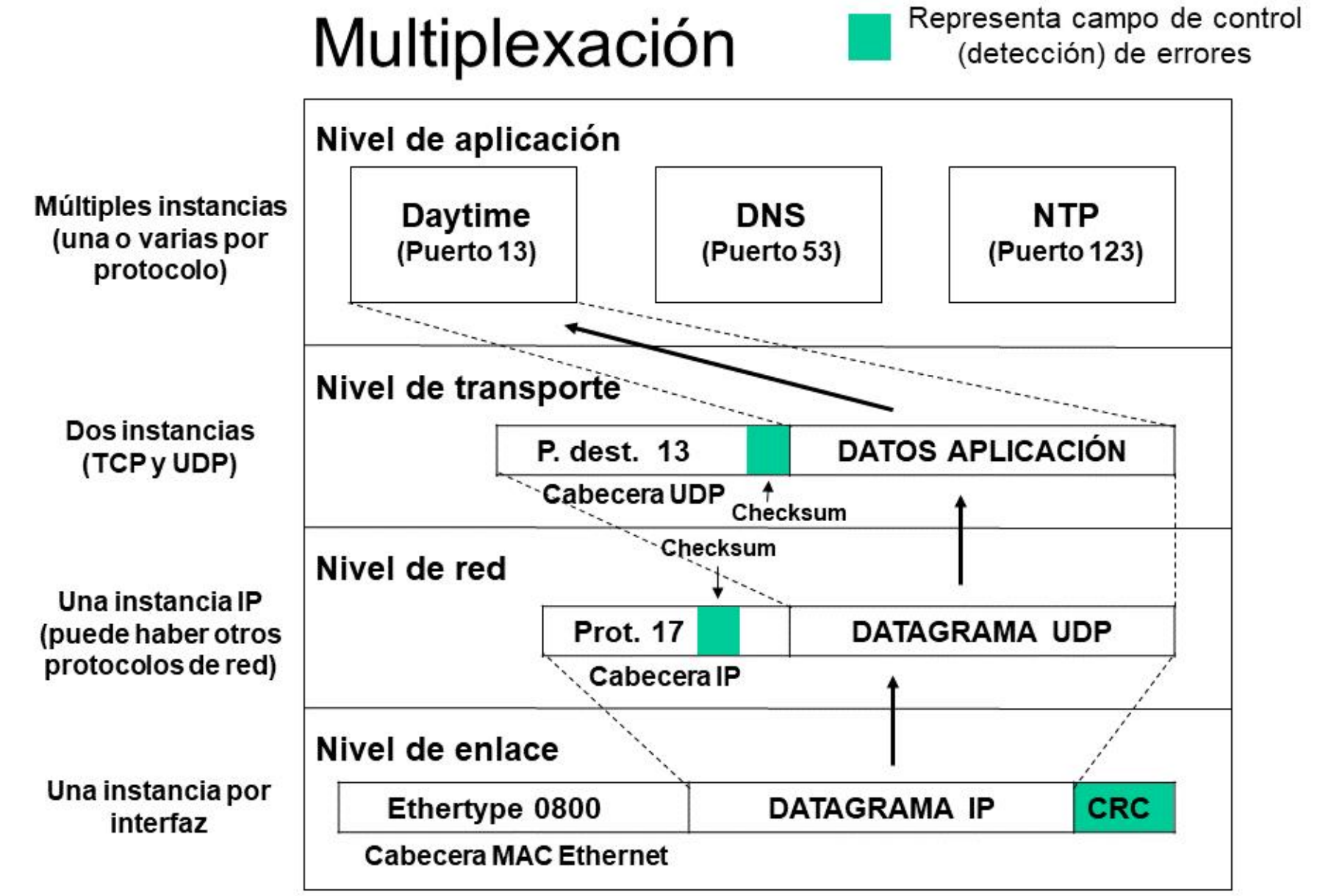
# Algunos ejemplos de puertos 'bien conocidos' y registrados

| Servicio | Puerto | TCP | UDP |
|---|---|---|---|
| Echo | 7 | X | X |
| DayTime | 13 | | X |
| Chargen | 19 | X | X |
| FTP | 21 | X | |
| SSH | 22 | X | |
| TelNet | 23 | X | |
| SMTP | 25 | X | |
| DNS | 53 | X | X |
| BOOTP | 67 | | X |

| Servicio | Puerto | TCP | UDP |
|---|---|---|---|
| TFTP | 69 | | X |
| HTTP | 80 | X | |
| POP3 | 110 | X | |
| NTP | 123 | | X |
| SNMP | 161 | | X |
| LDAP | 389 | | X |
| HTTPS | 443 | X | |
| Oracle DB | 1521 | X | |
| SIP | 5060 | | X |

El puerto se utiliza del mismo modo en TCP. Los valores tienen el mismo rango y mismo significado

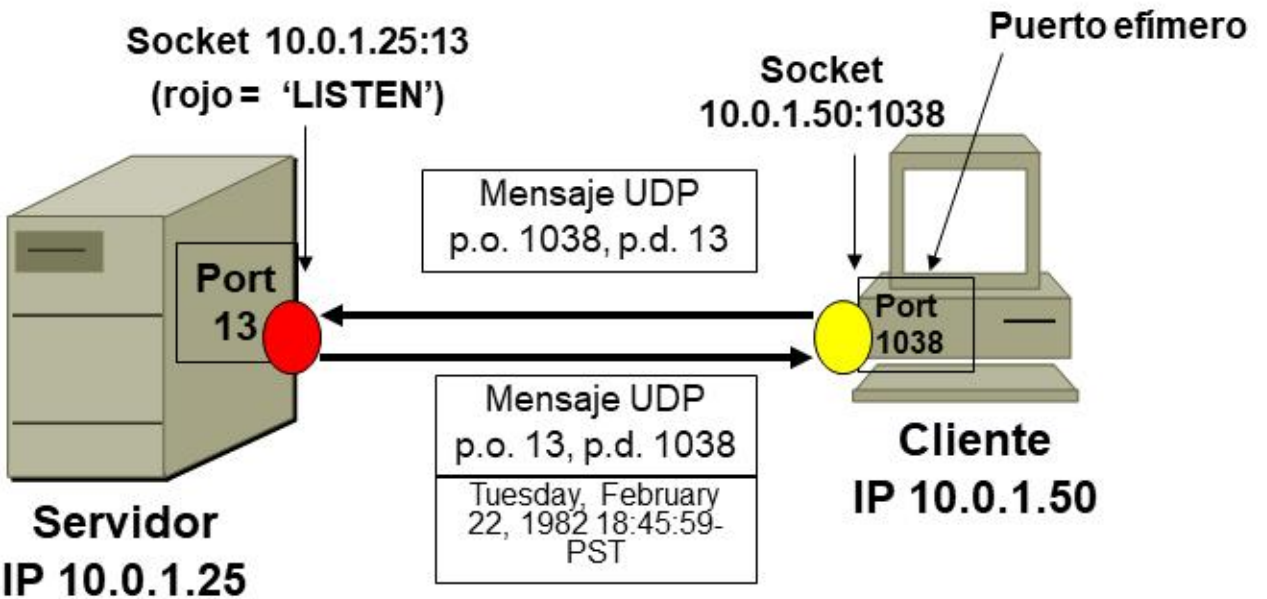
# Multiplexación



# Modelo cliente-servidor

- Para describir los servicios del nivel de transporte y de aplicación se suele utilizar el modelo cliente-servidor:
  - Cliente: el que realiza la petición o consulta. Es el que inicia la conexión
  - Servidor: es el que espera recibir peticiones de clientes
- En el nivel de aplicación algunos protocolos (Emule, Skype, Spotify) utilizan el modelo simétrico o de igual a igual (peer-to-peer) pero a nivel de transporte casi siempre se utiliza el modelo cliente-servidor

# Intercambio de datagramas UDP entre un cliente y un servidor




El puerto 13 corresponde al servicio daytime

**Si el socket del servidor recibe datagramas de múltiples sockets clientes los pondrá en una cola común y los atenderá secuencialmente**

**En UDP el puerto de origen es opcional, pero si se espera recibir una respuesta hay que ponerlo. Normalmente se pone siempre**

# Rango de puertos efímeros

La mayoría de los sistemas eligen los puertos para sus clientes (puertos efímeros) usando solo una parte del rango disponible

| Sistema operativo | Puertos efímeros |
|---|---|
| Windows XP y anteriores | 1025 – 5000 |
| Linux < 2.4<br>FreeBSD < 4.6 y BSD | 1024 – 5000 |
| FreeBSD ≥ 4.6<br>HP-UX<br>Windows Vista y siguientes | 49152 – 65535 |
| Linux ≥ 2.4 | 32768 – 61000 |
| Solaris y AIX | 32768 – 65535 |
| IRIX | 1024 – 65535 |

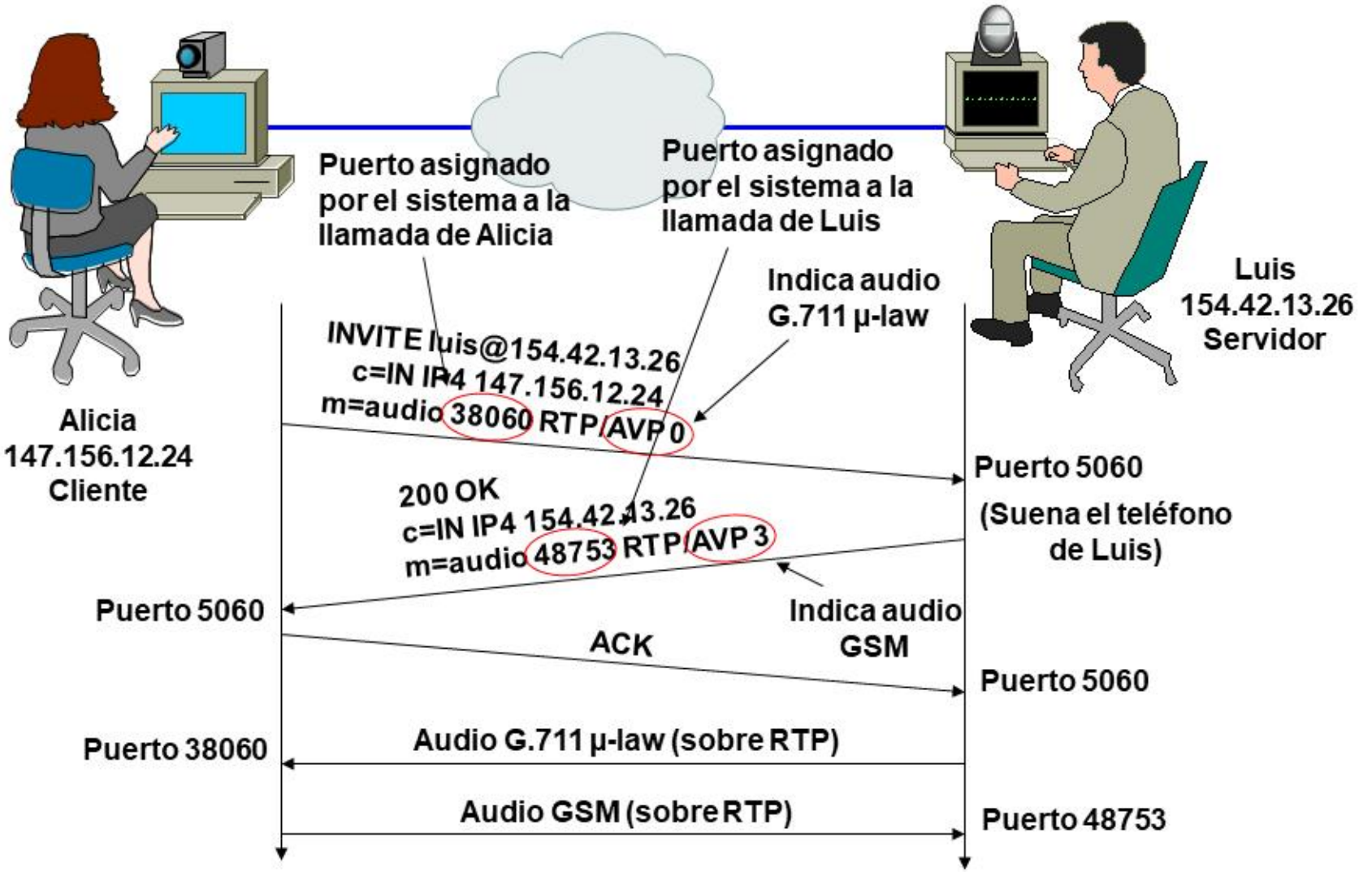
← **Recomendado IANA (rango dinámico)**

# Cabeceras IP y UDP en una petición/respuesta SNMP

```
IP:  ----- IP Header -----
IP:
IP: Version=4, header length=20 bytes
IP: DiffServ = 00
IP: Total length = 131 bytes
IP: Identification = 21066
IP: DF = 0, MF = 0
IP: Fragment offset = 0 bytes
IP: Time to live = 60 seconds/hops
IP: Protocol = 17 (UDP)
IP: Header checksum = 2A13 (correct)
IP: Source address = [128.1.1.1]
IP: Destination address = [128.1.1.10]
IP: No options
IP:
UDP: ----- UDP Header -----
UDP:
UDP: Source Port = 1227
UDP: Destination port = 161 (SNMP)
UDP: Length = 111
UDP: No checksum
UDP:
```

```
IP:  ----- IP Header -----
IP:
IP: Version=4, header length=20 bytes
IP: DiffServ = 00
IP: Total length = 160 bytes
IP: Identification = 2015
IP: DF = 0, MF = 0
IP: Fragment offset = 0 bytes
IP: Time to live = 64 seconds/hops
IP: Protocol = 17 (UDP)
IP: Header checksum = 7061 (correct)
IP: Source address = [128.1.1.10]
IP: Destination address = [128.1.1.1]
IP: No options
IP:
UDP: ----- UDP Header -----
UDP:
UDP: Source Port = 161 (SNMP)
UDP: Destination port = 1227
UDP: Length = 140
UDP: Checksum = 4D4F (correct)
UDP:
```

# Llamada SIP entre dos usuarios

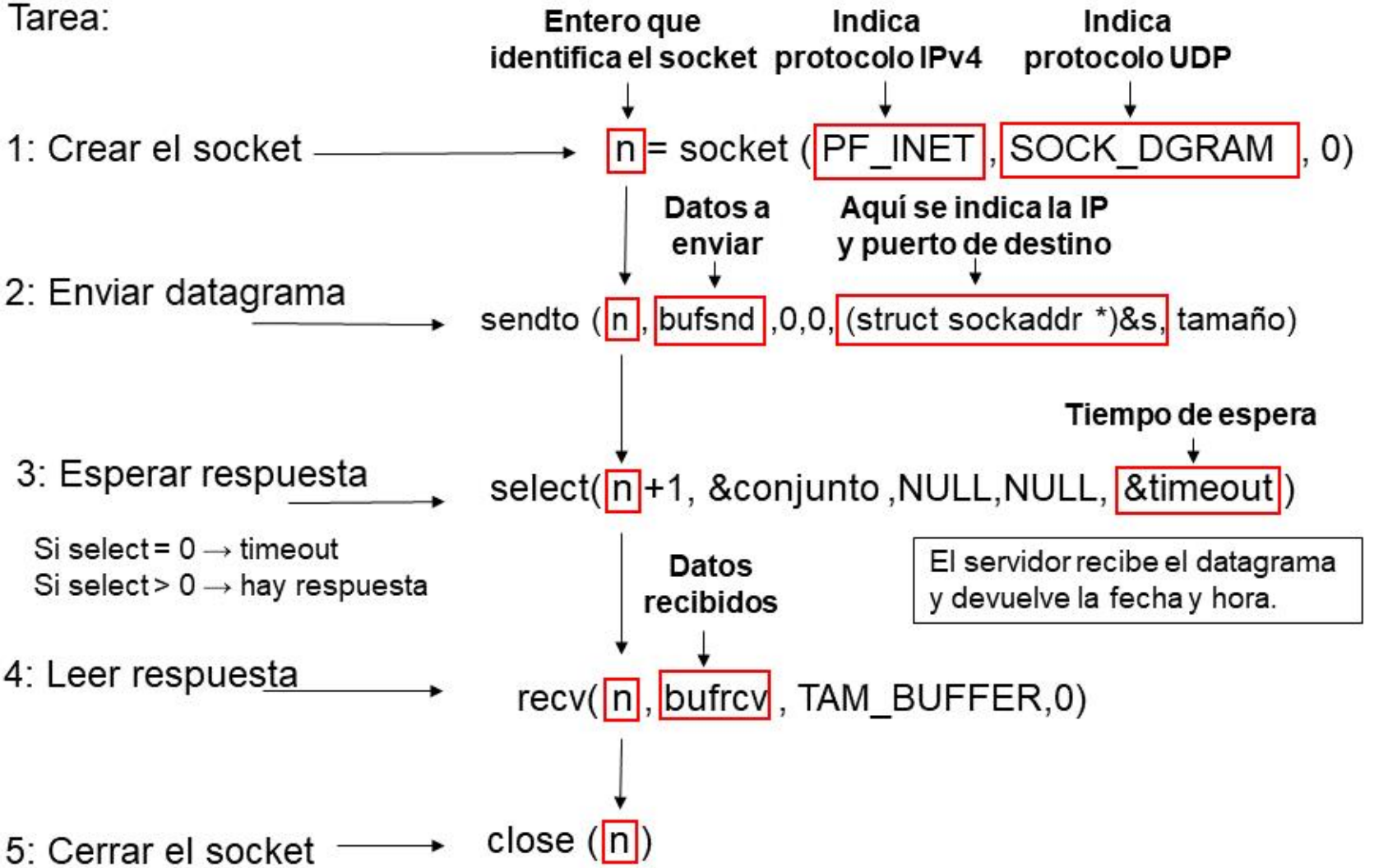


# Programación de aplicaciones

- Normalmente se utilizan APIs (Application Programming Interface) estandarizadas denominadas 'sockets'.
- La más extendida son los 'sockets de Berkeley', librería de funciones en C incluída en el UNIX BSD 4.2 en 1983 y posteriormente en el estándar POSIX. Hay versiones para casi todos los S.O., incluido Windows (winsock)
- Estas librerías simplifican las tareas más habituales y permiten al programador escribir código portable sin preocuparse de la arquitectura del host
- La estandarización de APIs se hace por portabilidad, no por interoperabilidad. Lo único que intercambian las aplicaciones son paquetes UDP o TCP

# Cliente UDP con Berkeley sockets

Tarea:



# ¿Cuándo y como se envían los datagramas UDP?

- Se envían:
  - Cada vez que la aplicación invoca un 'sendto' o similar el host manda el contenido del buffer en un datagrama IP al socket de destino.
  - Si el datagrama a enviar no cabe en la trama a nivel de enlace el nivel IP se encarga de fragmentarlo. UDP no se entera de nada
- Se reciben:
  - En cuanto llega el datagrama IP pasa su contenido a UDP y notifica a la aplicación. Ésta lo recoge cuando quiere invocando un 'recv' o similar
  - Si el datagrama llegó fragmentado el IP del receptor lo reensambla antes de pasarlo a UDP.

# Ataque flood (inundación) de UDP

- Cuando un host recibe un datagrama UDP, si el puerto esta abierto ha de entregar el paquete al proceso correspondiente; si no lo está debería responder al emisor con un mensaje ICMP Destino inaccesible
- Esto permite hacer ataques de denegación de servicio, central o distribuida, enviando muchos mensajes UDP por segundo a puertos cerrados en un host.
- Para evitarlo muchos sistemas omiten el envío de mensajes ICMP Destino inaccesible en estos casos

# Sumario

- Introducción
- Protocolo UDP
- **Protocolo TCP**
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

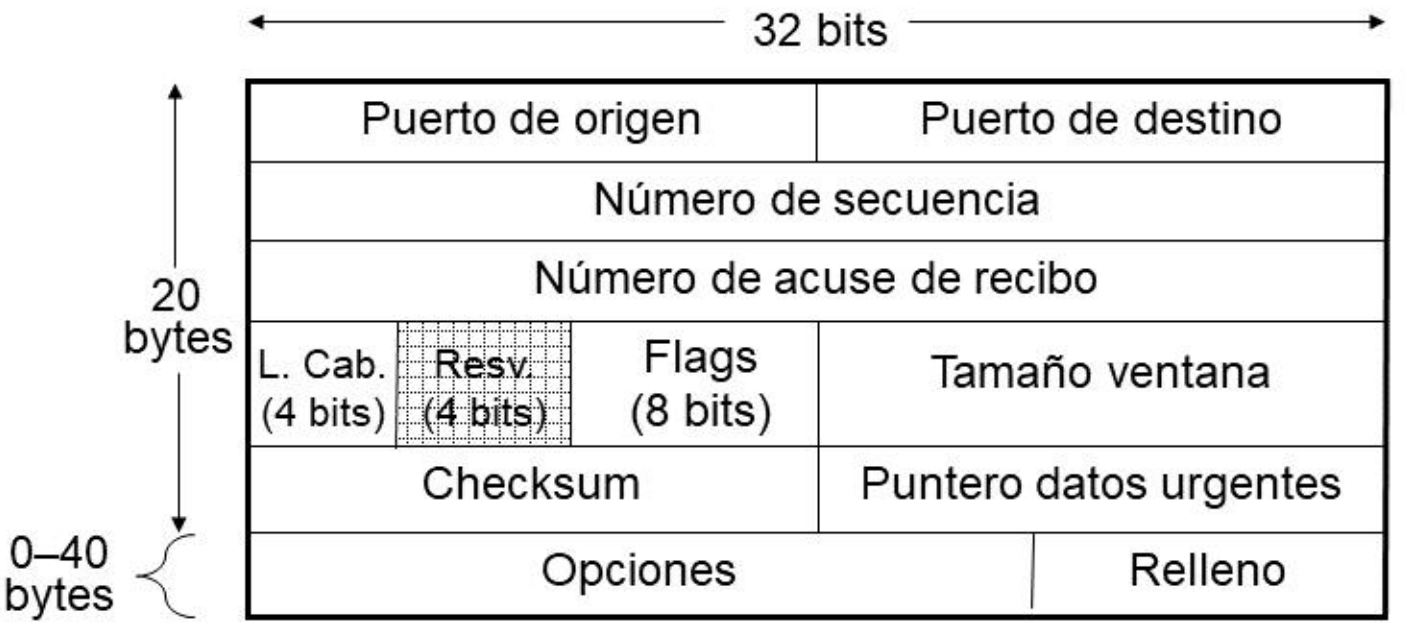
# TCP (Transmission Control Protocol)

- Está diseñado para ofrecer un transporte fiable sobre el servicio no fiable que le suministra IP
- Es un servicio orientado a conexión
- La conexión siempre se establece entre dos y sólo dos TCPs. En TCP no existe la posibilidad de envíos multicast o broadcast, en esos casos se usa siempre UDP
- Los paquetes de TCP se llaman **segmentos**.
- El TCP actual (v4) se especificó en el RFC 793 en 1981 y con pocos cambios sigue plenamente vigente.

# Funciones de TCP

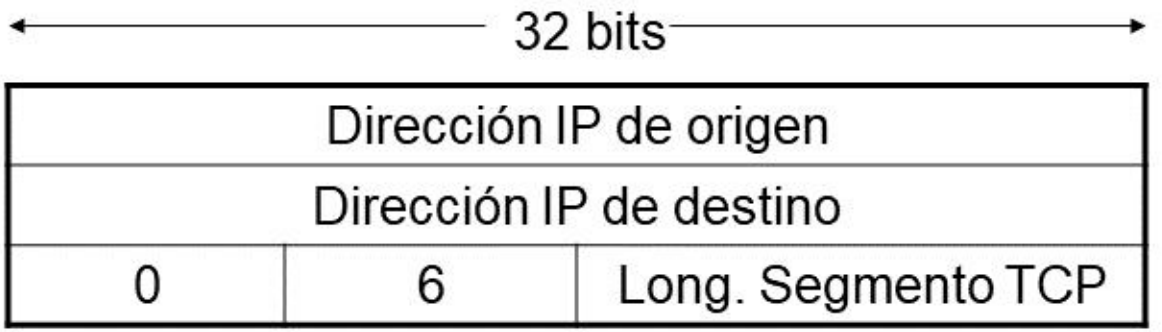
- Multiplexar el nivel de aplicación (puerto)
- Establecer y terminar conexiones
- Controlar errores, retransmitir segmentos perdidos o erróneos. Eliminar duplicados
- Gestionar los buffers y ejercer control de flujo de forma eficiente
- Organizar el envío de datos para hacer un uso eficiente de los recursos
- Efectuar control de congestión

# La cabecera TCP




| Flags: | | | |
|---|---|---|---|
| | 1 | CWR: | Congestion Window Reduced |
| | 2 | ECE: | ECN Echo (ECN=Explicit Congestion Notification) |
| | **3** | **URG:** | **el segmento contiene datos urgentes** |
| | **4** | **ACK:** | **el campo número de acuse de recibo tiene sentido** |
| | **5** | **PSH:** | **el segmento contiene datos 'Pushed'** |
| | **6** | **RST:** | **ha habido algún error y la conexión debe cerrarse** |
| | **7** | **SYN:** | **indica el inicio de una conexión** |
| | **8** | **FIN:** | **indica el final de una conexión** |

# La pseudocabecera TCP




La pseudocabecera se antepone a la cabecera TCP, pero no se envía, solo se utiliza para calcular el checksum. Permite a TCP comprobar que IP no se ha equivocado entregándole un segmento que no era para él.

El valor 6 indica el protocolo de transporte (TCP)

# Pregunta: tamaño de segmentos

P: El tamaño máximo de un segmento TCP es 65.515 octetos. ¿Podría explicar de donde viene ese valor?

R: La longitud máxima de un datagrama se especifica en un campo de 16 bits, por lo que es 65535 octetos. De estos al menos 20 son la cabecera IP, con lo que quedan 65515 octetos para el segmento TCP.

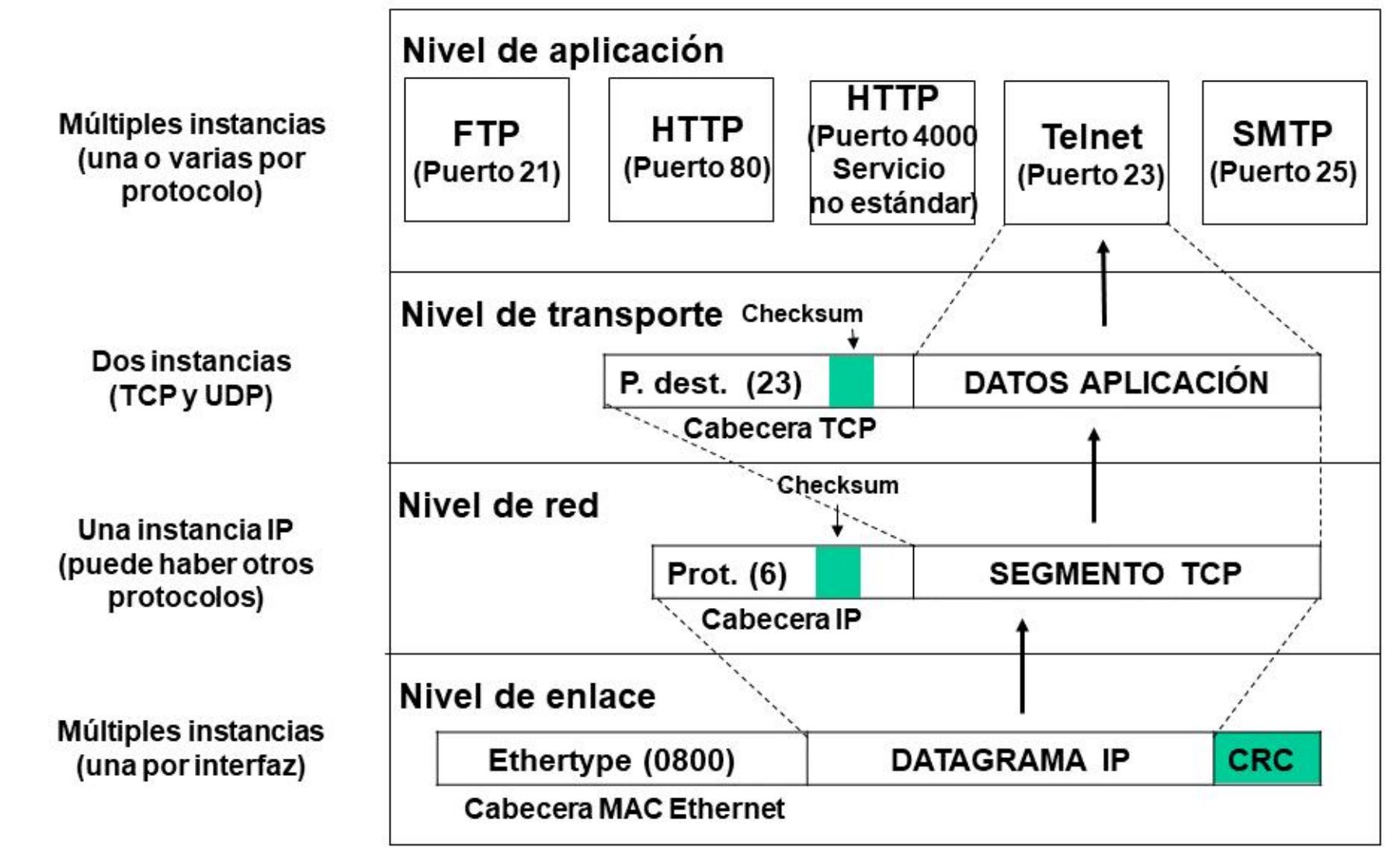
# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - **Multiplexación**
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

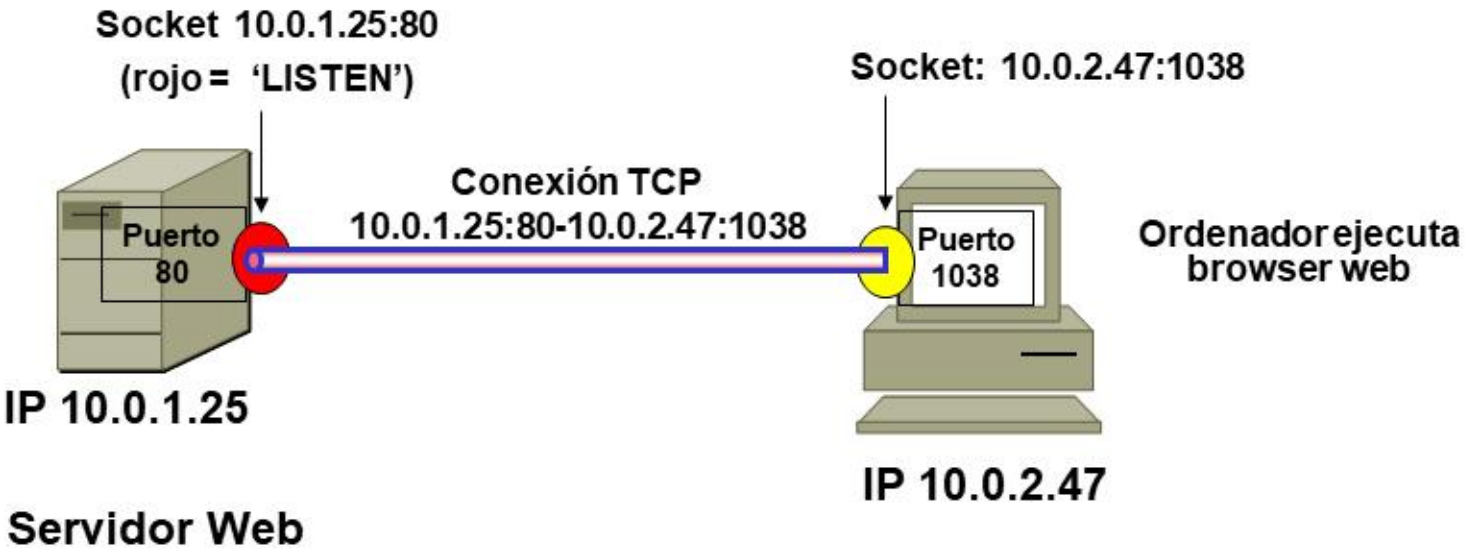
# Multiplexación

- Se utiliza el número de puerto (origen y destino), igual que en UDP. Puede valer de 0 a 65535.
- Como en UDP la combinación dirección_IP-puerto identifica un 'socket': **147.156.1.43:80**
- Una conexión TCP queda especificada de forma no ambigüa por los dos sockets que se conectan:
  IP-puerto ↔ IP-puerto
- No pueden coexistir dos conexiones TCP que coincidan en los cuatro valores: IP-puerto ↔ IP-puerto. En ese caso serían la misma conexión
- Los puertos se clasifican en bien conocidos, registrados y dinámicos (igual que en UDP)

# Multiplexación

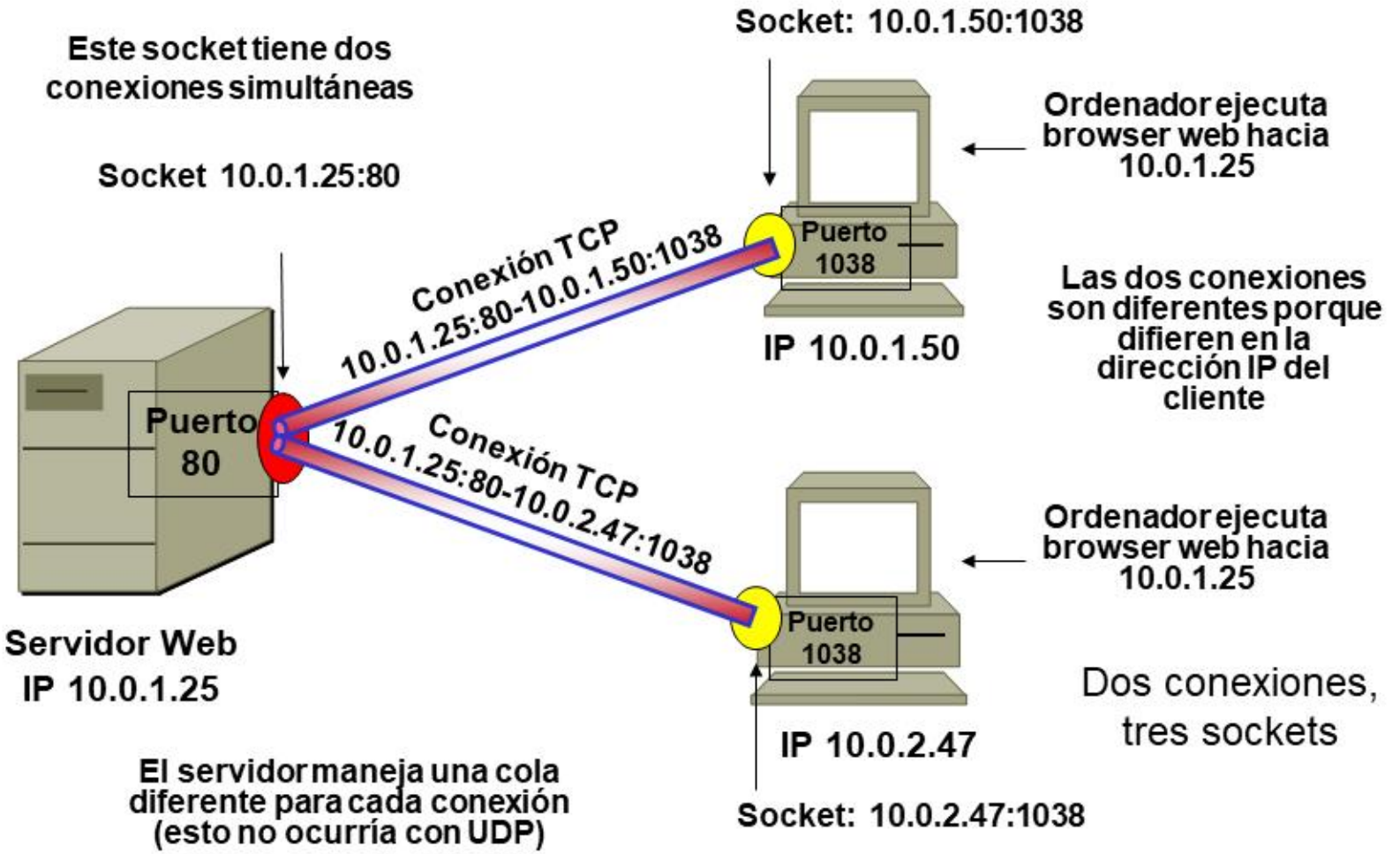


# Conexión de un cliente a un servidor

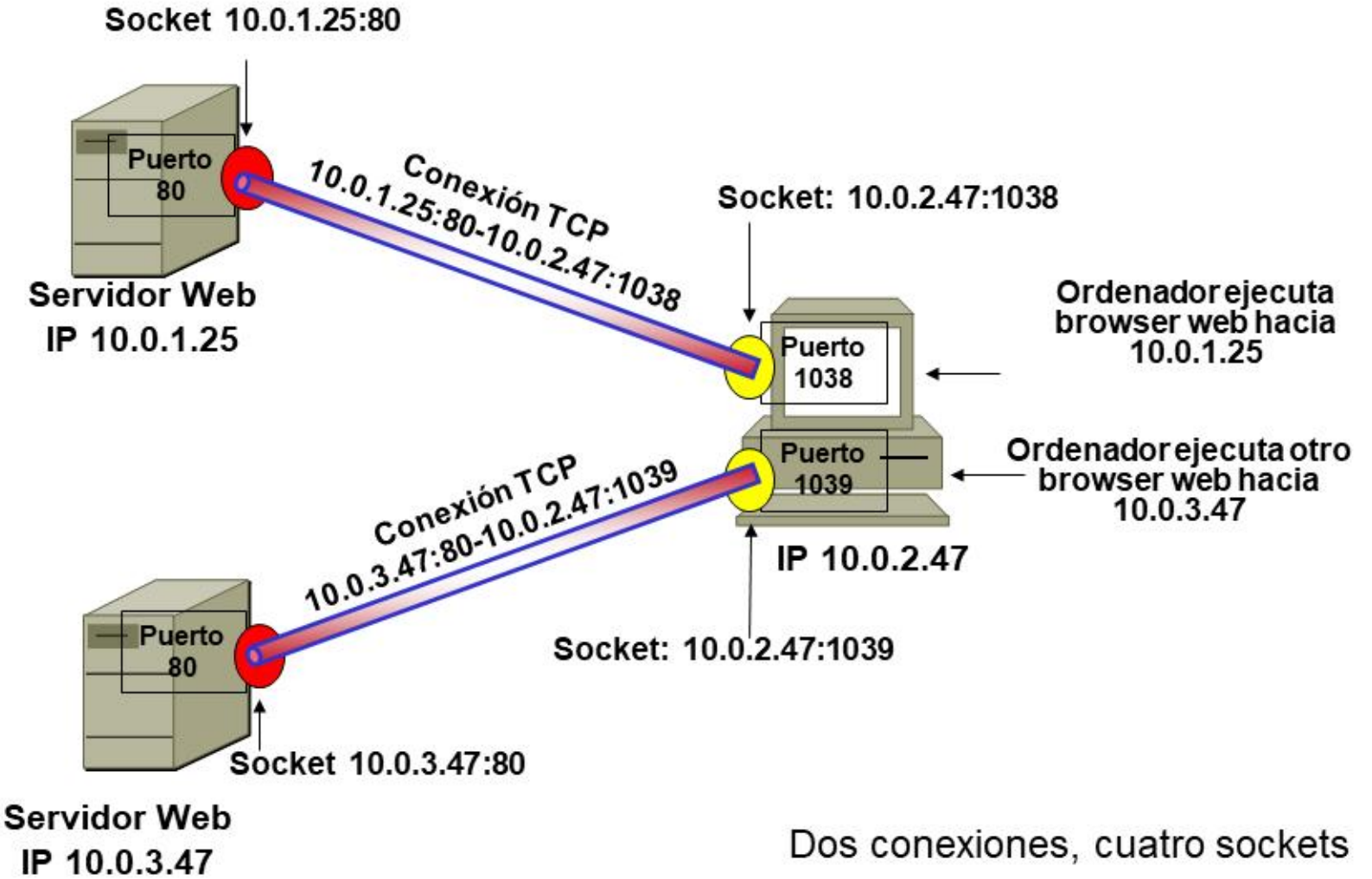



Una conexión, dos sockets

# Dos clientes hacia un mismo servidor

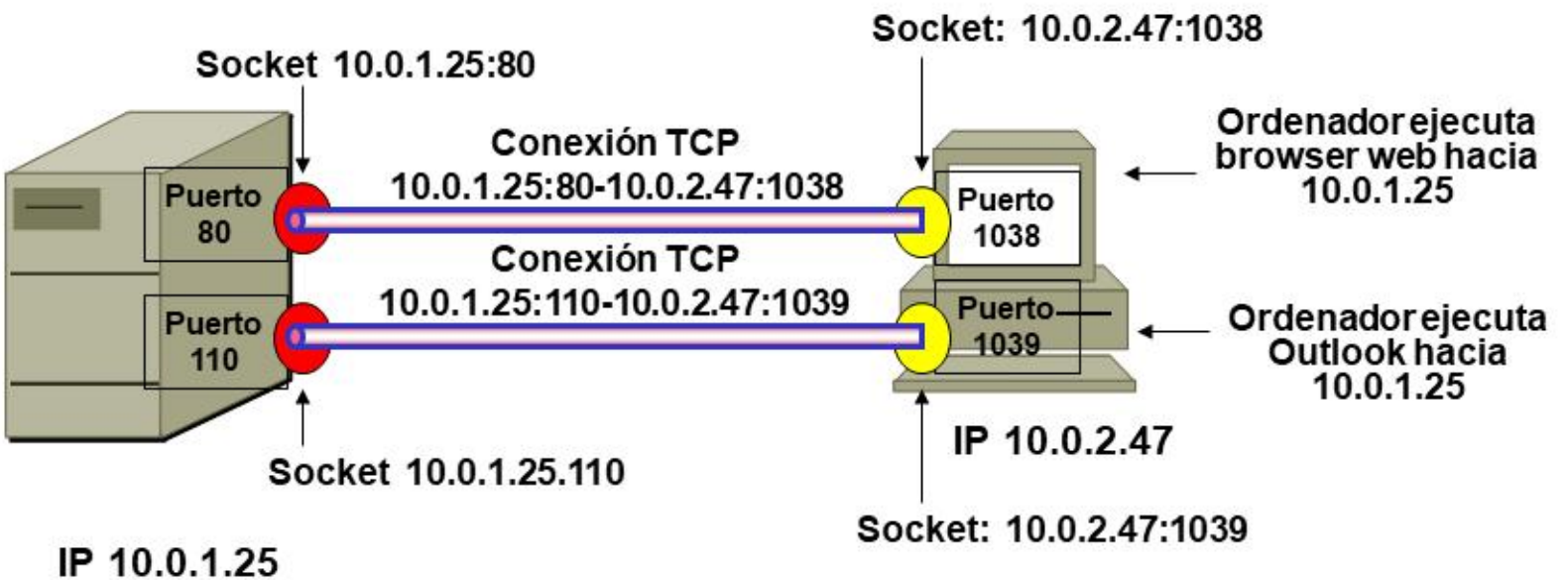


# Dos clientes (misma IP) hacia dos servidores




Dos conexiones, cuatro sockets

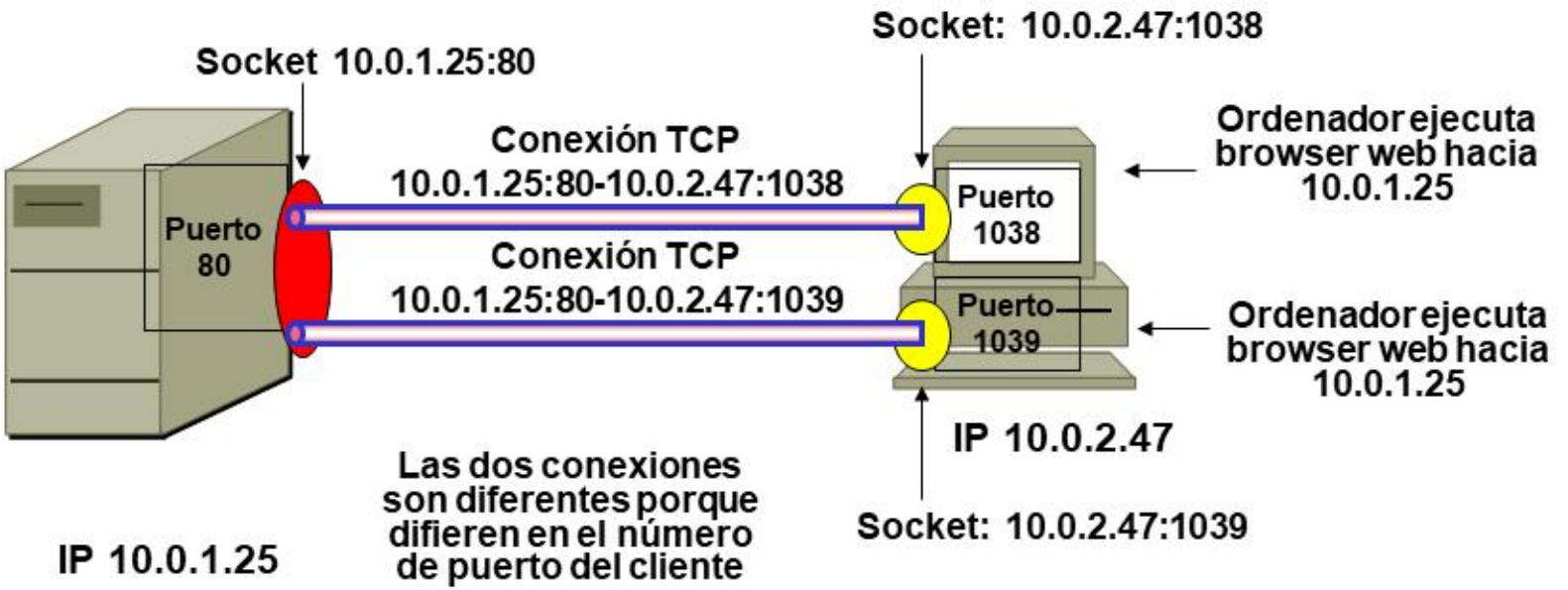
# Dos clientes (misma IP) hacia dos servidores (misma IP)




**Servidor Web y POP3**

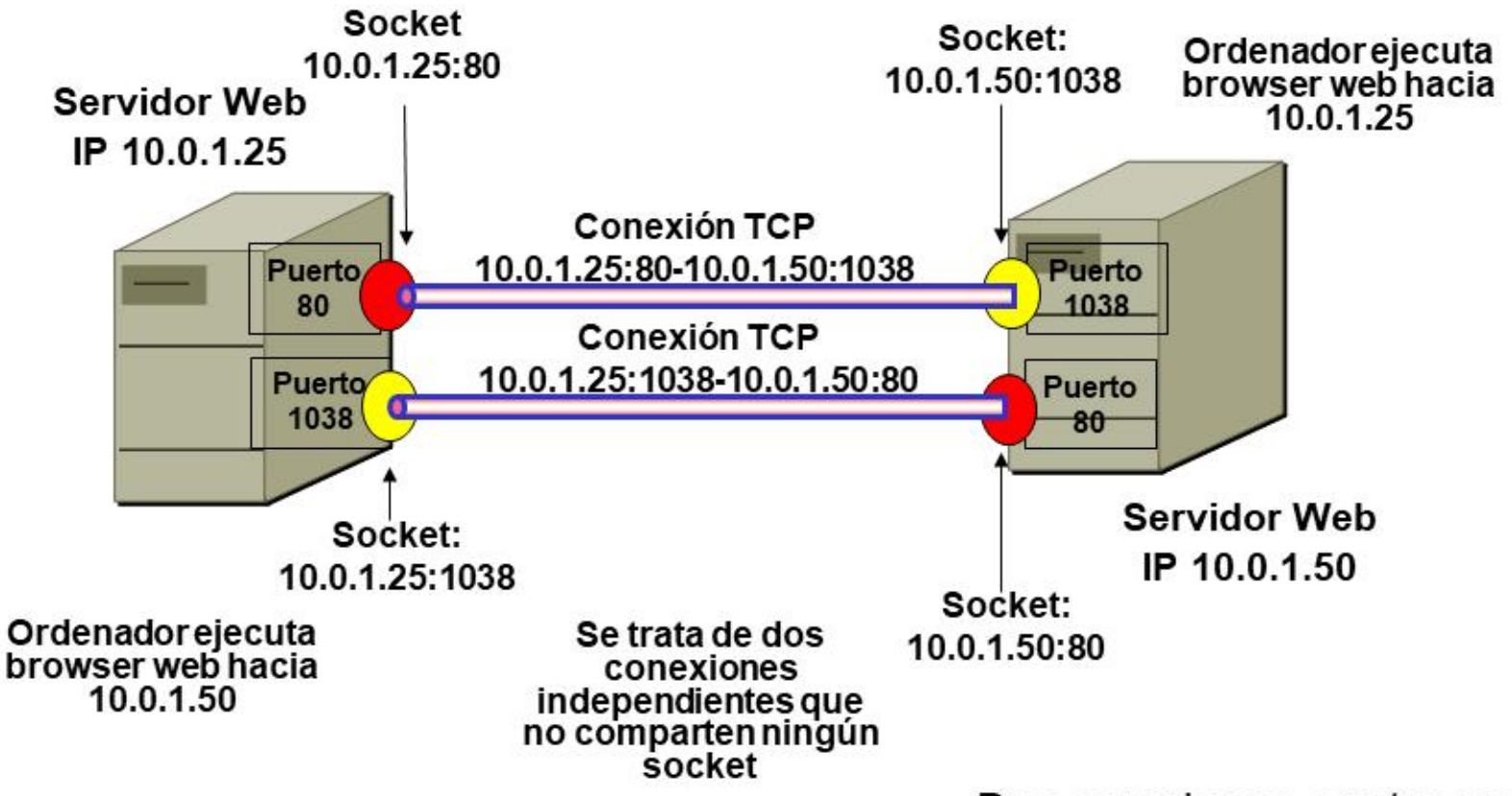
Dos conexiones, cuatro sockets

# Dos clientes (misma IP) hacia el mismo servidor




Dos conexiones, tres sockets

# Conexiones cruzadas cliente-servidor

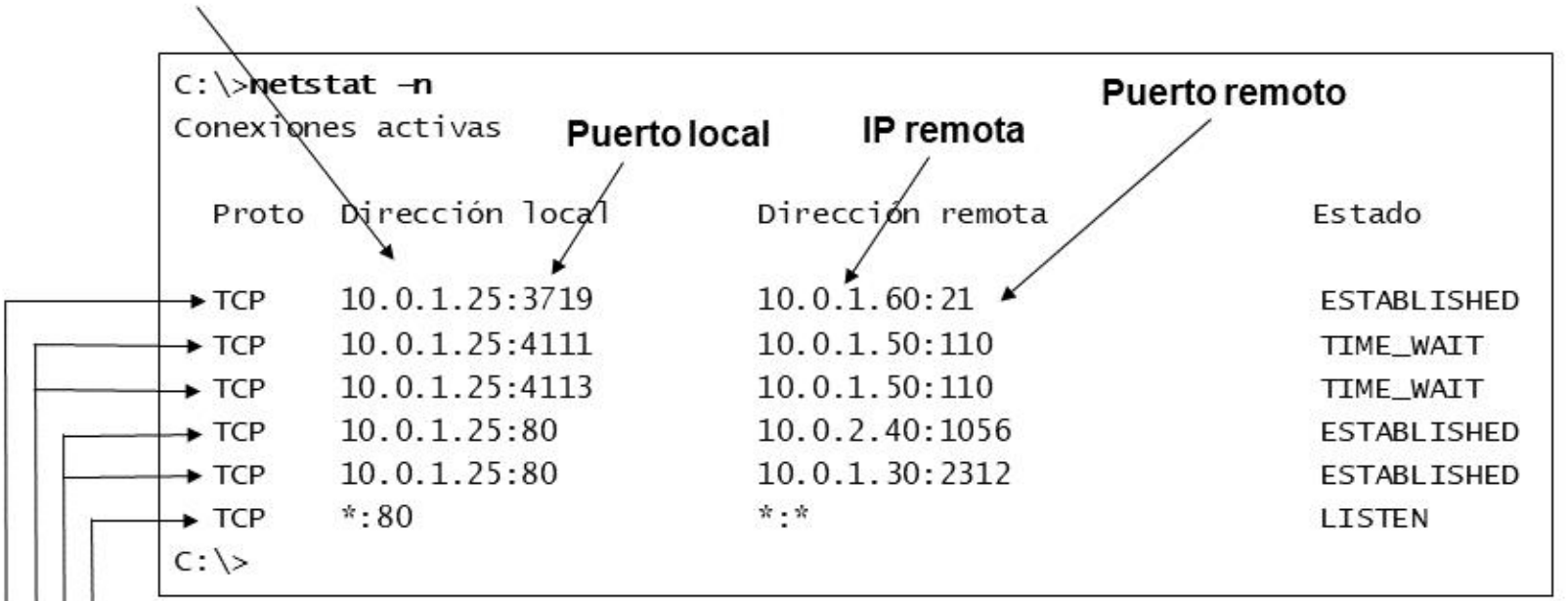



Dos conexiones, cuatro sockets

# Comando ‘netstat’

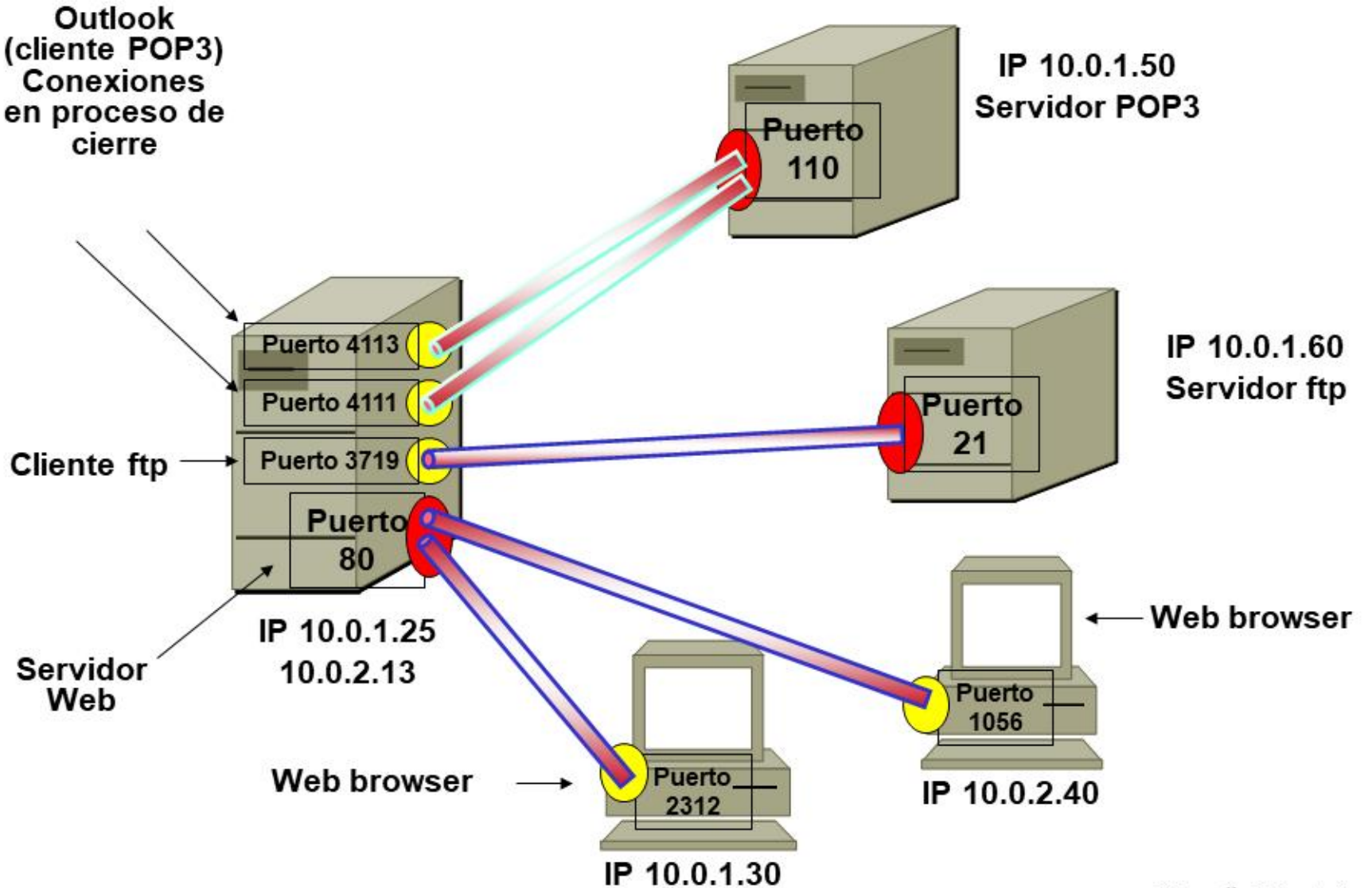
- Permite saber que conexiones TCP hay establecidas y que sockets las forman en el extremo local y remoto.
- También permite averiguar que puertos TCP y UDP están abiertos (en modo ‘LISTEN’)
- Una forma típica de protección de los cortafuegos es bloquear puertos innecesarios, es decir no dejar pasar paquetes cuyo número de puerto de destino no sea alguno de los servicios oficialmente abiertos

# Comando 'netstat' en un host




Si no se utiliza la opción '–n' el programa netstat intenta convertir las direcciones IP y los puertos a nombres siempre que puede (por ejemplo pone 'pop3' en vez de 110)

# Conexiones del netstat anterior



# Sumario

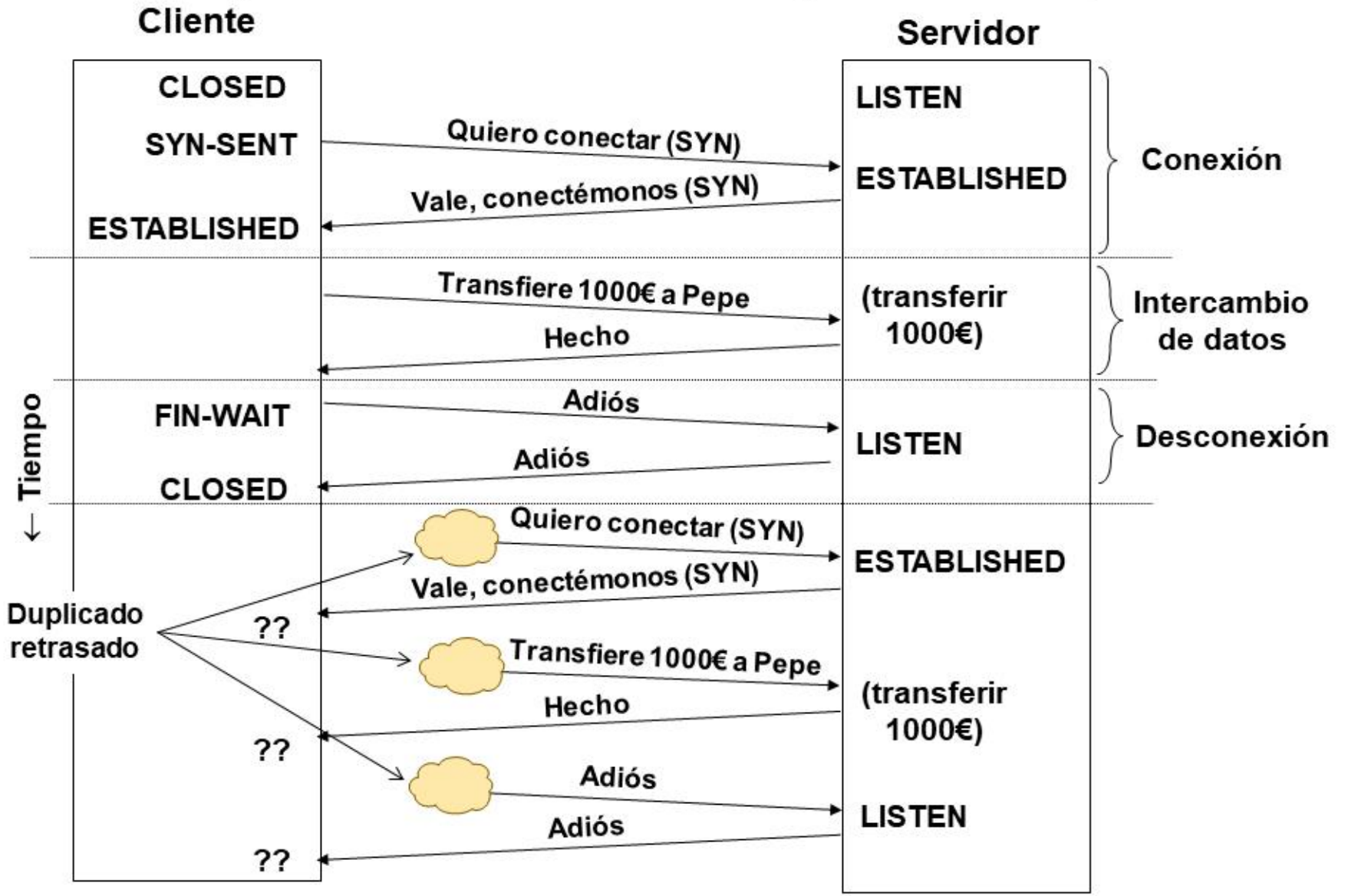
- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - **Conexión**
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

# Flag de conexión de TCP

| Puerto de Origen | | | Puerto de destino |
|---|---|---|---|
| Número de Secuencia | | | |
| Número de ACK | | | |
| L. Cab | Res. | Flags | Tamaño Ventana |
| Checksum | | | Punt. datos urgentes |

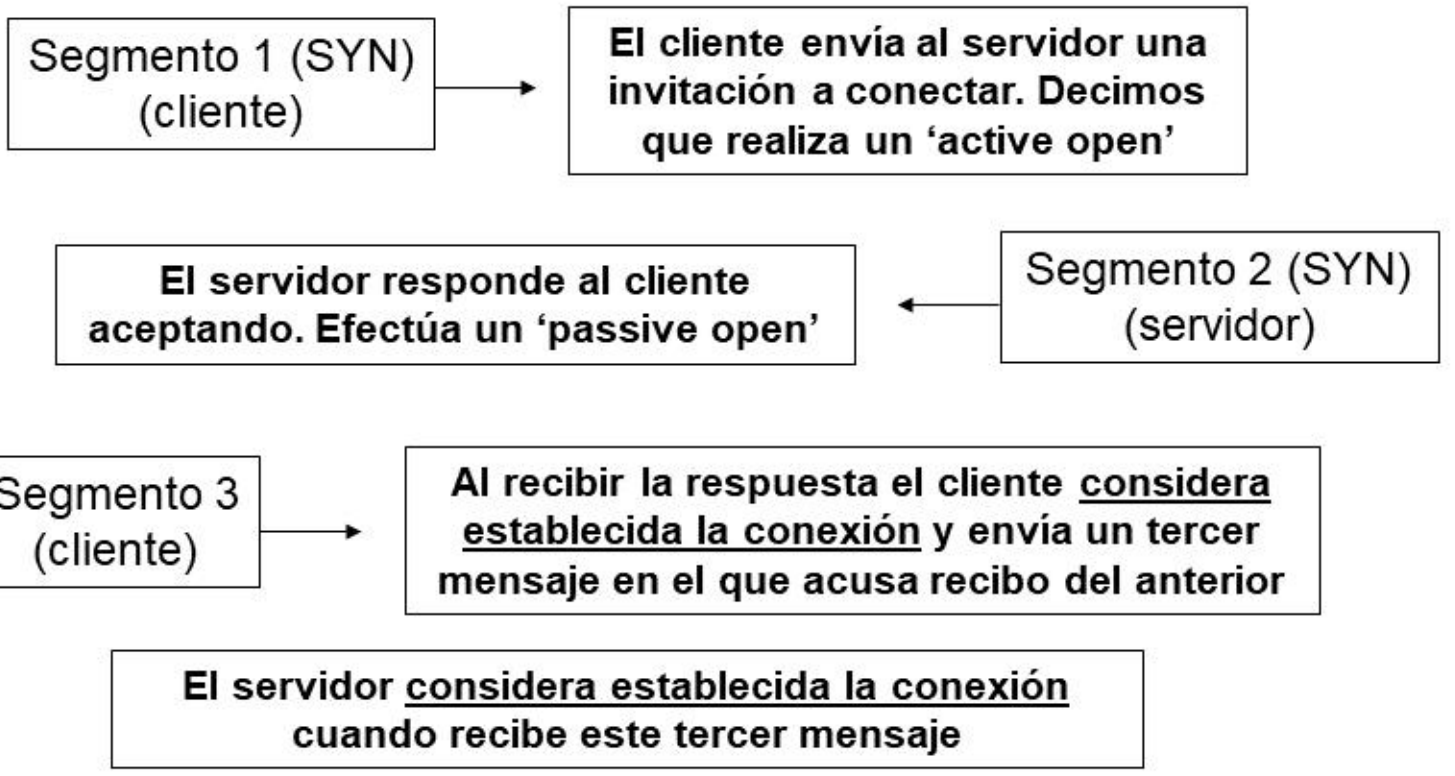
- El flag **SYN** (Synchronize) indica que se está estableciendo una conexión. Está puesto siempre en el primer segmento que se envía desde cada lado de una conexión TCP, y sólo en ese.
- Pero el solo uso de este flag no evita los problemas que pueden producir los duplicados retrasados

# Un protocolo de transporte simple



# Conexión por 'Saludo a tres vías'

- En TCP la conexión se establece por el intercambio de tres mensajes, por lo que se le conoce como **saludo a tres vías** ('**three way handshake**' en inglés):

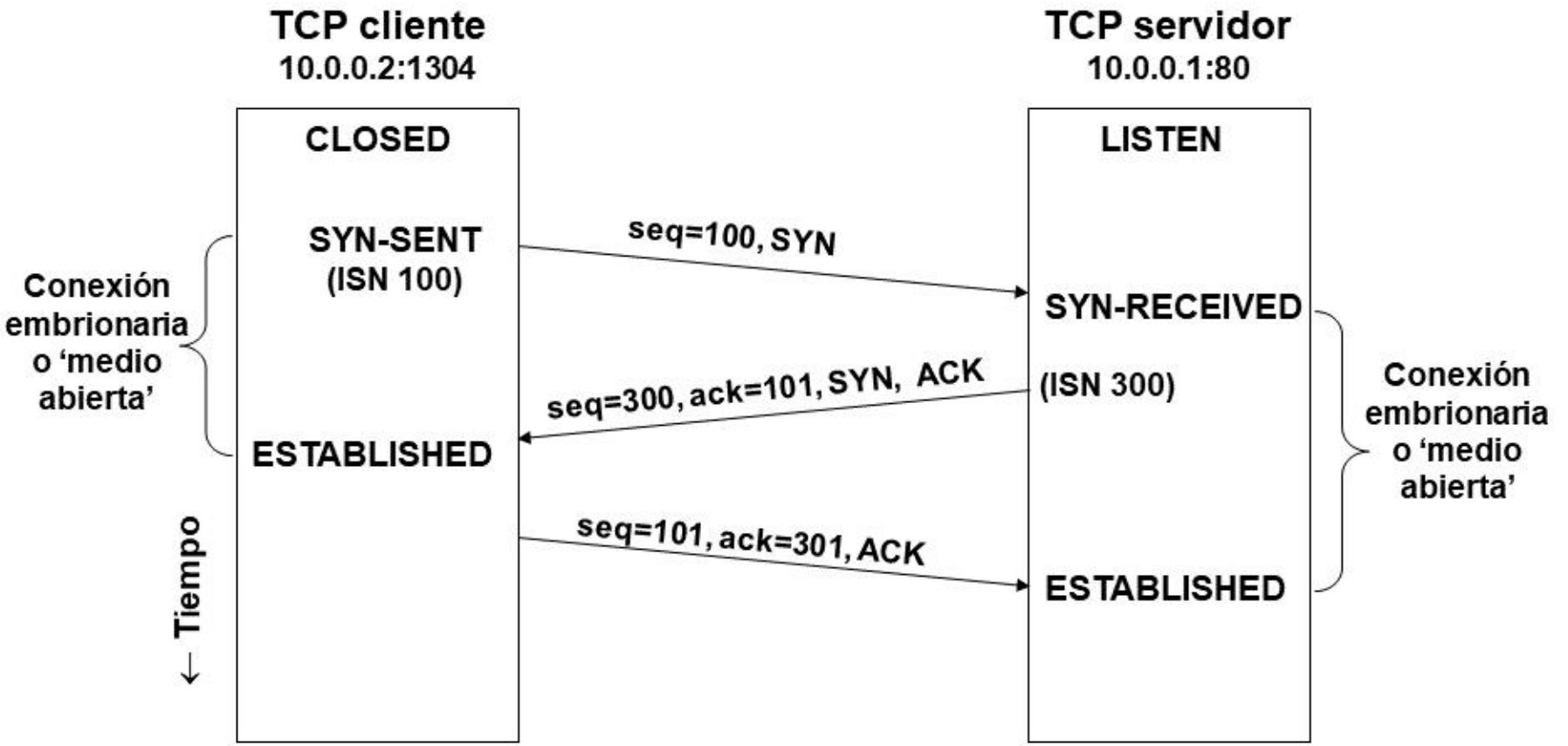


# Identificadores de conexión (ISN)




- Cada conexión TCP que se establece elige un valor diferente del número de secuencia inicial o ISN (Initial Sequence Number), que normalmente no es cero.
- Este ISN actúa como identificador de la conexión y minimiza el riesgo de que un duplicado retrasado (de una conexión anterior) provoque una conexión espúrea.
- Siempre hay dos ISN asociados con una conexión TCP, uno por cada 'lado'. Cada TCP elige independientemente el ISN que utilizará para la conexión.
- El ISN es un entero de 32 bits, sin signo.

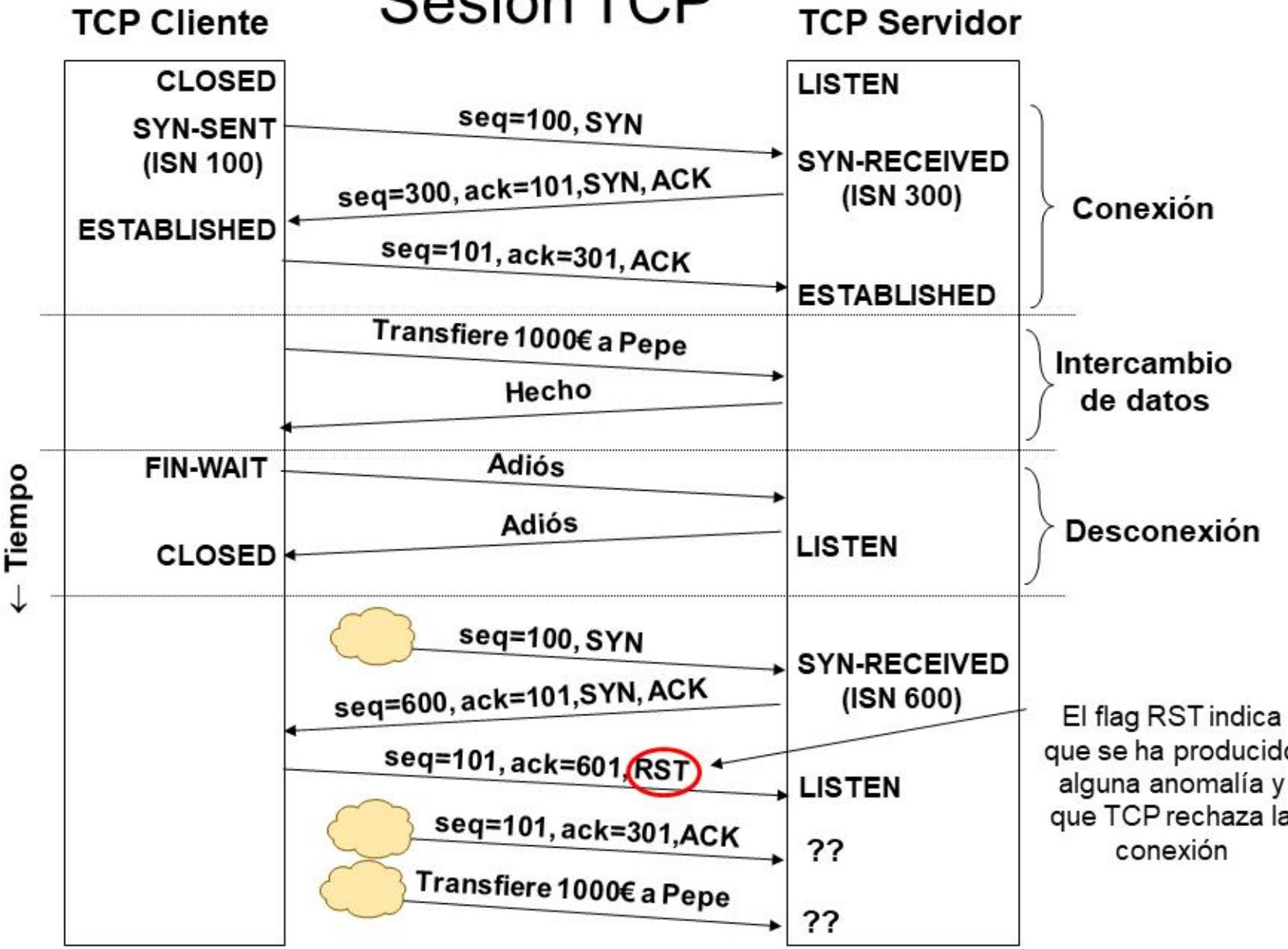
# Establecimiento de una conexión TCP por saludo a tres vías



# Elección del ISN

- Según el RFC 793 el proceso TCP debe mantener un contador que incrementará en 1 cada 4 microsegundos, y cuyo valor se usará como ISN para cada nueva conexión. Así los valores se repiten al cabo de unas 4 horas, tiempo más que suficiente para que los posibles duplicados retrasados que utilicen un mismo ISN ya hayan desaparecido.
- Para gastar menos CPU en la práctica se suele utilizar algoritmos más sencillos. Por ejemplo UNIX BSD 4.4 incrementa el ISN en 64000 cada medio segundo y cada vez que establece una conexión. Así dos conexiones consecutivas tendrán ISN bastante diferentes, aunque ocurran muy próximas en el tiempo.

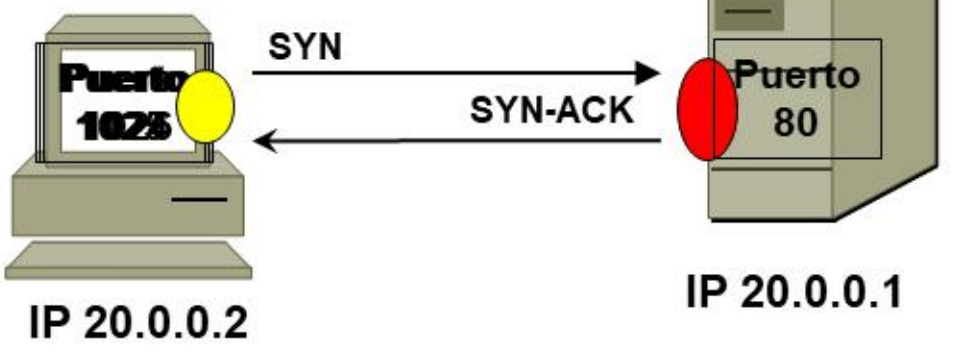
# Sesión TCP


TCP Cliente
TCP Servidor
CLOSED
SYN-SENT
(ISN 100)
ESTABLISHED
LISTEN
SYN-RECEIVED
(ISN 300)
ESTABLISHED
seq=100, SYN
seq=300, ack=101,SYN, ACK
seq=101, ack=301, ACK
Conexión
Transfiere 1000€ a Pepe
Hecho
Intercambio
de datos
FIN-WAIT
CLOSED
Adiós
Adiós
LISTEN
Desconexión
← Tiempo
seq=100, SYN
SYN-RECEIVED
(ISN 600)
seq=600, ack=101,SYN, ACK
seq=101, ack=601, RST
LISTEN
El flag RST indica
que se ha producido
alguna anomalía y
que TCP rechaza la
conexión
seq=101, ack=301,ACK
??
Transfiere 1000€ a Pepe
??

# Envíos TCP a puertos cerrados o inactivos

- Si un host recibe una petición de conexión TCP dirigida a un puerto que no está en modo 'listen' puede:
  - Devolver un mensaje 'ICMP Destino Inaccesible' (Windows)
  - Devolver un mensaje RST rechazando la conexión (UNIX)
  - No decir nada. Esta es una medida de seguridad que adoptan algunos cortafuegos

# Ataque SYN-flood de TCP




**Socket 20.0.0.1:80**
**Conexiones SYN-RECEIVED**
**(embrionarias)**

**20.0.0.2:1024-20.0.0.1:80**
**20.0.0.2:1025-20.0.0.1:80**
**20.0.0.2:1026-20.0.0.1:80**

. . . . . .

**Recursos agotados**

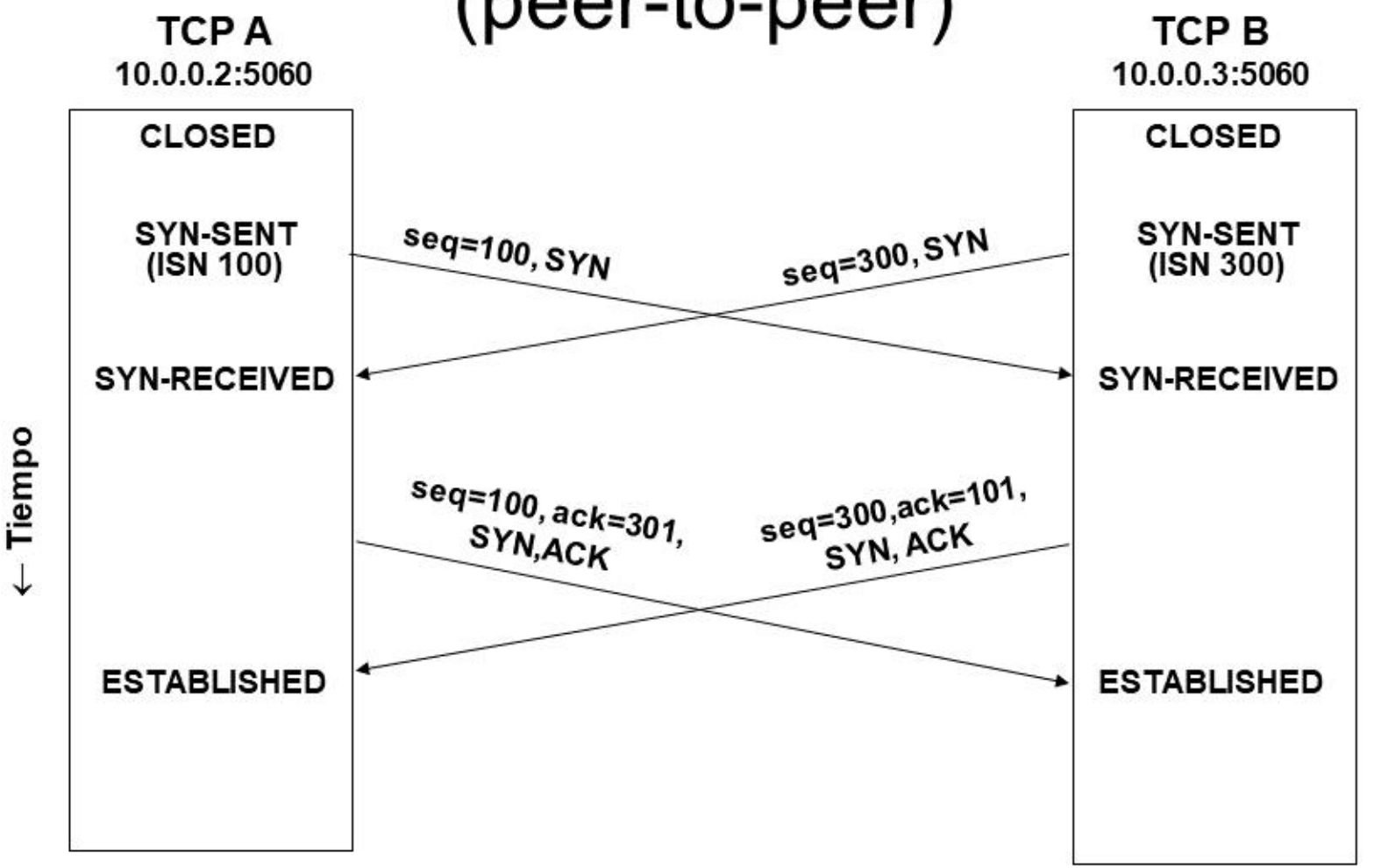
**En UNIX BSD el 'SYN-RECEIVED timeout' (tiempo de vida de una conexión embrionaria) es de 75 segundos**

**El RFC 4987 da diversas recomendaciones sobre como evitar este tipo de ataques**

# Conexión simultánea o simétrica

- Aunque poco probable, es posible que dos hosts inicien a la vez el proceso de conexión, cruzándose los SYN en el camino.
- En ese caso TCP prevé que se establezca sólo una conexión (no dos) y que ambos envíen el segundo mensaje (SYN-ACK) adoptando el papel de 'passive open' (o servidor) hacia el otro, dando por establecida la conexión inmediatamente sin esperar el tercer mensaje. En este caso se intercambian en total cuatro mensajes.
- Para que ocurra una conexión simultánea las aplicaciones en ambos hosts deben haber averiguado de alguna forma el puerto que han de utilizar. Puede ser el caso de SIP, donde el puerto utilizado es el 5060 en ambos lados.

# Conexión simultánea o simétrica (peer-to-peer)



# Números de secuencia

| Puerto de Origen | | | Puerto de destino |
|---|---|---|---|
| Número de Secuencia | | | |
| Número de ACK | | | |
| L. Cab | Res. | Flags | Tamaño Ventana |
| Checksum | | | Punt. datos urgentes |

- TCP utiliza el número de secuencia para numerar todos los bytes que transmite en la conexión.
- El número de secuencia indica el número del siguiente byte que se va a enviar. Si el segmento lleva datos ese es el número del primer byte que viaja en ese segmento, si no lleva datos ese byte se enviará más tarde
- Cuando el número de secuencia llega a su valor máximo ($2^{32}$-1 = 4.294.967.296) se da la vuelta y empieza desde cero. Como el valor inicial se elige al azar el tiempo que tarde en darse la vuelta el contador es impredecible

# Pregunta

P: ¿De que manera afecta el tamaño del número de secuencia al caudal máximo de una conexión TCP?

R: Para evitar ambigüedades el número de secuencia no debe darse la vuelta en el tiempo de vida de un segmento. Por tanto con un TTL de 120 el caudal máximo sería $2^{32} = 4.294.967.296$ bytes en 120 segundos:

$$4.294.967.296 \times 8 / 120 = \mathbf{286\ Mb/s}$$

Con un TTL de 255 el caudal sería de solo 135 Mb/s

# Números de secuencia en segmentos SYN




- *A efectos de números de secuencia el flag SYN equivale a 1 byte*. Podemos considerar que el segmento SYN equivale a un byte de datos 'virtual'
- Normalmente el segmento SYN no lleva datos, pero podría llevarlos. En ese caso el número de secuencia se incrementa en el número de bytes que contiene el segmento más uno.

# Números de ACK y flag ACK




- El número de ACK indica el número del primer byte se espera recibir, en el próximo segmento, del otro TCP.
- Sirve para indicar al otro TCP que todos los bytes anteriores a ese se han recibido correctamente
- El flag ACK indica que el contenido del campo 'número de ACK' tiene sentido o significado. Esto ocurre en todos los segmentos intercambiados en una conexión TCP excepto el primero de todos. Consecuentemente todos menos el primero tienen puesto el flag ACK
- La presencia del flag ACK no incrementa el número de secuencia.

# Cabeceras TCP del inicio de una conexión Telnet

```
1. SYN
TCP:  --- TCP header  ---
TCP:
TCP: Source port = 2345
TCP: Dest port = 23 (Telnet)
TCP: Initial seq. Number = 16421121

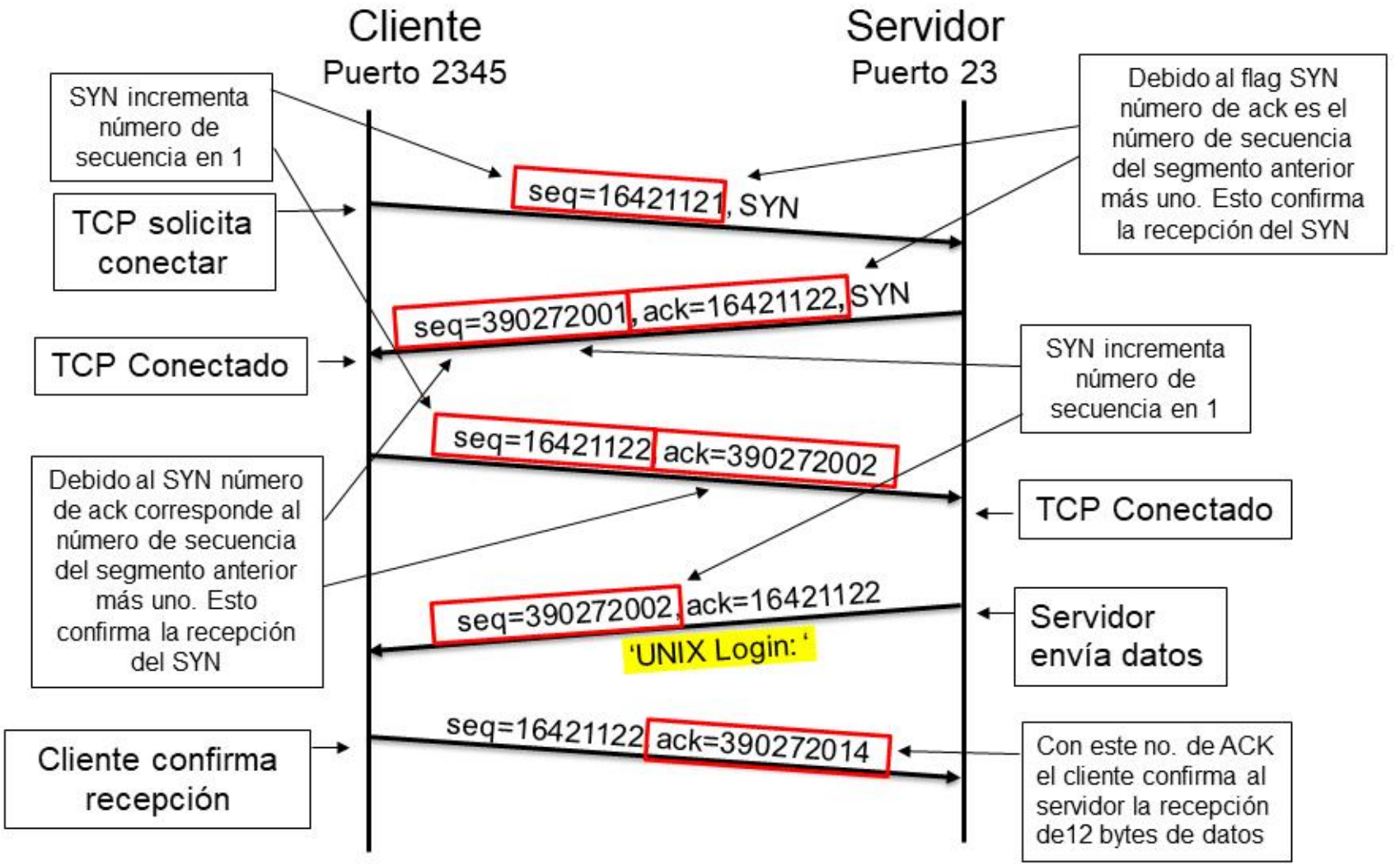
TCP: Data offset = 24 bytes
TCP: Flags = 02 (SYN)
TCP: Window = 2048
TCP: Checksum = F2DA (correct)
TCP:
TCP: Options follow
TCP: Max segment size = 1460
```

```
2. SYN
TCP:  --- TCP header  --
TCP:
TCP: Source port = 23 (Telnet)
TCP: Dest port = 2345
TCP: Initial seq. Number = 390272001
TCP: Acknowledgment Number = 16421122
TCP: Data offset = 24 bytes
TCP: Flags = 12 (ACK,SYN)
TCP: Window = 4096
TCP: Checksum = C13A (correct)
TCP:
TCP: Options follow
TCP: Max segment size = 1024
```

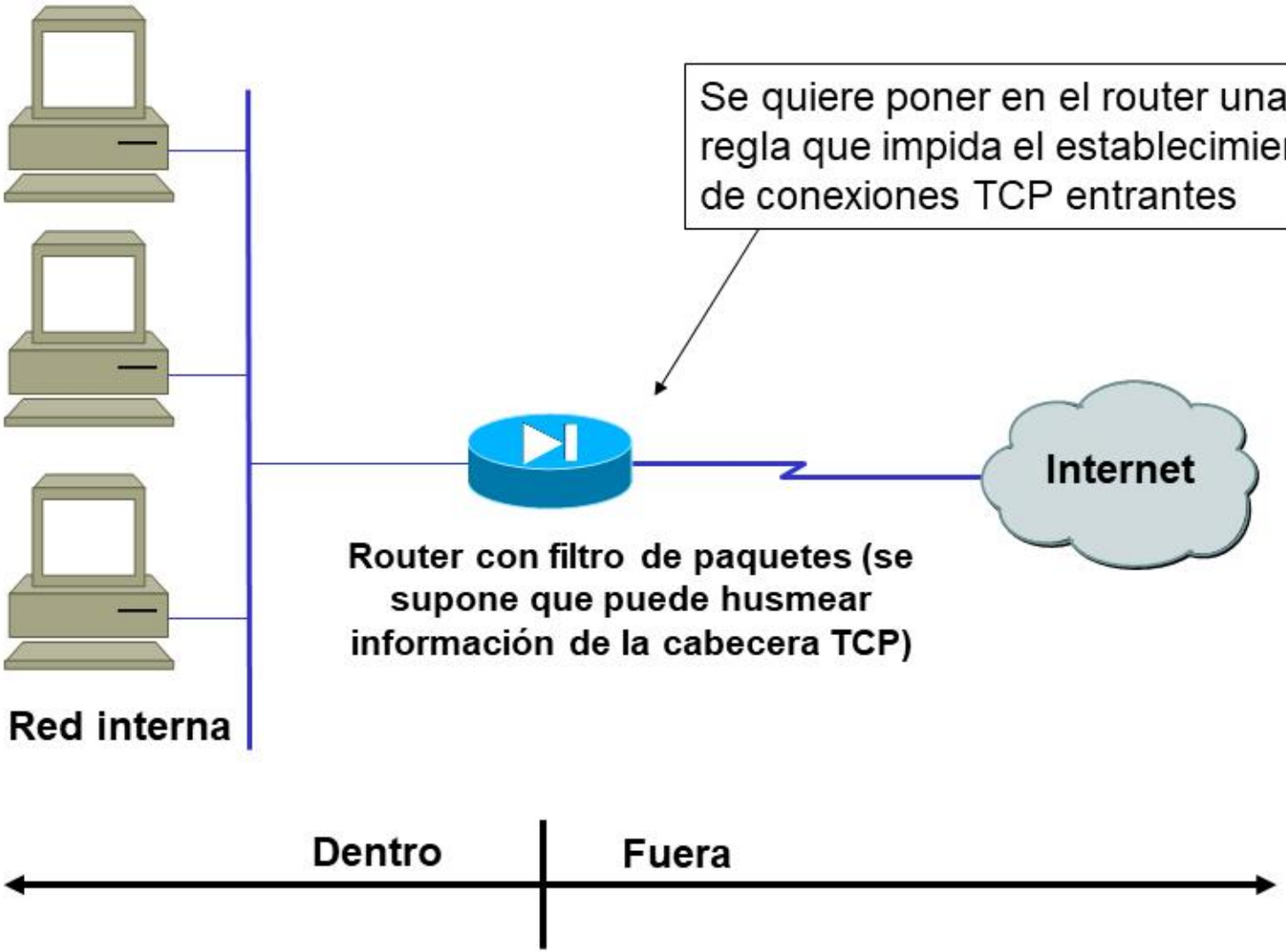
```
3. ACK
TCP:  --- TCP header  ---
TCP:
TCP: Source port = 2345
TCP: Dest port = 23 (Telnet)
TCP: Seq. Number = 16421122
TCP: Acknowledgment Number =
     390272002
TCP: Data offset = 20 bytes
TCP: Flags = 10 (ACK)
TCP: Window = 2048
TCP: Checksum = DF43 (correct)
TCP: No TCP options
```

```
4. DATA
TCP:  --- TCP header  ---
TCP:
TCP: Source port = 23 (Telnet)
TCP: Dest port = 2345
TCP: Seq. Number = 390272002
TCP: Acknowledgment Number = 16421122
TCP: Data offset = 20 bytes
TCP: Flags = 18 (ACK,PSH)
TCP: Window = 4096
TCP: Checksum = 9FEF (correct)
TCP: No TCP options
TCP: [12 byte(s) of data]
```

# Intercambio de segmentos del caso anterior



# Ejercicio: Cortafuegos básico



# Ejercicio: solución

- El router descartará los datagramas que entren por la interfaz serie y que cumplan estas dos condiciones:
  - Tener el valor 6 (TCP) en el campo protocolo de la cabecera IP, y
  - Tener a 1 el flag SYN y a 0 el ACK en la cabecera TCP.
- El segmento que solicita establecer la conexión (el primero de todos) es el único de toda la sesión TCP que tiene la combinación SYN = 1, ACK = 0

# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - **Desconexión**
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

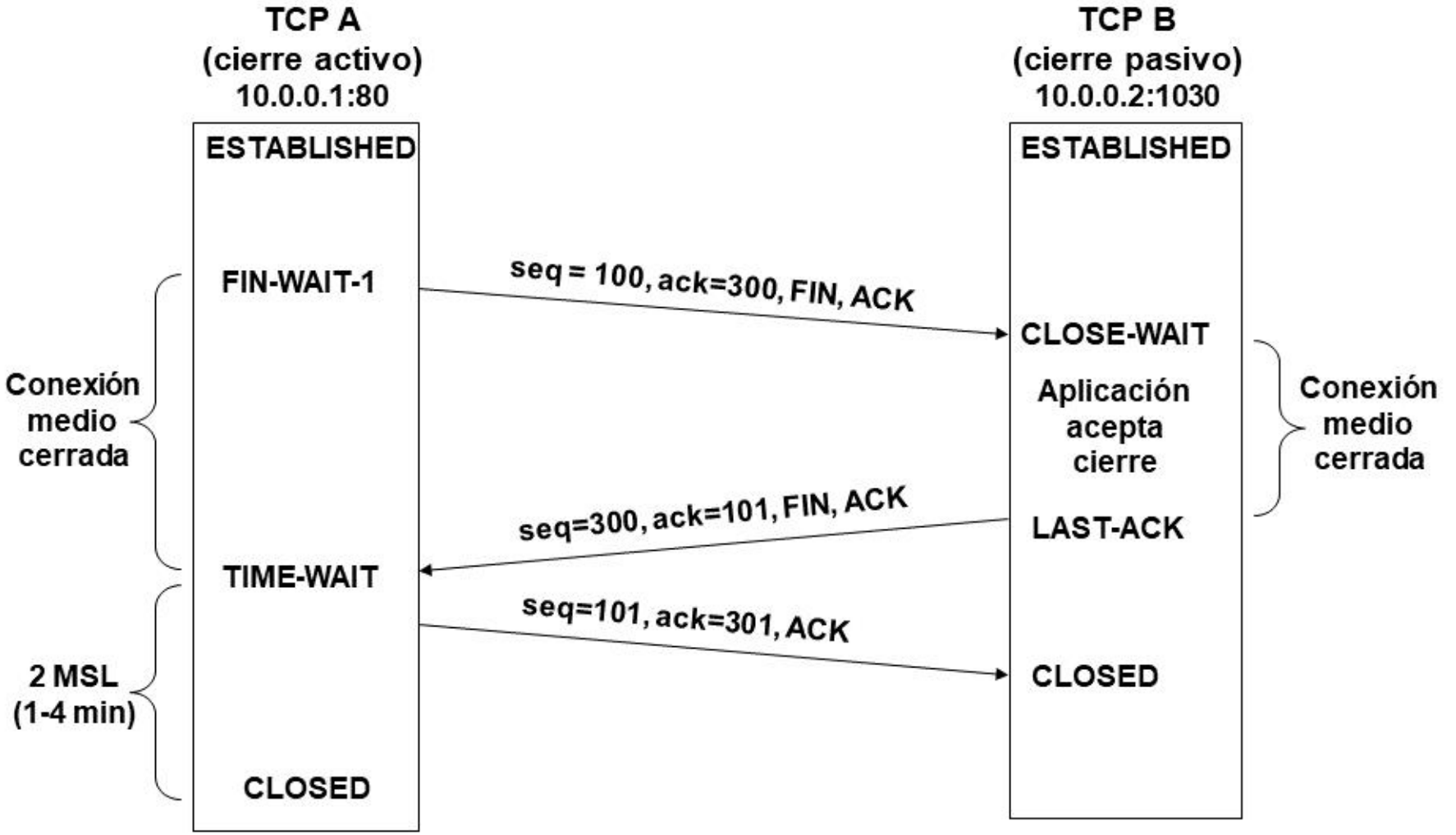
# Desconexión

- La desconexión en TCP puede ser de dos tipos:
  - **Ordenada, consensuada o simétrica**: la conexión se considera formada por dos circuitos simplex y cada host solo puede cortar el sentido en el que él envía datos. El cierre de un sentido por parte de un host se interpreta como una 'invitación' a cerrar al otro. Para cerrar ambos utilizan el flag FIN
  - **Desordenada, unilateral o asimétrica**: un host termina y cierra de forma inmediata, sin esperar una confirmación. No da oportunidad al otro de enviar los datos que tuviera pendientes de la aplicación, por lo que puede provocar la pérdida de información. Se hace utilizando el flag RST

# Desconexión con 3 mensajes

- La desconexión ordenada de TCP se basa en que cada TCP cierra la conexión desde su lado enviando un FIN y cada FIN debe ser confirmado por el otro. Esto requiere como mínimo el envío de tres mensajes (despedida a tres vías). Así se asegura que no se pierden datos
- Normalmente la desconexión la inicia el cliente, pero puede iniciarla el servidor. El que toma la iniciativa realiza un cierre activo o 'active close' y el otro lleva a cabo un cierre pasivo o 'passive close'.
- Durante el tiempo que va desde que se envía (recibe) el primer FIN hasta que se recibe (envía) el segundo tenemos una conexión 'medio cerrada'

# Desconexión ordenada con 3 mensajes

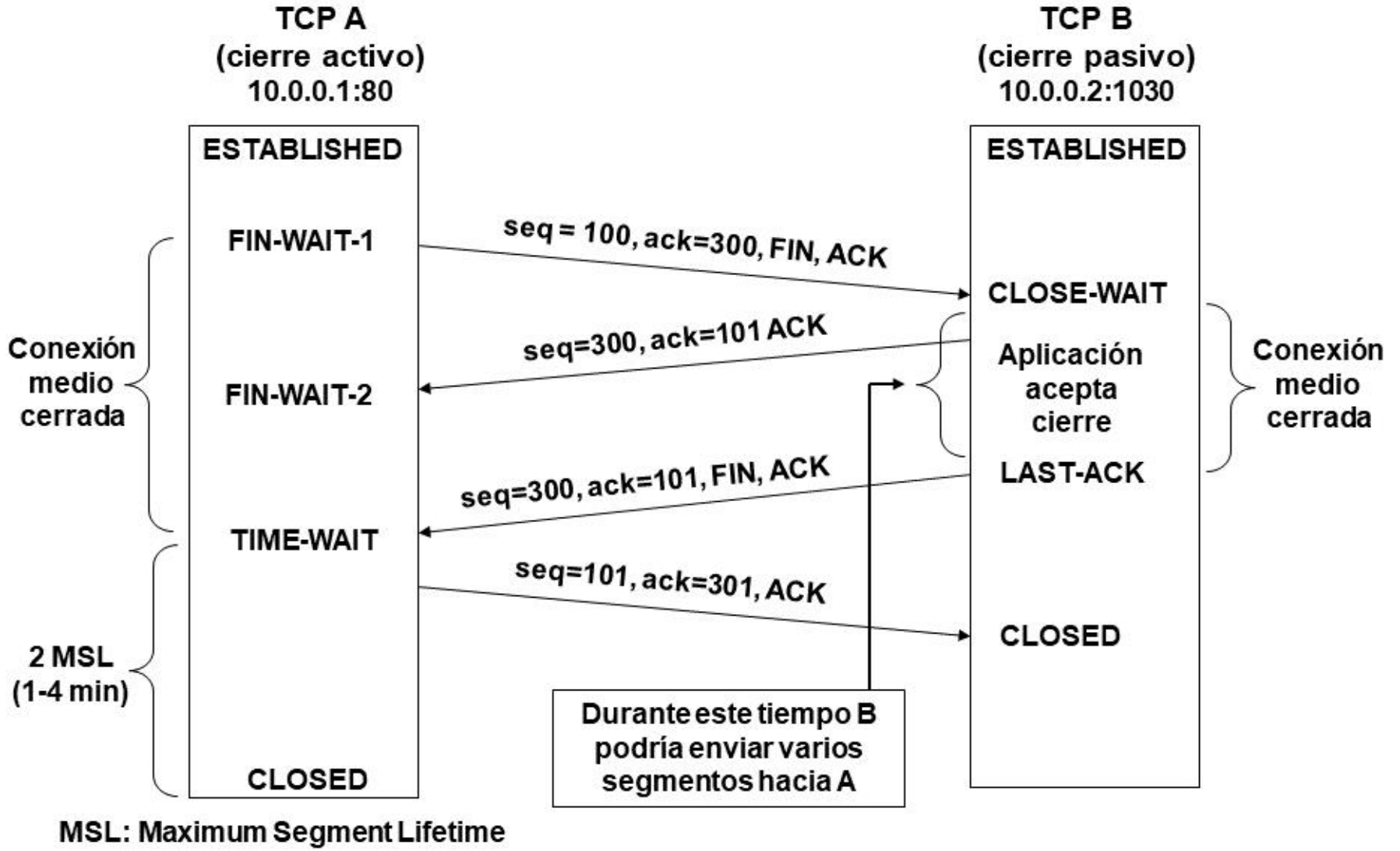



MSL: Maximum Segment Lifetime

# Desconexión con 4 mensajes

- El TCP que realiza el cierre pasivo tiene que recibir permiso de su aplicación para aceptar el cierre.
- En la mayoría de los casos el TCP pasivo devuelve enseguida un ACK sin FIN; cuando más tarde recibe el permiso de la aplicación envía el FIN y el otro TCP devuelve un ACK. En ese caso el cierre requiere cuatro mensajes.
- En teoría la aplicación que ejecuta el cierre pasivo aún puede en esta fase enviar cualquier cantidad de datos, lo cual podría requerir el envío de más mensajes
- En la práctica llegados a este punto ya casi no se transmiten datos, por lo que casi siempre las conexiones TCP se cierran intercambiando 3 ó 4 segmentos.

# Desconexión ordenada con 4 mensajes




MSL: Maximum Segment Lifetime

# Estado TIME-WAIT

- Cuando el TCP que hace el 'active close' recibe el FIN no cierra la conexión inmediatamente, sino que la mantiene en TIME-WAIT un tiempo, por si llegan paquetes retrasados.
- Ese tiempo es el doble del MSL (Maximum Segment Lifetime), un parámetro del sistema cuyo valor suele estar entre 30 seg. y 2 min.
- Puesto que el TTL de la cabecera IP puede valer hasta 255 el tiempo de vida máximo de un datagrama IP sería en teoría de 4 min. 15 seg. En Internet habitualmente se considera que el tiempo de vida máximo de un datagrama son 2 min. (equivalente a un TTL de 120)

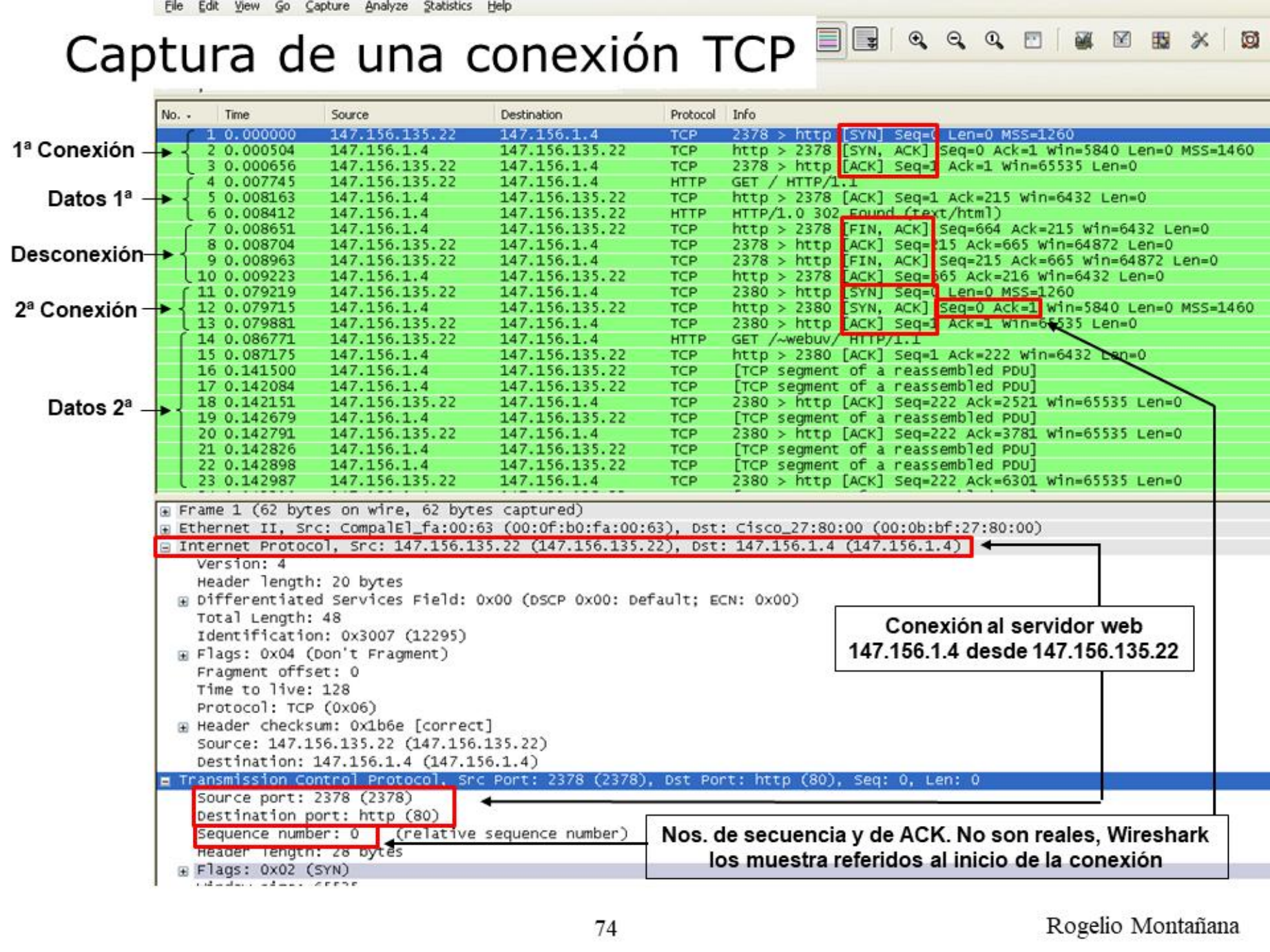
# Estado TIME-WAIT

- La finalidad del TIME-WAIT no es procesar los segmentos retrasados que puedan llegar. Una conexión en TIME-WAIT ya no admite datos, y si llega algo se descarta
- El objetivo es bloquear esa conexión (IP-puerto_IP-puerto) para evitar que una 'reencarnación' de esa conexión, es decir una nueva conexión entre el mismo par de sockets, reciba paquetes retrasados y los interprete como propios
- Cuando un host se reinicia debe esperar al menos 2*MSL antes de crear conexiones TCP, para evitar capturar paquetes de encarnaciones anteriores.

# Limitaciones derivadas del tiempo en TIME-WAIT

- Con el rango de puertos efímeros recomendado por la IANA (16384, de 49152 a 65535) para no reutilizar un socket en 120 segundos habría que hacer no más de 136 conexiones por segundo (16384/120).
- En realidad el socket se podría reutilizar antes, siempre y cuando el socket del otro extremo no sea también el mismo. Pero para simplificar la comprobación muchas implementaciones simplemente bloquean el socket durante el tiempo TIME-WAIT cualquiera que sea el socket con el que se quiera conectar en el otro extremo.

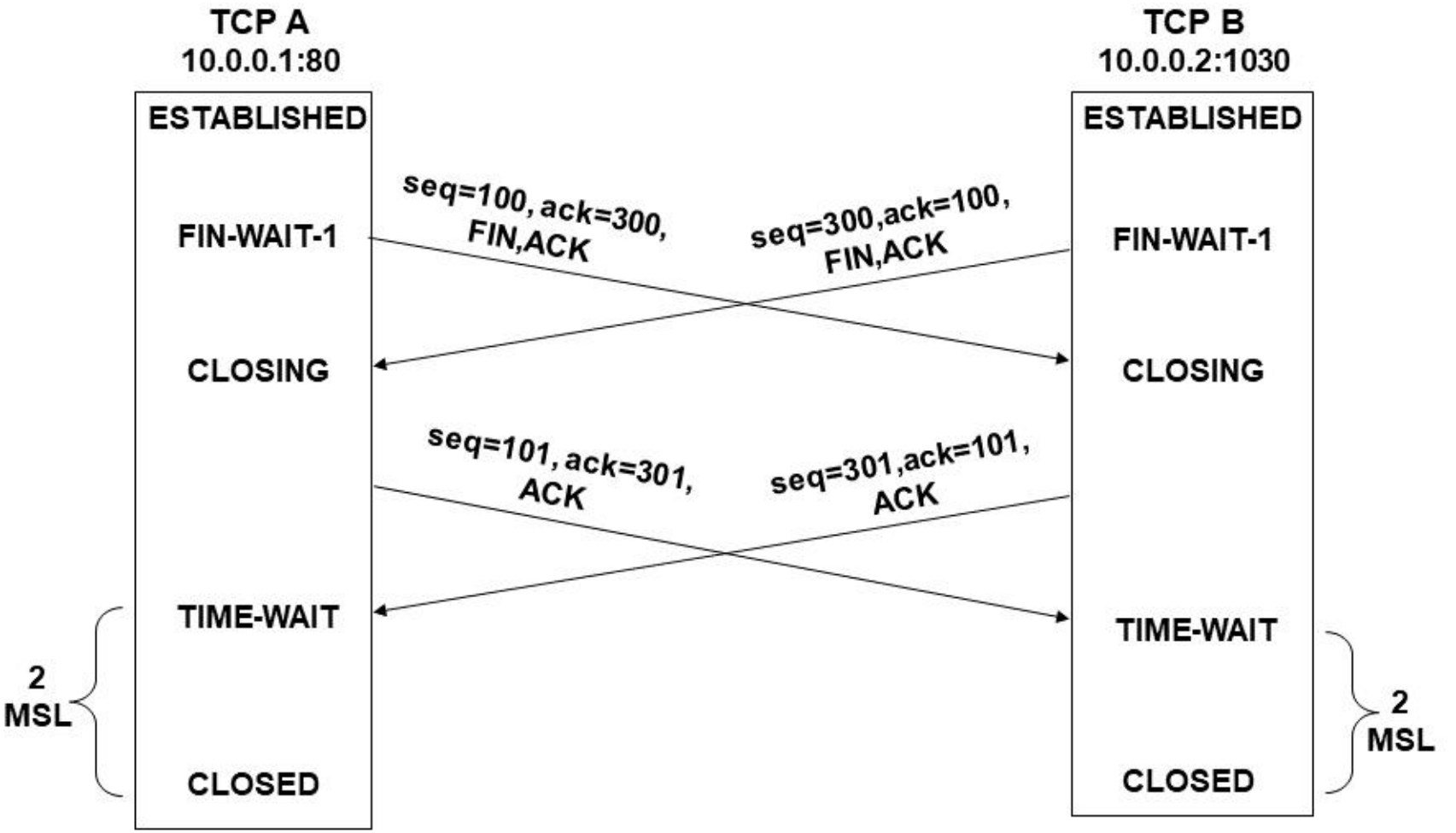
# Captura de una conexión TCP



# Desconexión ordenada simultánea

- Es posible que dos TCP decidan simultáneamente terminar una conexión, cruzándose los mensajes FIN en el camino
- En ese caso cada TCP envía al otro un ACK confirmando la recepción del FIN, y el cierre se realiza intercambiando siempre 4 segmentos.
- Ambos TCP han de esperar el tiempo 2*MSL antes de liberar por completo la conexión .
- La evolución de estados es ligeramente distinta del caso normal, apareciendo un nuevo estado (CLOSING).

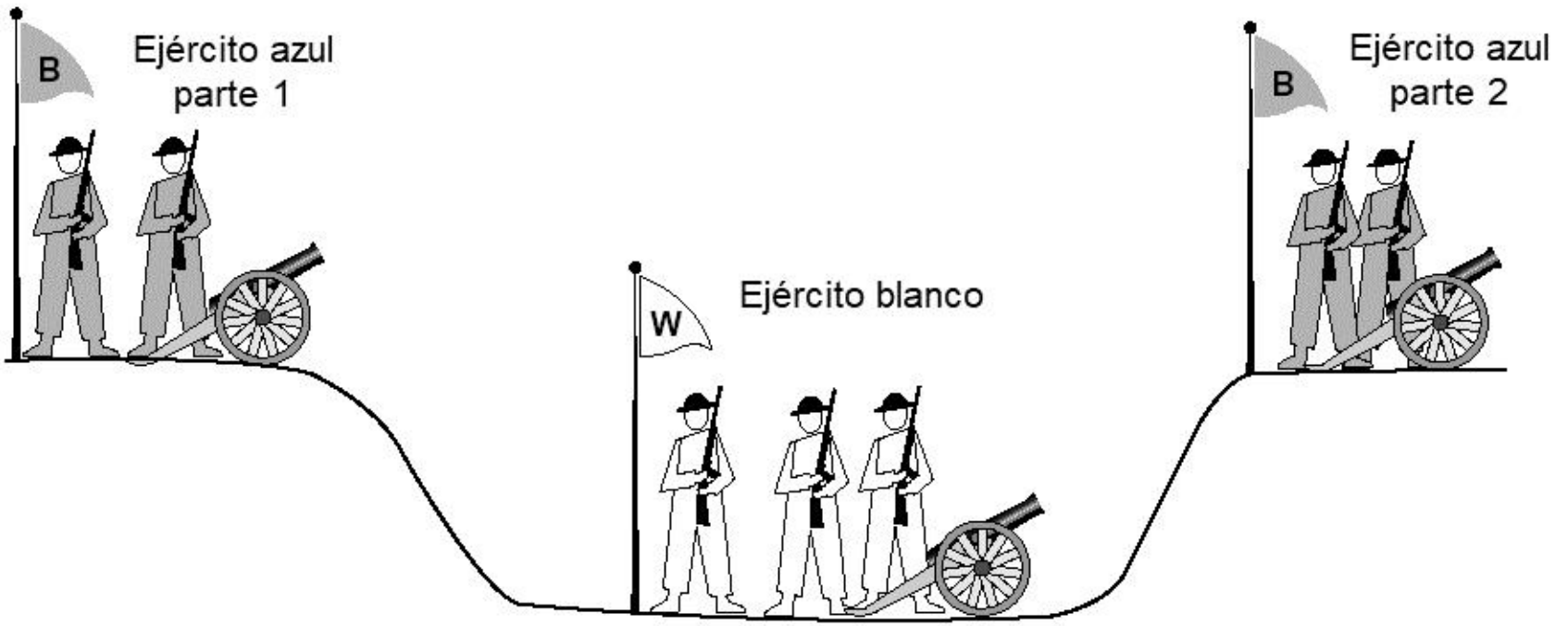
# Desconexión ordenada simultánea



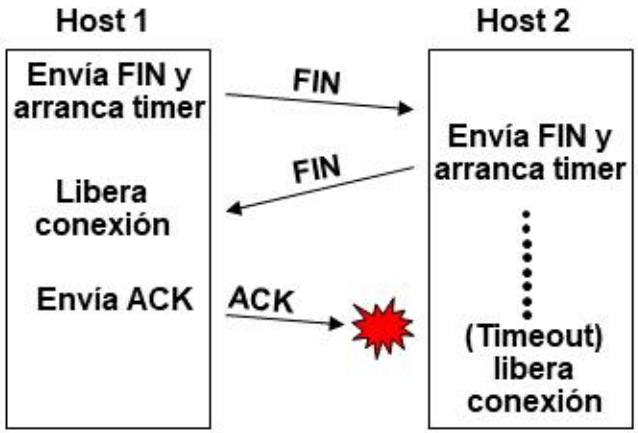
# Pérdida de mensajes de desconexión

- El mensaje FIN solicitando la desconexión se puede perder. Por eso se pide una confirmación.
- Pero la confirmación también puede perderse, por lo que habría que enviar una confirmación de la confirmación, y así sucesivamente.
- Este problema no tiene solución, pues estamos intentando usar un canal no fiable para asegurar el envío de información. Es lo que se conoce como el problema de los dos ejércitos.

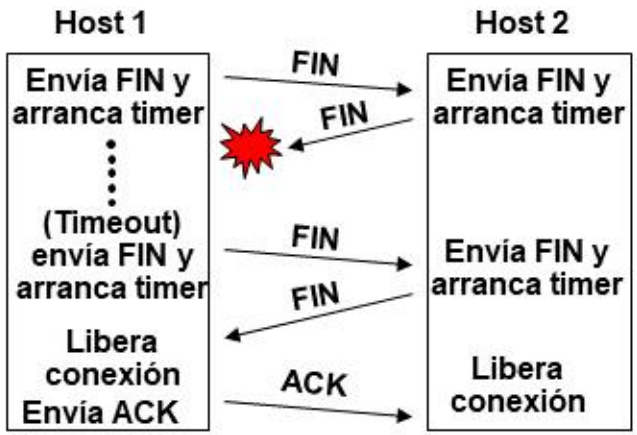
# El problema de los dos ejércitos



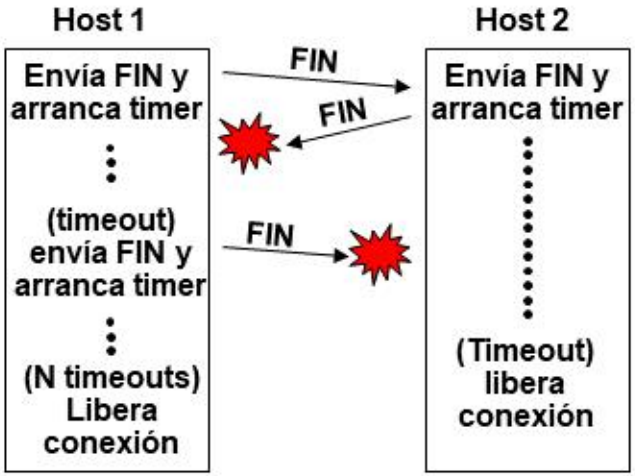
# Pérdida de segmentos en la desconexión



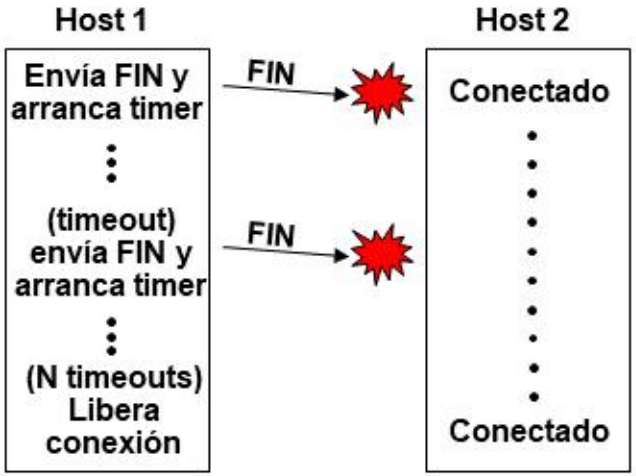

1: Pérdida del ACK final




2: Pérdida del segundo FIN




3: Pérdida de todo excepto el 1er FIN
(Usuario con prisa, cierra la sesión e inmediatamente desconecta el cable)



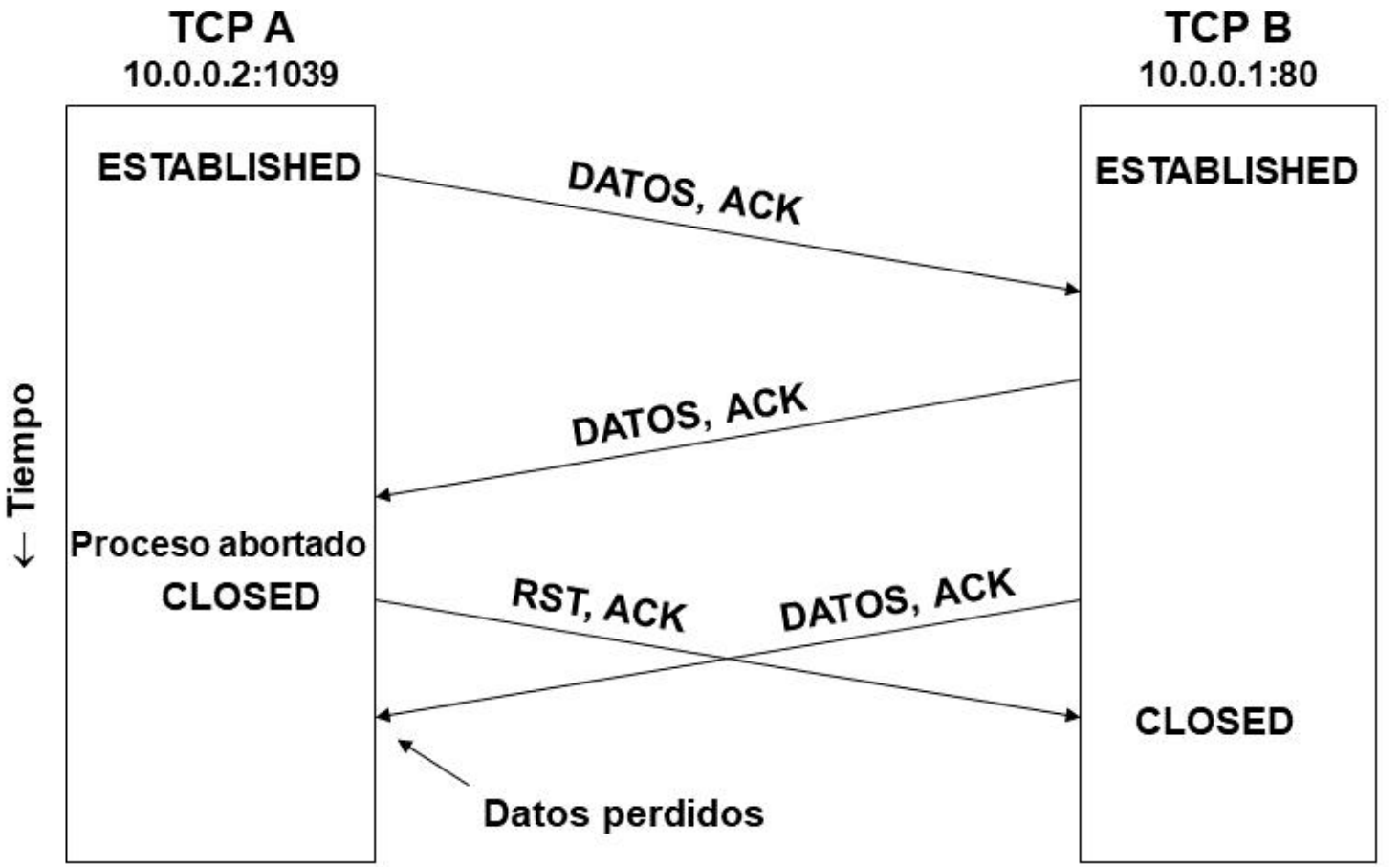

Conexión medio abierta

4: Pérdida de todo en todos los intentos
(Usuario desconecta el cable antes de cerrar la sesión)

# Desconexión desordenada o unilateral

- Un TCP puede cerrar inmediatamente una conexión, sin esperar la aceptación del otro. Para ello envía un segmento con el flag RST, sin datos
- Al enviar el RST pasa inmediatamente al estado CLOSED perdiéndose los datos que pudieran venir de camino. No se espera 2*MSL.
- El TCP que recibe el RST pasa también al estado CLOSED de manera inmediata
- En este caso el cierre se hace con un mensaje solamente
- Las aplicaciones siempre intentan cerrar ordenadamente. Pero si desde el S.O. abortamos un proceso las conexiones se cierran de forma inmediata mandando un RST

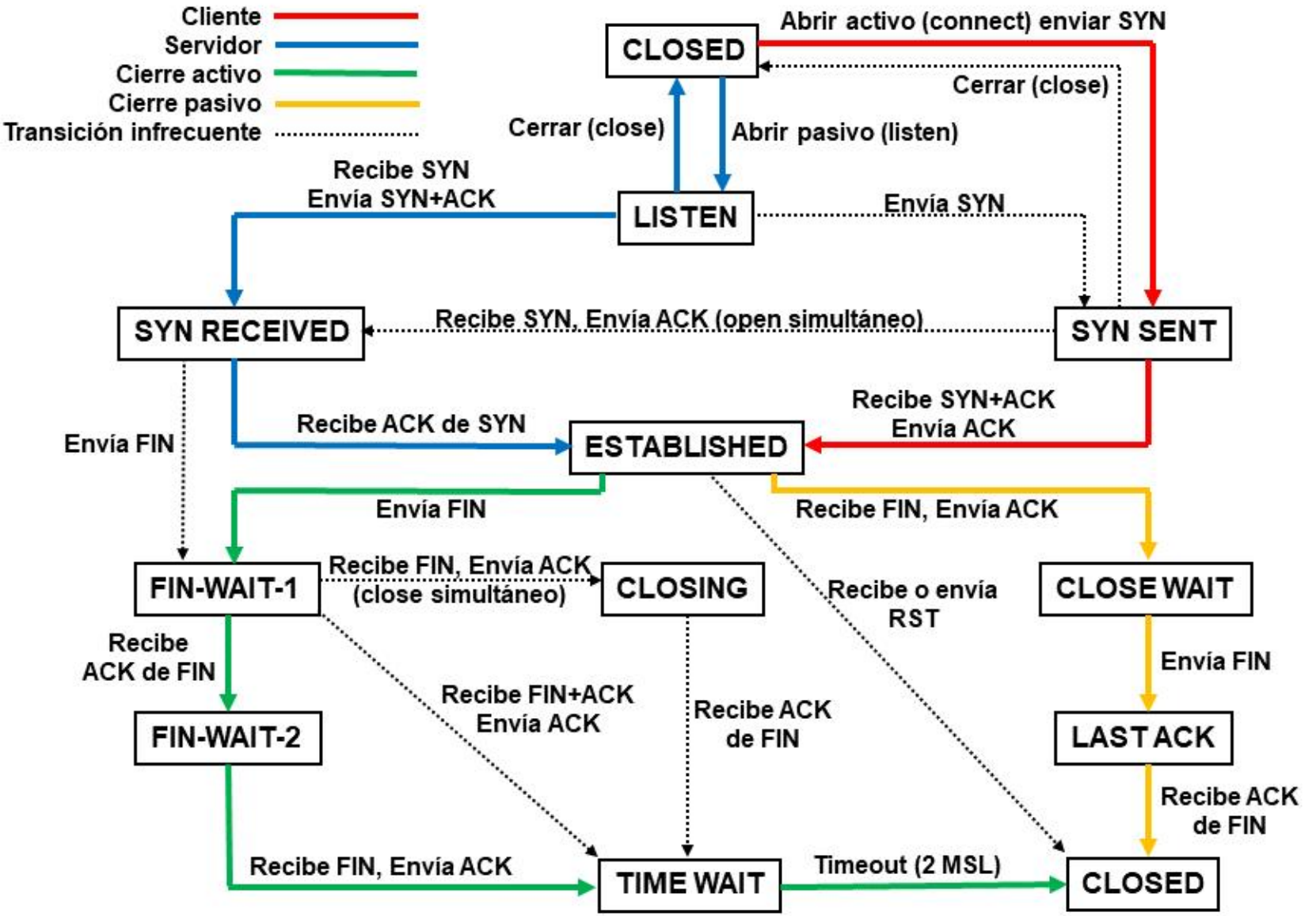
# Desconexión desordenada o unilateral



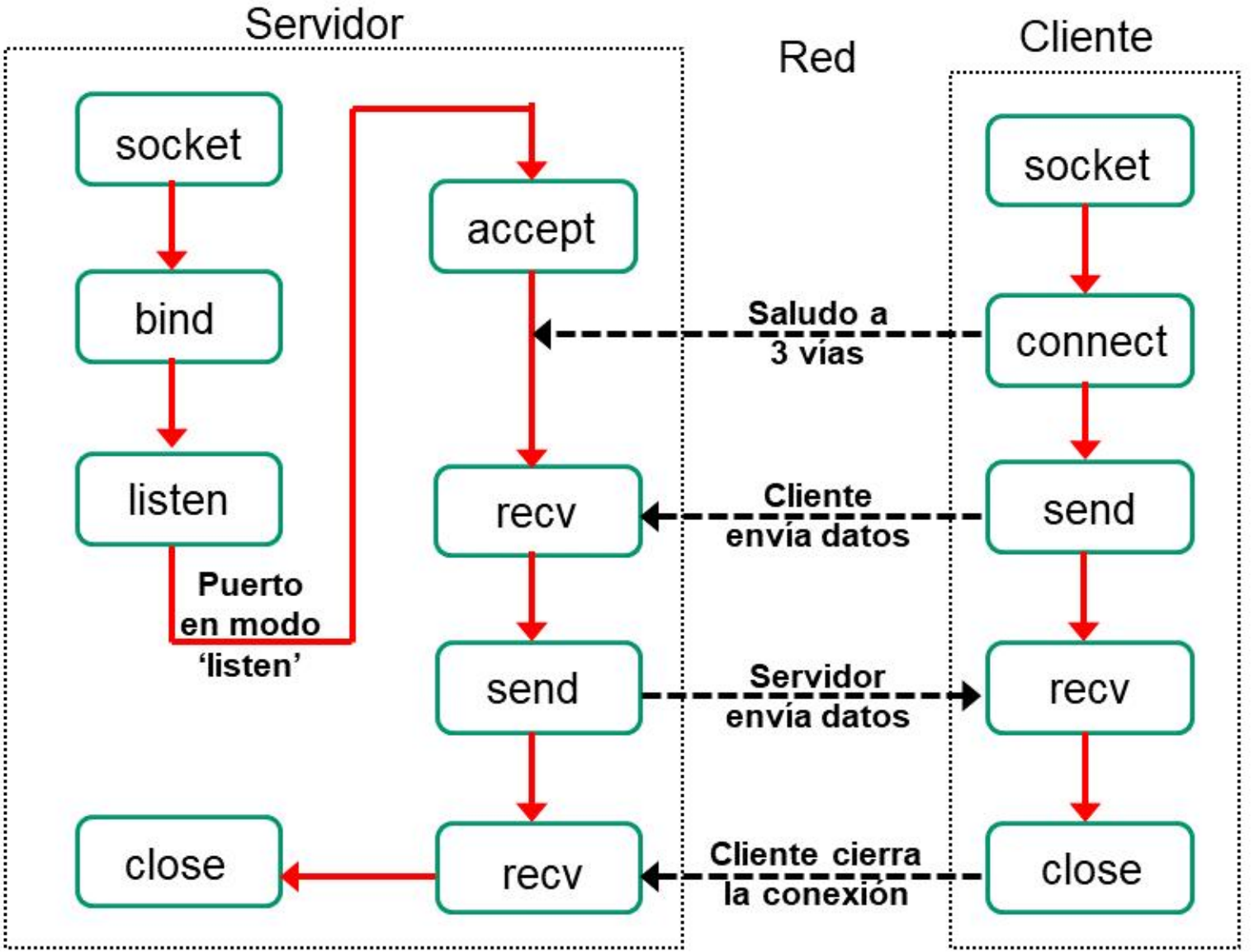
# Formas de cerrar una aplicación

- Si el proceso que ejecuta la aplicación se cierra ordenadamente se envían mensajes FIN en ambos sentidos y se hace un cierre ordenado simétrico
- Si el proceso se aborta desde el S.O. (comando kill o similar) este envía un RST antes de morir y se hace un cierre unilateral
- Si el S.O. cae de forma imprevista (por ejemplo por un fallo de corriente) o si el cable se desenchufa sin cerrar antes la conexión se queda medio abierta en el otro lado

# Diagrama de estados de TCP



# Diagrama de flujo cliente/servidor TCP con Berkeley sockets



# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - **Intercambio de datos y control de flujo**
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

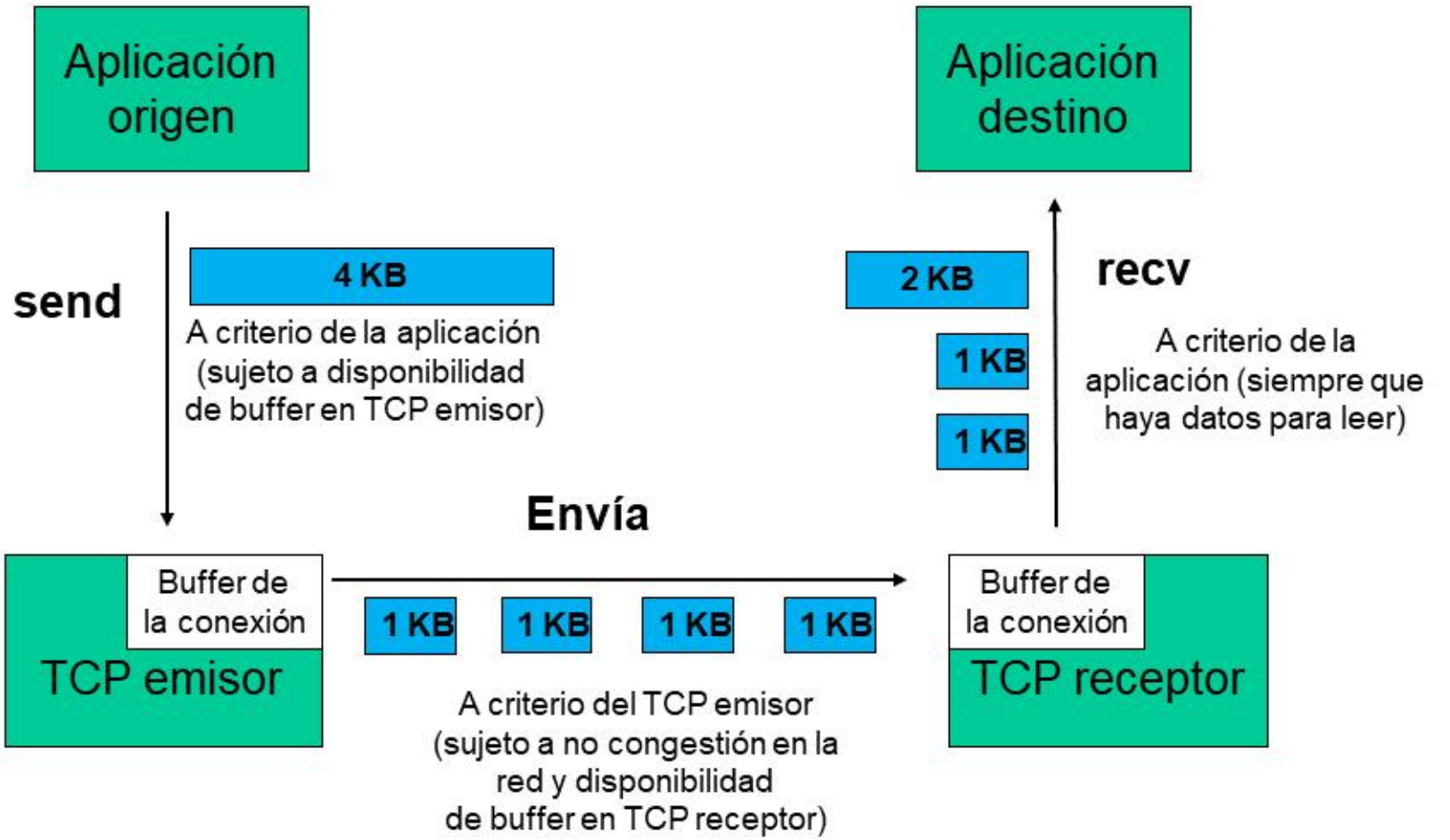
# Intercambio de datos TCP ↔ aplicación

- La aplicación ve a TCP como un fichero, donde puede leer y escribir cuando quiera y cuanto quiera (siempre que haya datos para leer y sitio para escribir en el buffer que TCP ha asignado a esa conexión)
- Una vez la aplicación entrega los datos a TCP éste tiene total libertad para decidir como los reagrupa y envía al otro TCP
- Para TCP los datos son un flujo continuo de bytes, independientemente de la agrupación lógica que pueda manejar la aplicación (registros, etc.). Es responsabilidad de la aplicación asegurarse de que esa agrupación, si existe, se mantendrá después de transmitidos los datos.

# Intercambio de datos TCP ↔ TCP

- Cada segmento TCP se acomoda en un datagrama IP. Los segmentos nunca se agrupan ni se fragmentan.
- Si el datagrama creado es demasiado grande será IP el encargado de fragmentar, nunca TCP
- El TCP emisor manda los datos cuando él quiere y los agrupa en segmentos como él quiere. A veces agrupa lo que le llega de la aplicación y otras lo divide.
- Se intenta que los segmentos sean lo más grandes posible, pero evitando la fragmentación.

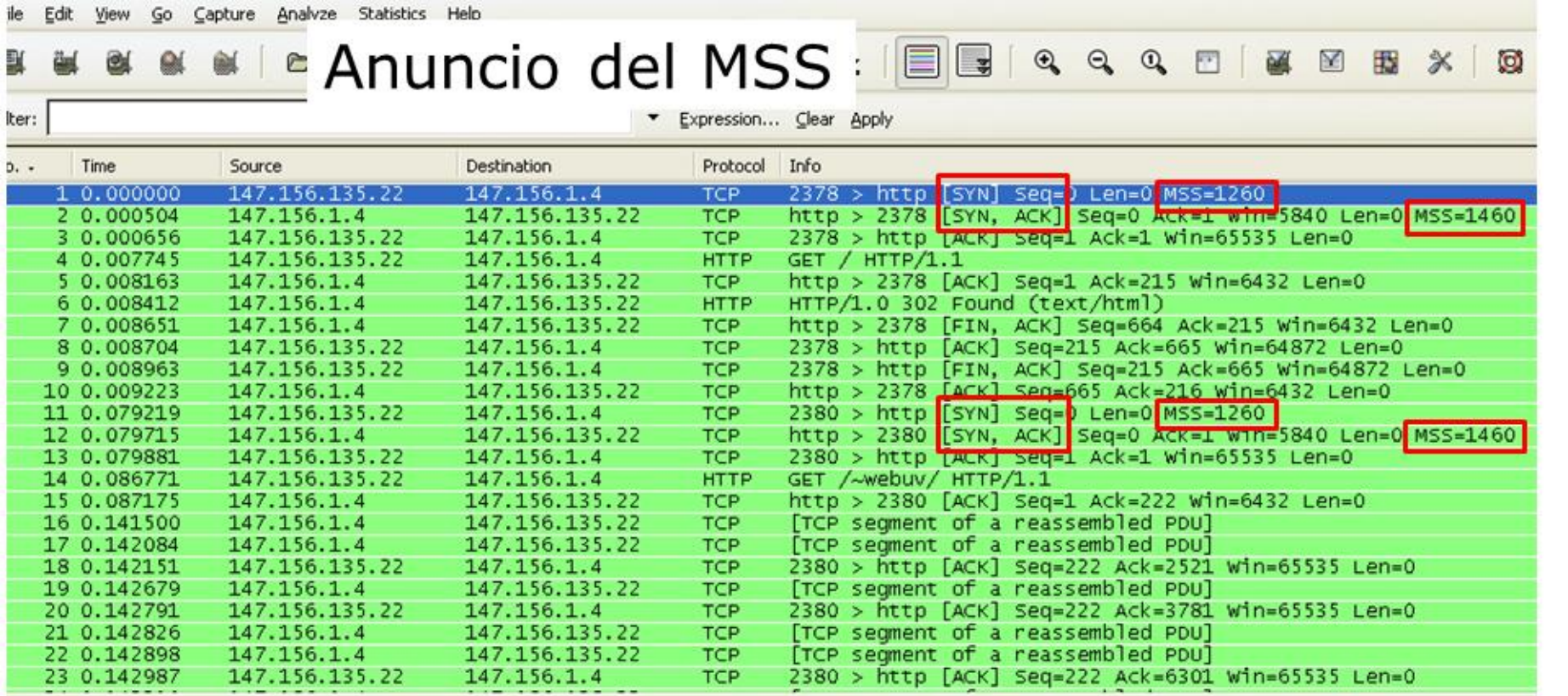
# Intercambio de datos
# TCP ↔ Aplicación y TCP ↔ TCP



# Anuncio del MSS (Maximum Segment Size)

- Cuando se conectan dos TCP cada uno le dice al otro que MSS (Maximum Segment Size) admitirá. El otro puede usar ese valor u otro menor, pero nunca uno mayor
- El MSS se indica mediante una opción de la cabecera TCP que sólo se puede usar en los segmentos SYN, y que casi siempre está presente.
- El MSS se mide en bytes y sólo cuenta los datos, no la cabecera TCP o IP. Por ejemplo un MSS de 1460 equivale a un datagrama de 1500 bytes (lo normal es que las cabeceras no lleven opciones)
- El MSS anunciado depende del tipo de S.O y del tipo de interfaz. Además es configurable

# Anuncio del MSS



# SMSS (Sender MSS)

- El MSS utilizado por el TCP emisor (SMSS, Sender MSS) es el mas pequeño de los siguientes valores:
  - El anunciado por el otro TCP durante la conexión (opción MSS de la cabecera). Si la opción no está presente se utiliza por defecto 536 (se supone que este valor es compatible con la mayoría de las redes)
  - La MTU de la interfaz de salida menos 40 (cabeceras TCP e IP)
- La elección del SMSS intenta satisfacer al receptor y evitar la fragmentación en origen, pero no puede evitar la fragmentación en ruta
- El SMSS elegido puede ser diferente para cada sentido

# Descubrimiento de la MTU del trayecto (Path MTU Discovery, PMTUD)

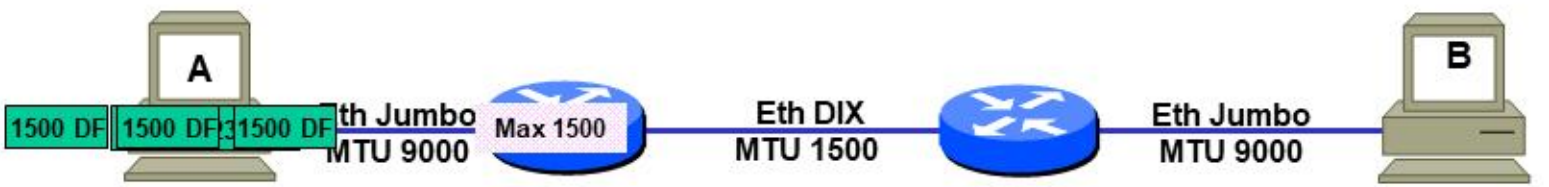
- Es una técnica que aplica TCP (a veces también UDP) para evitar la fragmentación en ruta.
- Una vez elegido el SMSS TCP envía todos los segmentos en datagramas IP marcados con el bit DF=1.
- De esta manera si el paquete no puede pasar el router devolverá un mensaje ICMP destino inaccesible y a partir de la información recibida el TCP emisor podrá ajustar el segmento al tamaño óptimo
- El uso de túneles VPN hace muy frecuente hoy en día tener que utilizar tamaños menores que la MTU de la interfaz

# Funcionamiento de 'Path MTU Discovery'

1: Al conectarse B le anuncia a A (en el SYN) un MSS de 8192

2: A decide utilizar un SMSS de 8192 y envía a B un paquete de 8232 bytes con DF=1.

3: X descarta el paquete y responde a A con un ICMP 'destino inaccesible' fragmentación necesaria. En el mensaje le indica además que si hubiera sido de 1500 bytes habría pasado.




4: A fragmenta la información y a partir de ahora no mandará a B paquetes de más de 1500 bytes. Sigue usando el bit DF todo el tiempo.

Segmento TCP

ICMP 'Destination Unreachable'

# PMTUD

- Si una vez fijado el tamaño óptimo la ruta cambia a otra de PMTU más pequeña el emisor recibe inmediatamente un ICMP Destino inaccesible.
- El RFC 1191 aconseja probar cada 10 min. a enviar un segmento mayor (nunca mayor que el SMSS) por si ha cambiado la ruta y la PMTU ha aumentado. Tanto Windows como Linux siguen ese consejo.
- Los sistemas conservan (normalmente 10 min.) la PMTU para cada destino en una cache de rutas:
  - Linux: `ip route show cache`
  - Windows: `netsh interface ipv4 show destinationcache`

# Pregunta: fragmentación en TCP

- P: ¿Debe TCP preocuparse de reordenar los fragmentos de un datagrama?
- R: NO. El nivel IP reconstruye el datagrama original antes de entregar el segmento a TCP; para ello usa los campos identificación, fragment offset y MF (More Fragments). TCP no podría reordenar los fragmentos aunque quisiera, puesto que normalmente solo el primero tiene cabecera TCP.

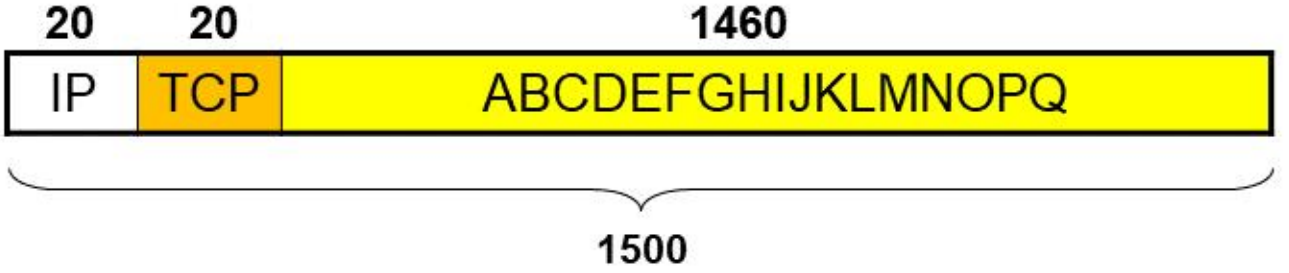
# Ejercicio: fragmentación en TCP

- Tenemos un TCP que envía segmentos del tamaño máximo permitido
- El host tiene una interfaz Ethernet (MTU 1500 bytes)
- El otro TCP ha anunciado un MSS de 1460 bytes.
- No se utiliza 'descubrimiento de la MTU del trayecto'
- Sabemos que la ruta pasa por una red con una MTU de 760 bytes
- Indique cuantos datagramas y bytes recibe el nivel de red en el host de destino por cada segmento enviado

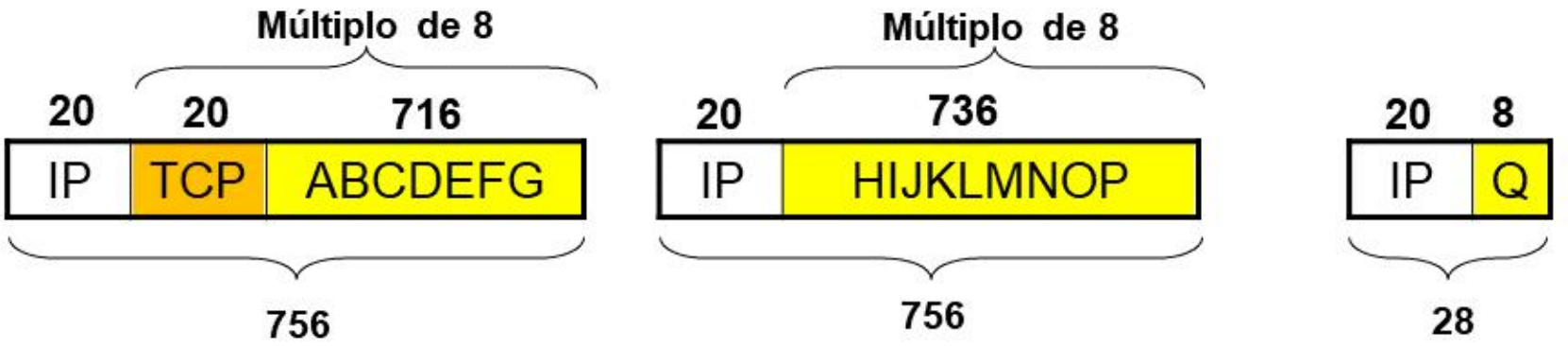
# Ejercicio: solución

**MSS: 1460 -> Segmento TCP: 1480 -> Datagrama IP: 1500**
**MTU: 760 bytes**

**Datagrama emitido por el host:**




**Fragmentos creados por el router:**



# Problemas del PMTUD

- Para funcionar el PMTUD requiere que se dejen pasar los mensajes ICMP Destino Inacceisible
- Algunos cortafuegos no dejan pasar ningún tipo de mensaje ICMP. Esto provoca lo que se conoce como 'conexiones de agujero negro'
- Si tenemos problemas en algunas conexiones y sospechamos que la causa es un agujero negro podemos reducir el MSS, bien en el host emisor o en el router. Esto se conoce como 'MSS-clamping' o 'atornillar el MSS'

# Datos 'pushed'

| Puerto de Origen | | | Puerto de destino |
|---|---|---|---|
| Número de Secuencia | | | |
| Número de ACK | | | |
| L. Cab | Res. | Flags | Tamaño Ventana |
| Checksum | | | Punt. datos urgentes |

- En teoría el flag PSH (Push) sirve para indicar a TCP que debe enviar los datos lo antes posible, sin esperar a tener más datos para poder hacer un segmento mayor.
- En la práctica muchas APIs no prevén una forma para que la aplicación le indique a TCP que quiere poner el flag PSH. Generalmente el flag PSH lo utiliza el TCP emisor para indicar que ha vaciado el buffer de emisión.

# Datos urgentes (flag URG)




- Los datos urgentes se entregan a la aplicación sin esperar a que esta ejecute un 'read'. Por ejemplo, abortar con CTRL-C un programa que ha caído en un bucle cuando estamos conectados mediante una sesión telnet.
- Por su propia naturaleza, los datos urgentes se 'cuelan' por delante de cualesquiera datos que hubiera pendientes en el buffer
- Los datos urgentes siempre van al principio del segmento marcado URG. Detrás puede haber datos no urgente. El 'puntero de datos urgentes' indica en que posición del segmento empiezan esos datos.

# Flujo de datos de TCP

- Para analizar la forma como TCP envía los datos vamos a distinguir dos tipos de tráfico:
  - **Tráfico interactivo**: es el que producen aplicaciones que transmiten datos en pequeñas cantidades, por ejemplo telnet, rlogin, X windows, escritorio remoto, etc. Normalmente generan segmentos muy pequeños
  - **Tráfico masivo**: lo generan aplicaciones que envían grandes cantidades de datos, como FTP, SMTP o HTTP. Normalmente producen, en uno de los sentidos, segmentos del tamaño máximo
- Aunque los mecanismos en uno y otro caso son diferentes, TCP se adapta bastante bien a ambas situaciones

# Ejemplo de tráfico interactivo: caso telnet

- Telnet (Teletype Network) es un protocolo de nivel de aplicación para la emulación de terminal remoto.
- Para su correcto funcionamiento telnet necesita generalmente utilizar el eco remoto, es decir que los caracteres escritos en el teclado sean representados en la pantalla por el host remoto.
- Esto significa que cada vez que se pulsa una tecla se transmiten como mínimo dos paquetes, uno de ida con el carácter pulsado y otro de vuelta con el carácter que aparecerá en pantalla. Esto es preciso para que funcionen cosas como la ocultación de contraseñas.

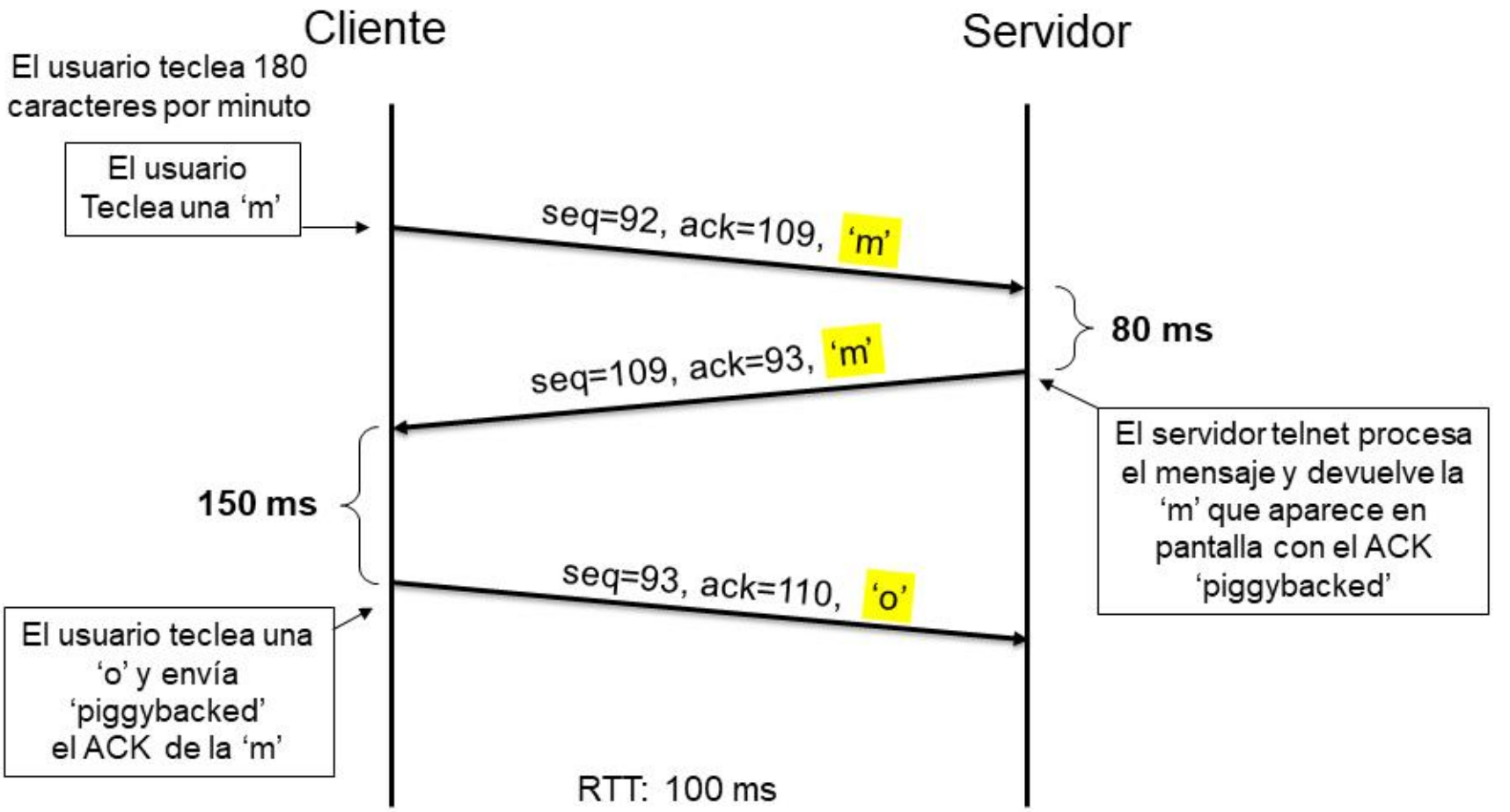
# Tráfico interactivo TelNet

- En realidad, por cada tecla pulsada telnet intercambia 2, 3 ó 4 segmentos, dependiendo de la rapidez relativa del usuario (el cliente) y del servidor:

| | |
|---|---|
| Caso 1: Usuario y servidor rápidos | 2 segmentos por pulsación |
| Caso 2: Usuario lento, servidor rápido | 3 segmentos por pulsación |
| Caso 3: Usuario rápido, servidor lento | 3 segmentos por pulsación |
| Caso 4: Usuario y servidor lentos | 4 segmentos por pulsación |

Se considera rápido si el tiempo de respuesta es de 200 ms o menor (equivalente a 300 pulsaciones por minuto)

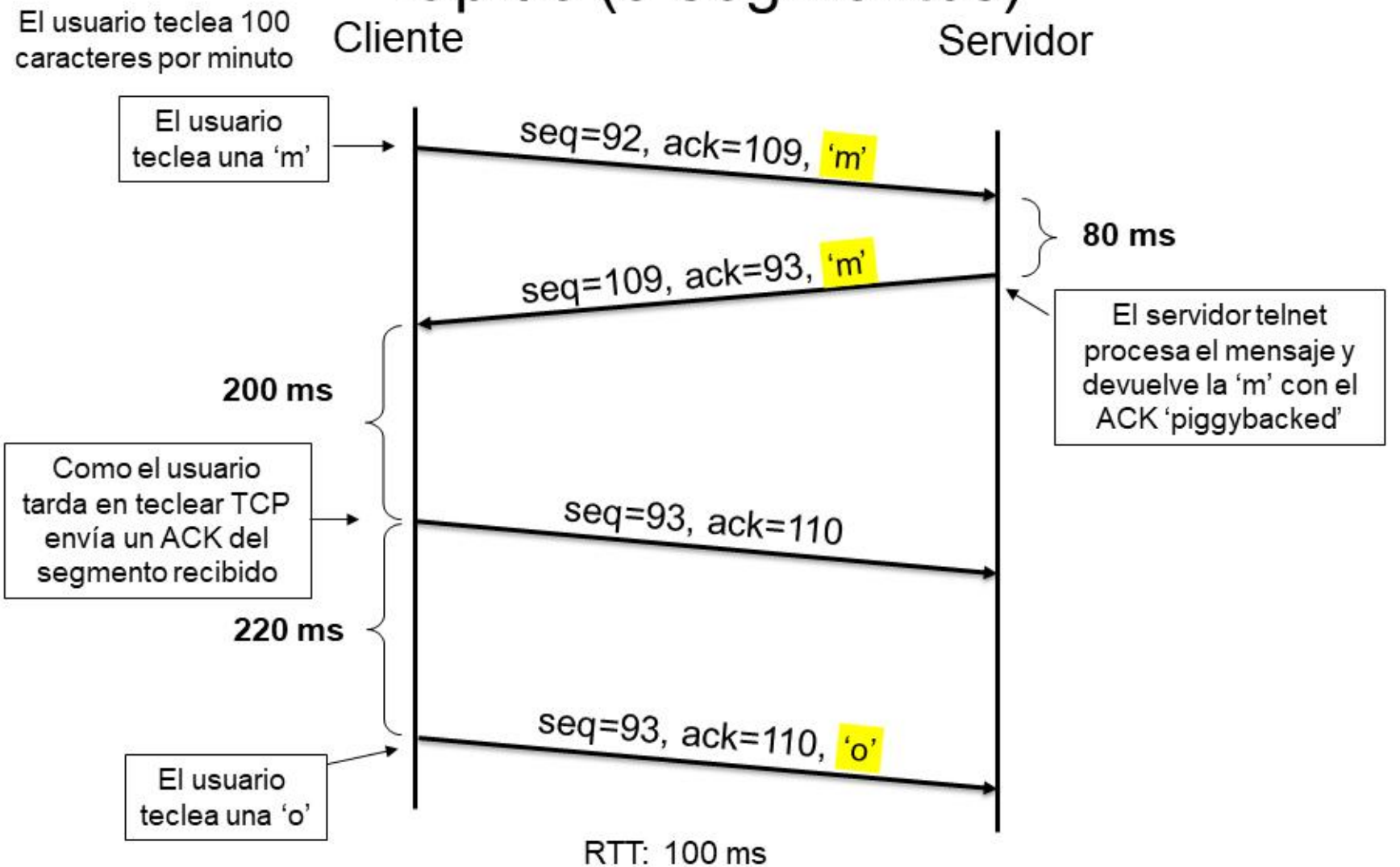
# Caso 1: usuario y servidor rápidos (2 segmentos)



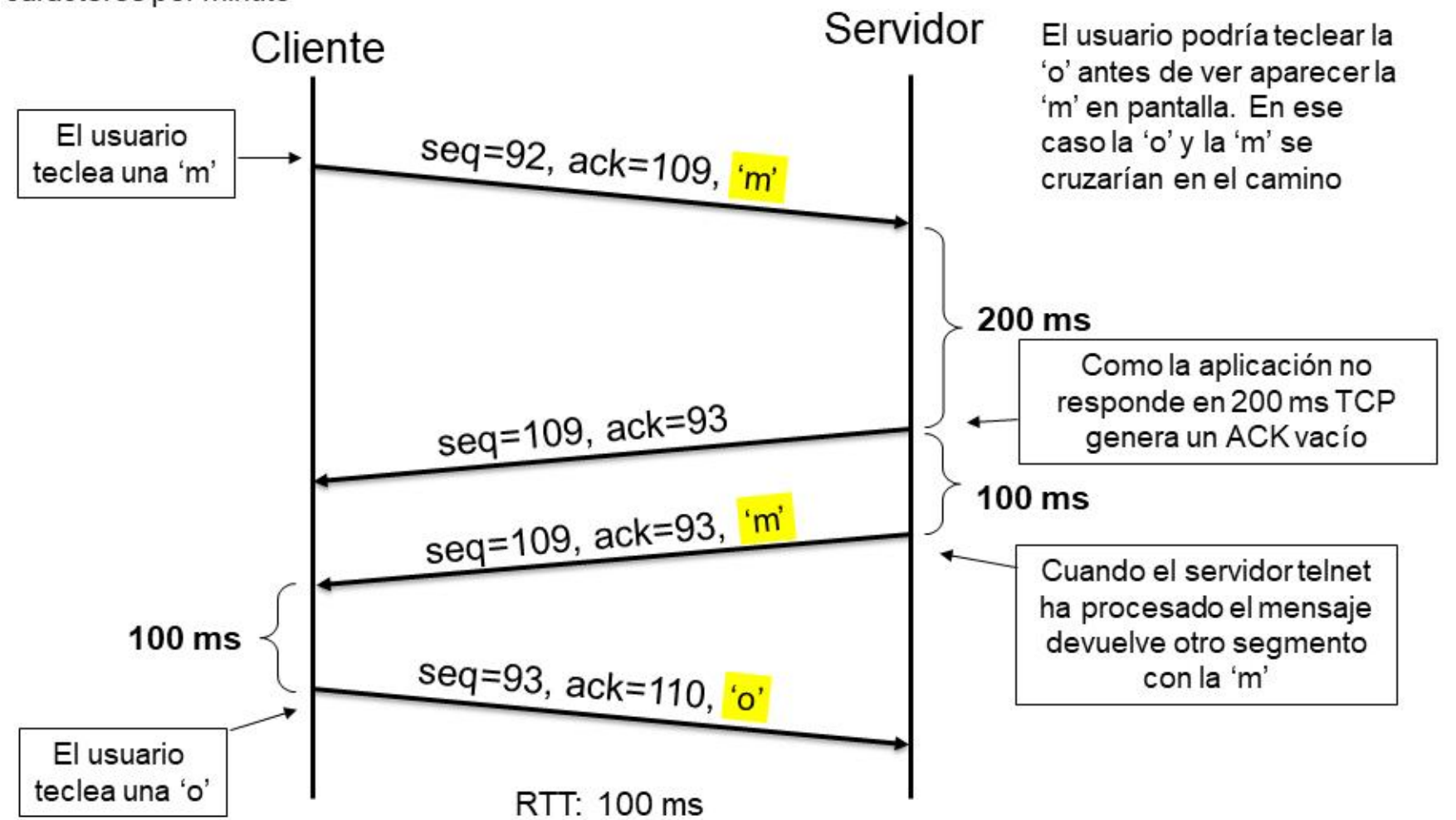
# Timer de ACK retrasado

- Cuando TCP recibe un segmento con datos y no tiene nada para enviar de vuelta no devuelve el ACK enseguida, en su lugar se espera un poco a ver si la aplicación genera datos y así aprovecha para enviar juntos el ACK y los datos. Decimos que en ese caso el ACK va 'piggybacked'.
- Ese tiempo de espera se conoce como 'timer de ACK retrasado'. En muchos sistemas suele ser de unos 200 ms.
- Si se agota el timer sin que la aplicación haya producido datos se envía el ACK en un segmento sin datos
- Si el usuario o el servidor no le entregan rápido a TCP el carácter a transmitir se ha de enviar un ACK vacío

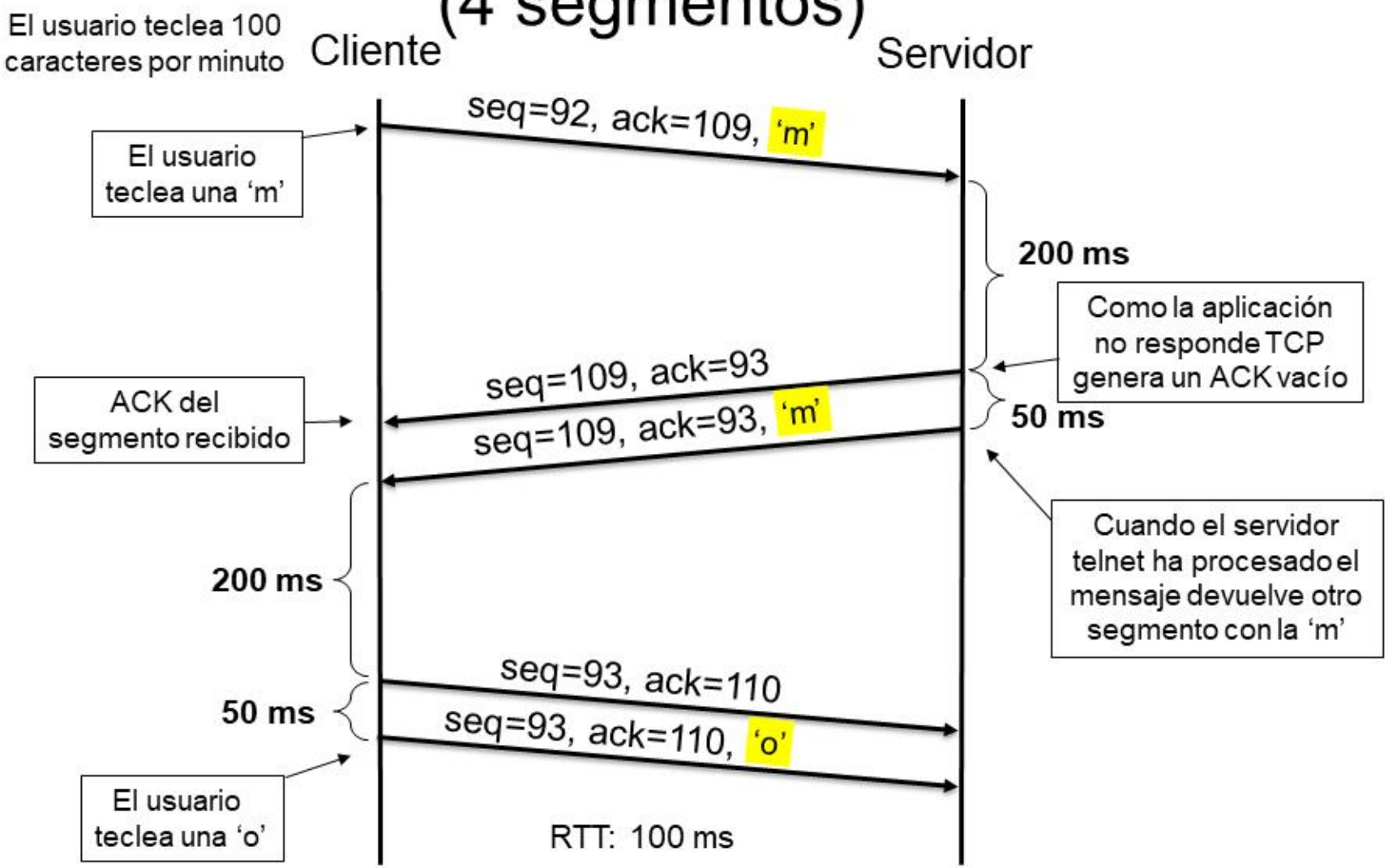
# Caso 2: usuario 'patoso', servidor rápido (3 segmentos)



# Caso 3: usuario rápido, servidor lento (3 segmentos)

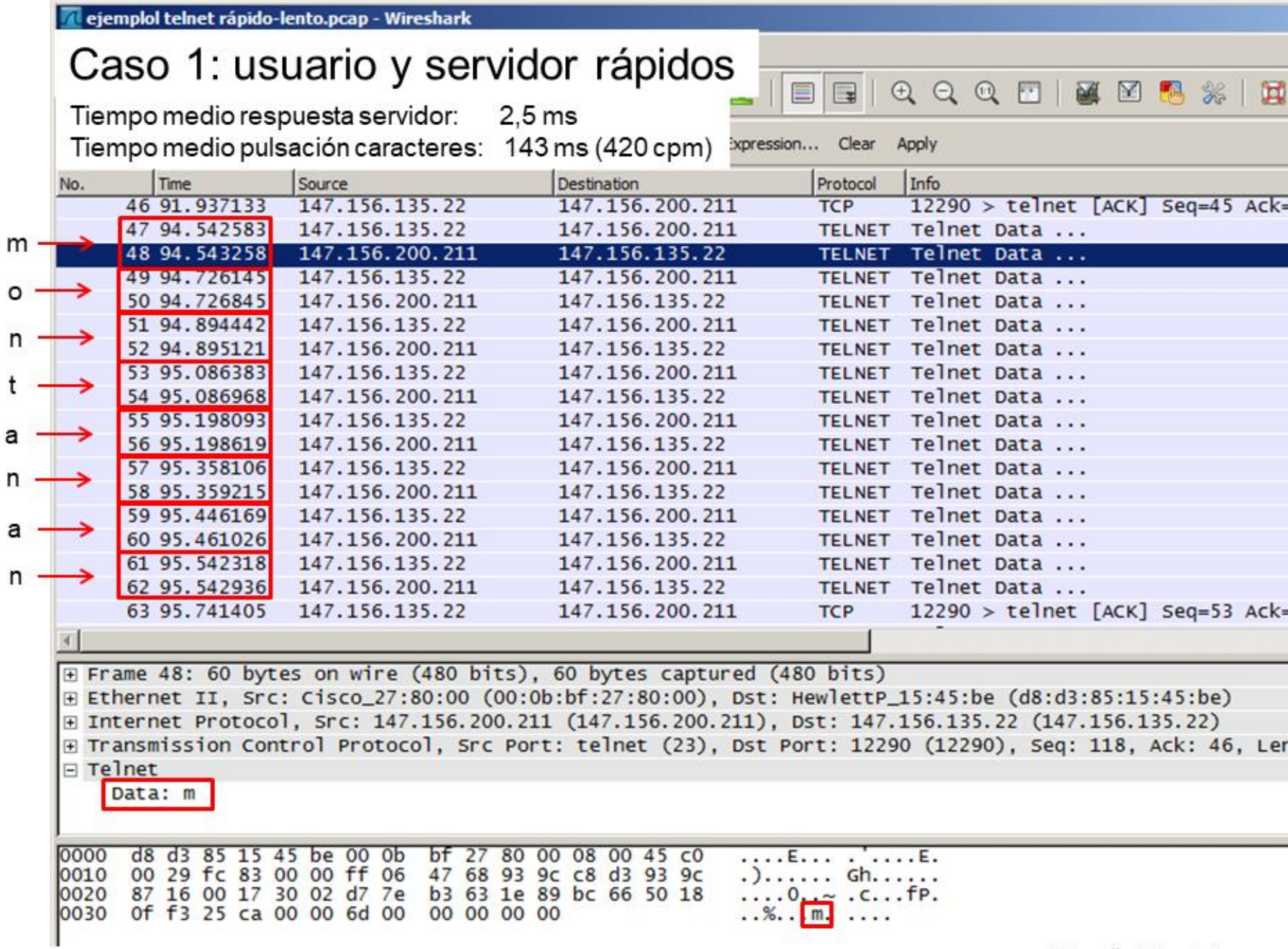


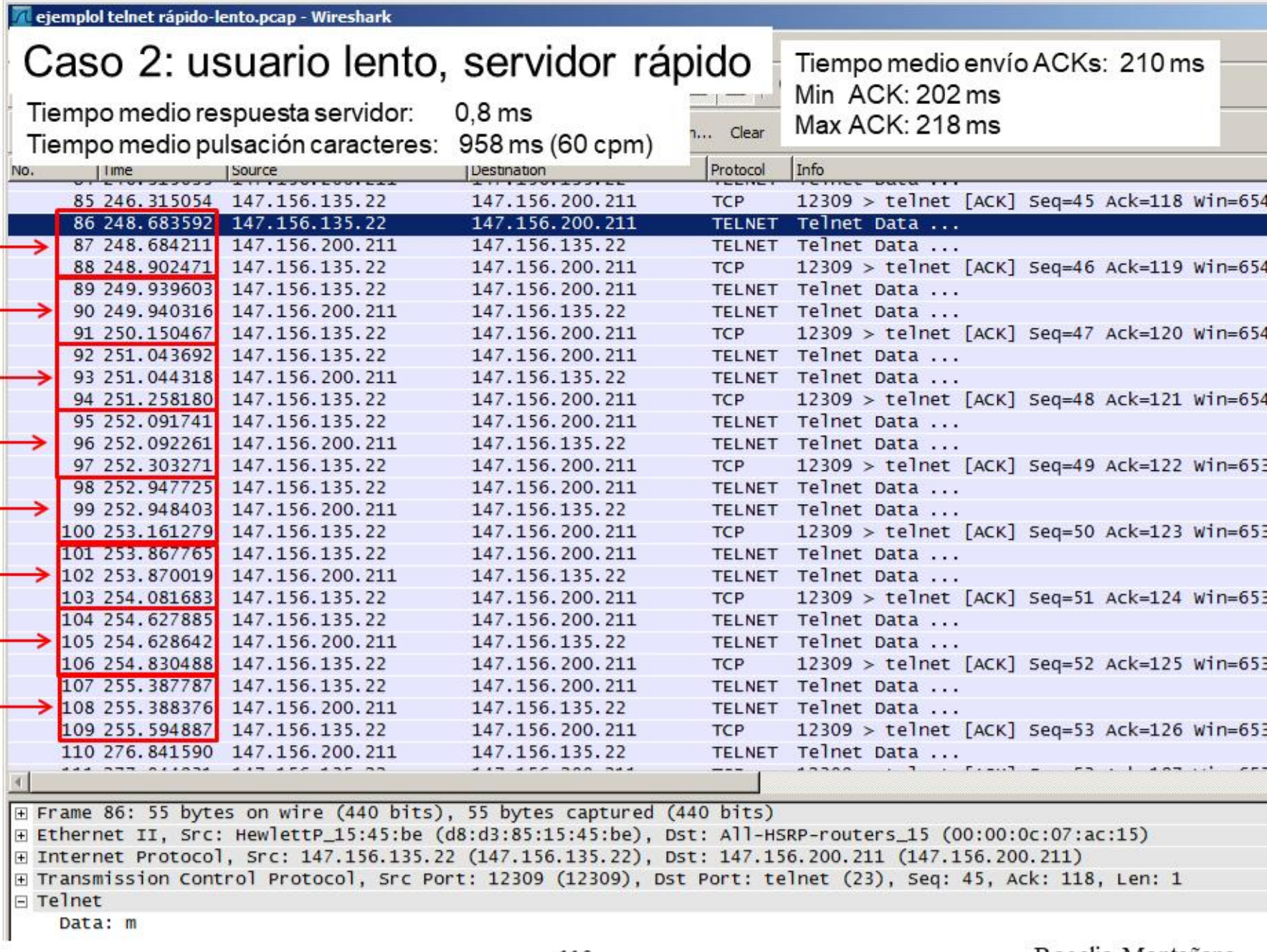
# Caso 4: usuario y servidor lentos (4 segmentos)



# Caso 1: usuario y servidor rápidos

Tiempo medio respuesta servidor: 2,5 ms
Tiempo medio pulsación caracteres: 143 ms (420 cpm)

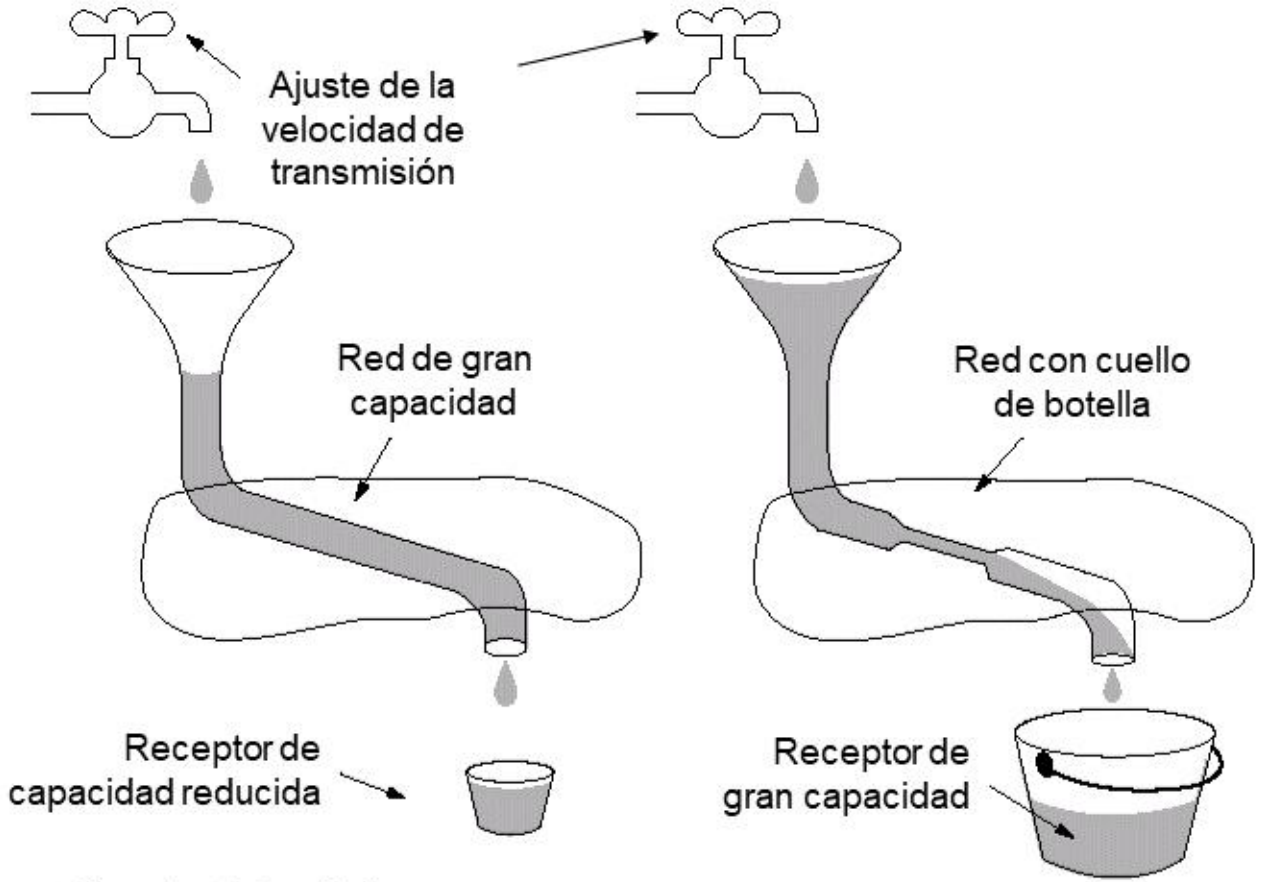



ejemplo1 telnet rápido-lento.pcap - Wireshark
Caso 2: usuario lento, servidor rápido
Tiempo medio respuesta servidor: 0,8 ms
Tiempo medio pulsación caracteres: 958 ms (60 cpm)
Tiempo medio envío ACKs: 210 ms
Min ACK: 202 ms
Max ACK: 218 ms
Clear
No. Time Source Destination Protocol Info
85 246.315054 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=45 Ack=118 Win=654
86 248.683592 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
m
87 248.684211 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
88 248.902471 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=46 Ack=119 Win=654
89 249.939603 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
o
90 249.940316 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
91 250.150467 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=47 Ack=120 Win=654
92 251.043692 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
n
93 251.044318 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
94 251.258180 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=48 Ack=121 Win=654
95 252.091741 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
t
96 252.092261 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
97 252.303271 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=49 Ack=122 Win=653
98 252.947725 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
a
99 252.948403 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
100 253.161279 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=50 Ack=123 Win=653
101 253.867765 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
n
102 253.870019 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
103 254.081683 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=51 Ack=124 Win=653
104 254.627885 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
a
105 254.628642 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
106 254.830488 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=52 Ack=125 Win=653
107 255.387787 147.156.135.22 147.156.200.211 TELNET Telnet Data ...
n
108 255.388376 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
109 255.594887 147.156.135.22 147.156.200.211 TCP 12309 > telnet [ACK] Seq=53 Ack=126 Win=653
110 276.841590 147.156.200.211 147.156.135.22 TELNET Telnet Data ...
Frame 86: 55 bytes on wire (440 bits), 55 bytes captured (440 bits)
Ethernet II, Src: HewlettP_15:45:be (d8:d3:85:15:45:be), Dst: All-HSRP-routers_15 (00:00:0c:07:ac:15)
Internet Protocol, Src: 147.156.135.22 (147.156.135.22), Dst: 147.156.200.211 (147.156.200.211)
Transmission Control Protocol, Src Port: 12309 (12309), Dst Port: telnet (23), Seq: 45, Ack: 118, Len: 1
Telnet
Data: m

# Tráfico masivo en TCP

- Los principales problemas del tráfico masivo son:
  - La saturación del receptor, que podría agotar el espacio disponible en su buffer. Si esto ocurriera el receptor se vería obligado a descartar datos. Para evitarlo TCP incorpora un mecanismo de **control de flujo**.
  - La saturación en la red, que dejaría sin espacio en el buffer a algún router, conmutador u otro dispositivo intermedio. Si esto ocurriera el dispositivo se vería obligado a descartar datos. Para evitarlo TCP incorpora un mecanismo de **control de congestión**.

# Control de flujo y control de congestión




Control de flujo
El problema se da en el receptor

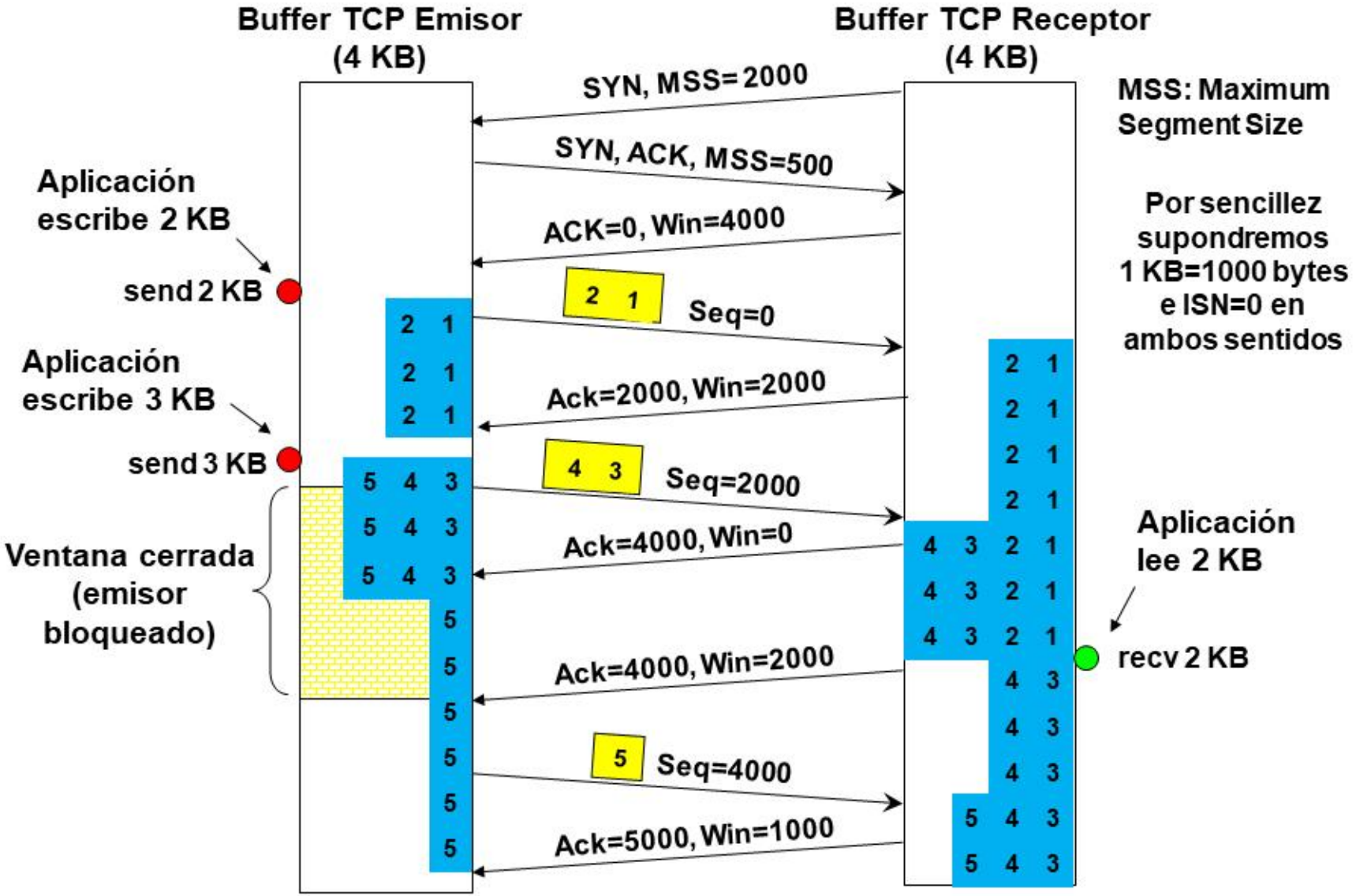
Control de congestión
El problema se da en la red

# Control de Flujo (gestión de buffers)




- En cada segmento que envía TCP informa del espacio que le queda libre en el buffer de recepción. Para ello usa el campo tamaño de ventana de la cabecera (16 bits).
- La ventana anunciada es el espacio que el TCP receptor tiene libre y reservado en exclusiva para esa conexión.
- Los tamaños de ventana se expresan en bytes, como los números de secuencia.
- Cuando un TCP recibe datos los coloca en el buffer y en cada ACK que envía le anuncia al otro el espacio remanente. Cuando el buffer se llena anuncia ventana cero y deja bloqueado al emisor. Decimos que ha **cerrado la ventana**.

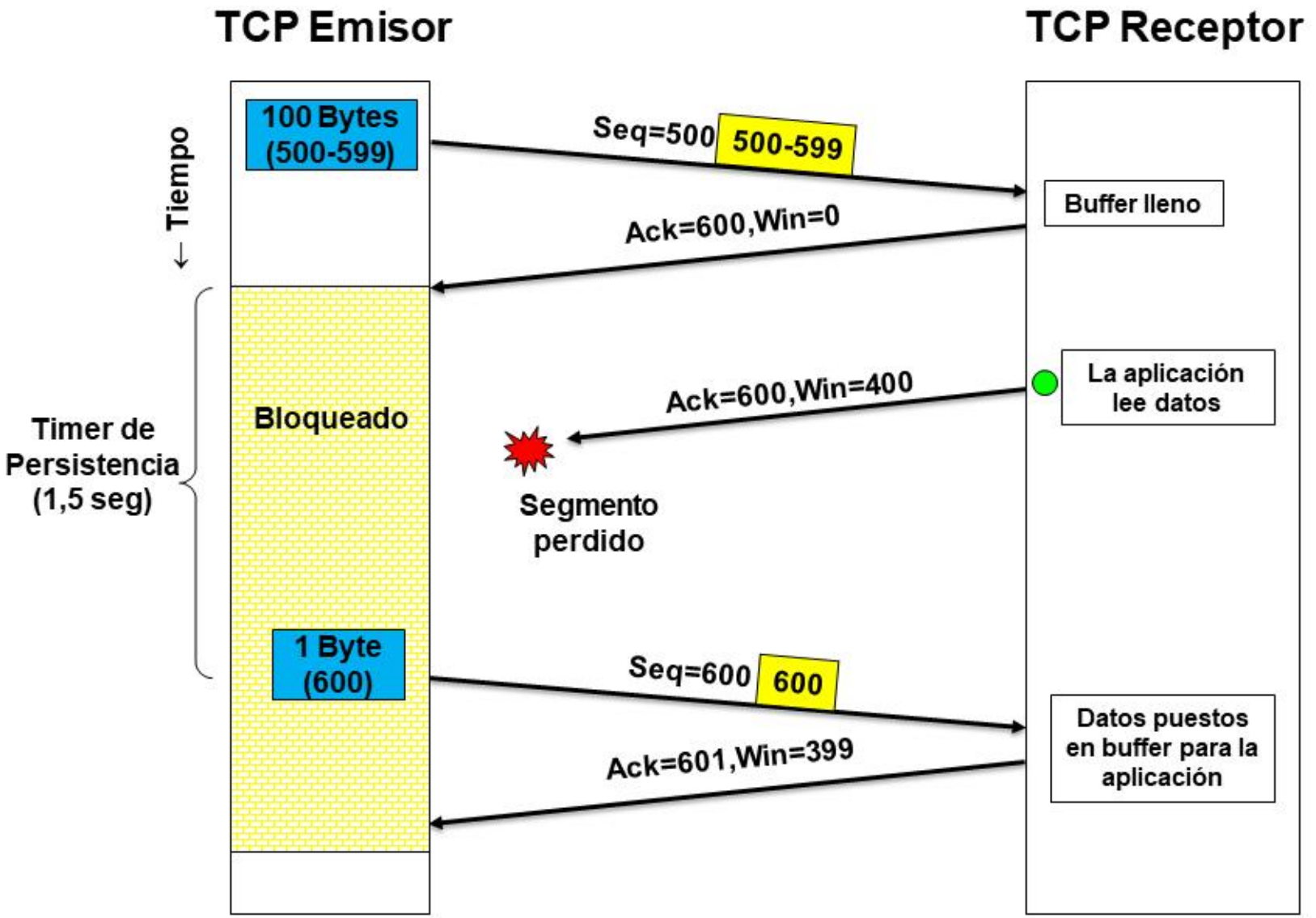
# Gestión de buffers y Control de flujo



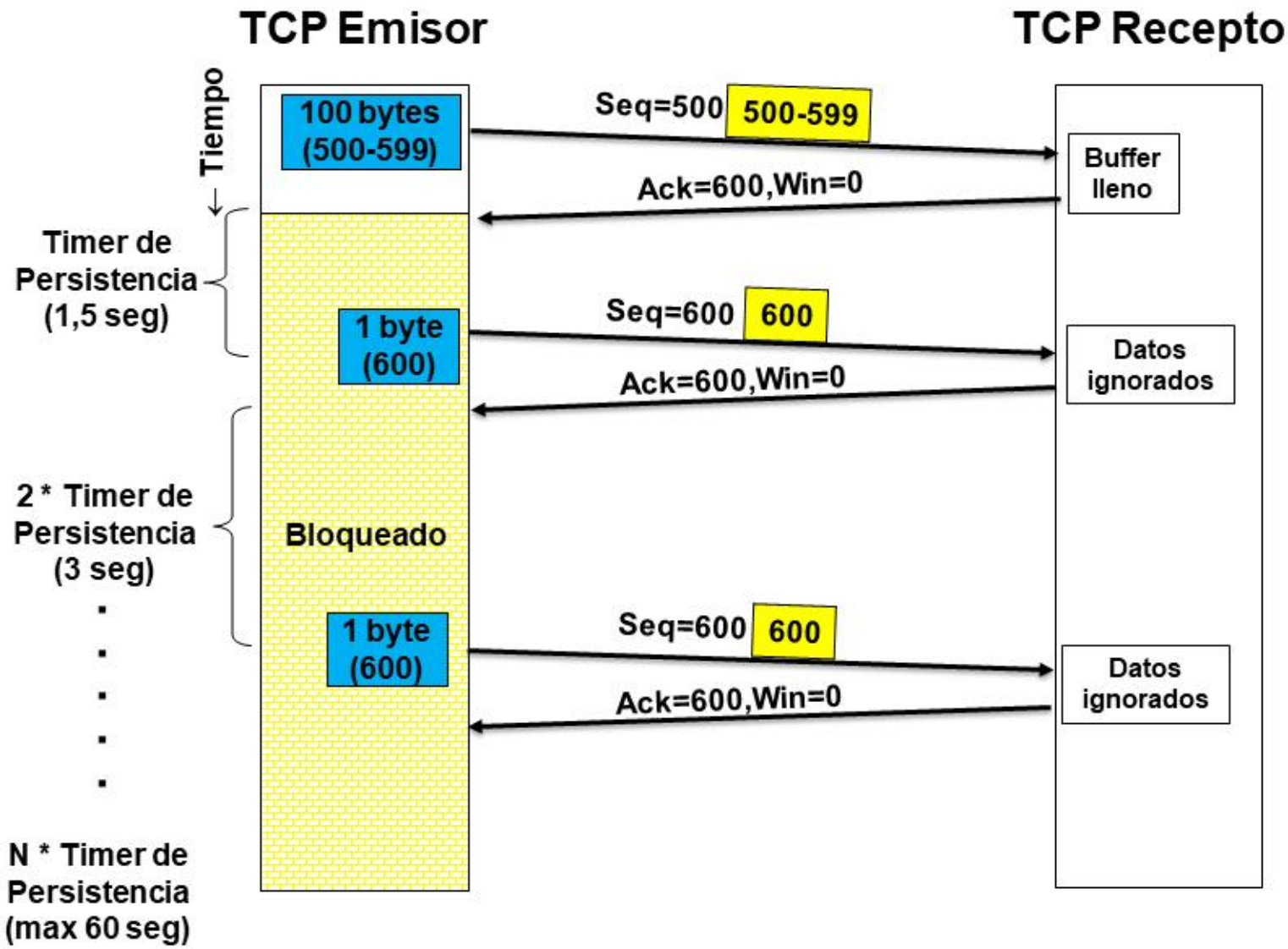
# Timer de Persistencia

- En TCP se puede dar una situación de bloqueo si después de anunciar ventana cero un TCP anuncia una ventana mayor, y el mensaje se pierde. Al ser ese anuncio un segmento sin datos no genera ACK por lo que el emisor no se entera de la pérdida
- Para evitarlo, cuando a un TCP le han cerrado la ventana envía de vez en cuando un segmento 'sonda' con el siguiente byte de datos; al llevar datos el receptor está obligado a enviar un ACK, de modo que el emisor confirma si la ventana sigue cerrada (o no)
- La frecuencia con que TCP envía los segmentos sonda es el 'Timer de Persistencia'.

# Timer de persistencia, anuncio perdido



# Timer de persistencia, receptor bloqueado



# Contracción de ventana

- Se dice que un TCP '**contrae la ventana**' cuando retira un espacio en buffer que había reservado para una conexión y que ya había anunciado al otro TCP.
- TCP nunca debe contraer la ventana
- Sin embargo cualquier TCP debe estar preparado por si el otro TCP le contrae la ventana de repente

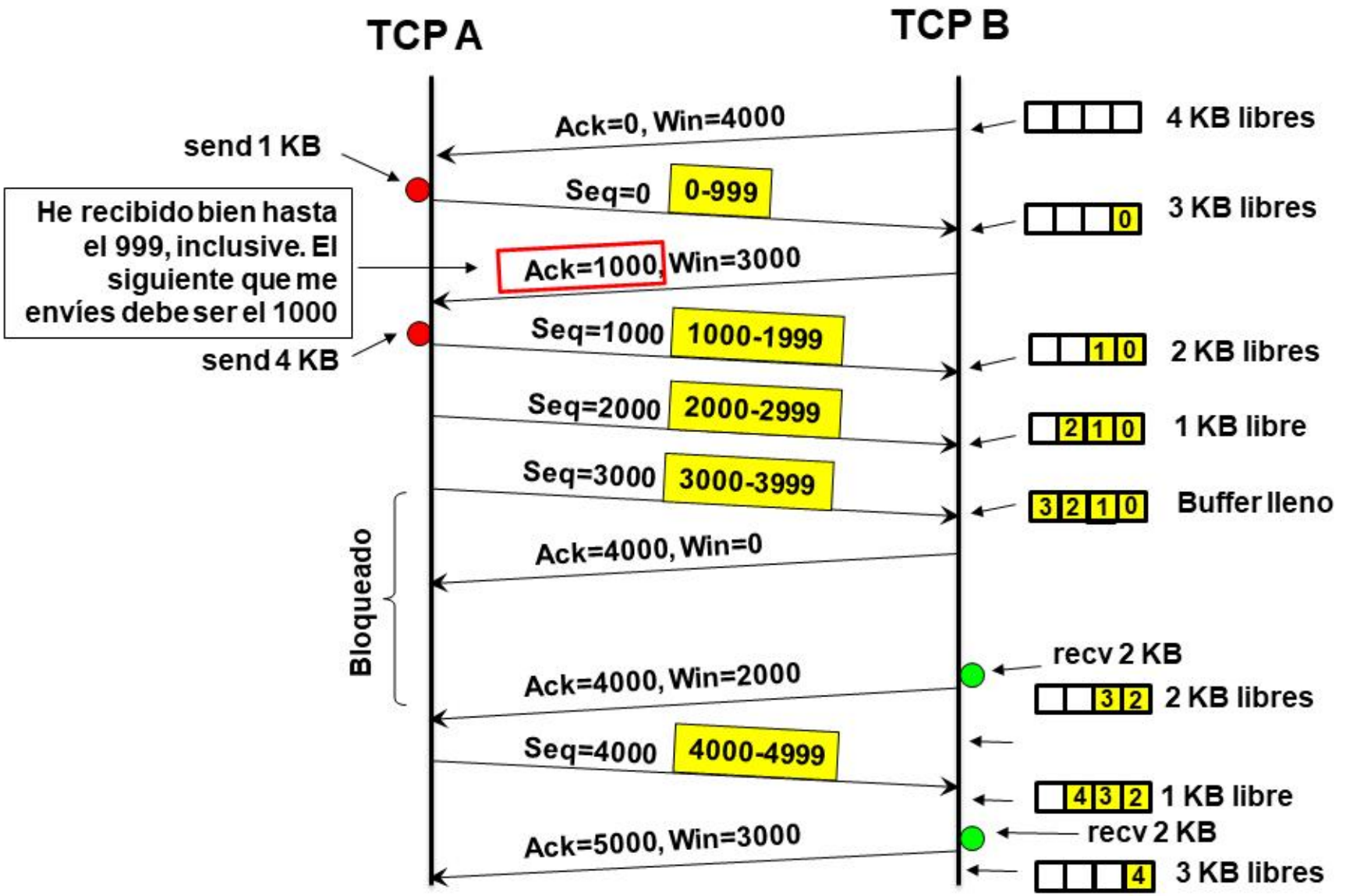
Recordemos la Ley de Postel:

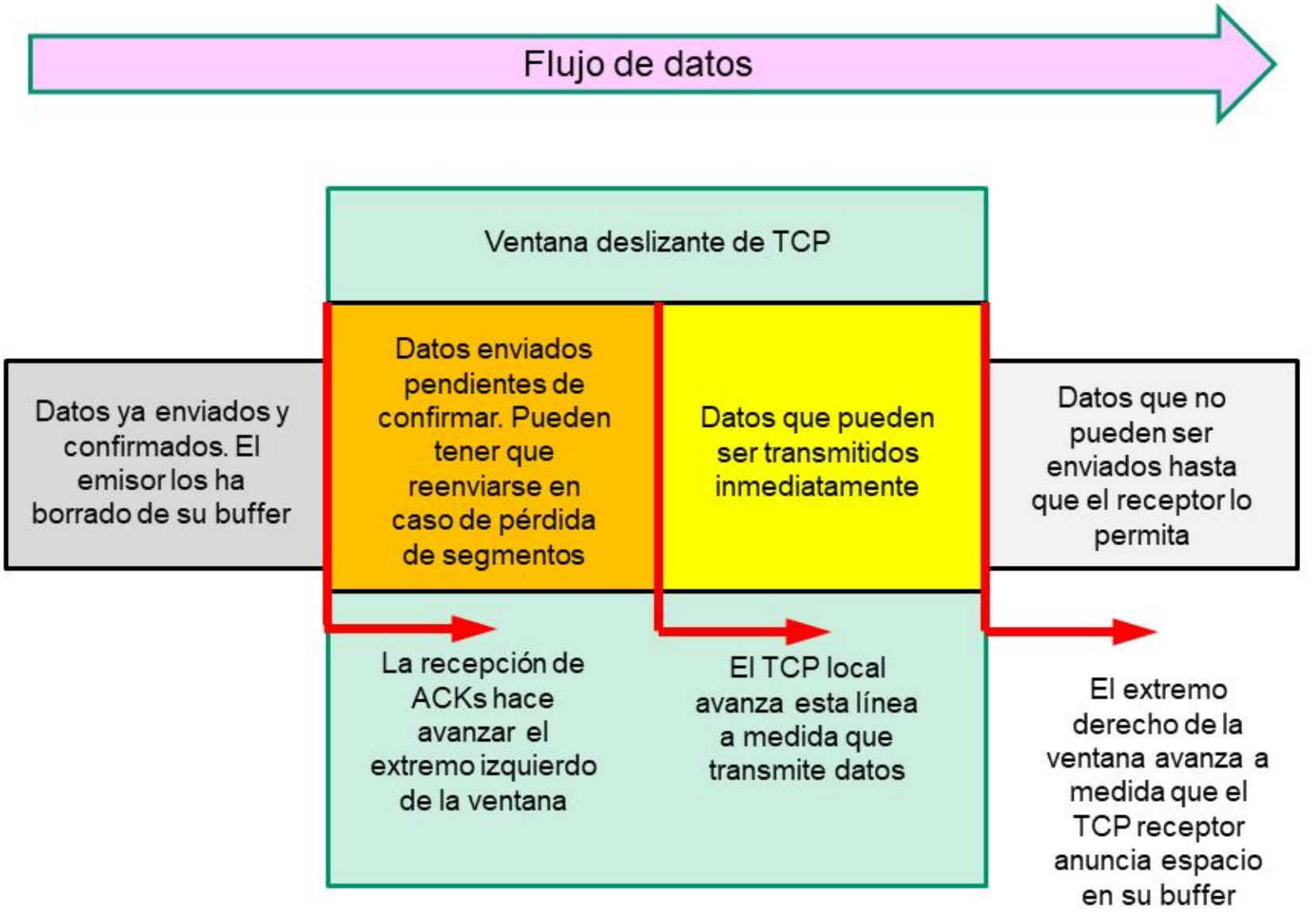
*'Sé estricto al enviar y tolerante al recibir'*

# Funcionamiento en 'pipeline'

- Para mejorar la eficiencia TCP emplea un mecanismo de 'pipeline' con ventana deslizante, es decir puede enviar varios segmentos en ráfaga, sin necesidad de esperar el ACK de uno para enviar el siguiente
- El 'pipeline' mejora notablemente el rendimiento cuando se hacen transferencias masivas de datos, especialmente en redes con elevado RTT (Round Trip Time) pues se evita tener parada la conexión

# Pipeline de TCP

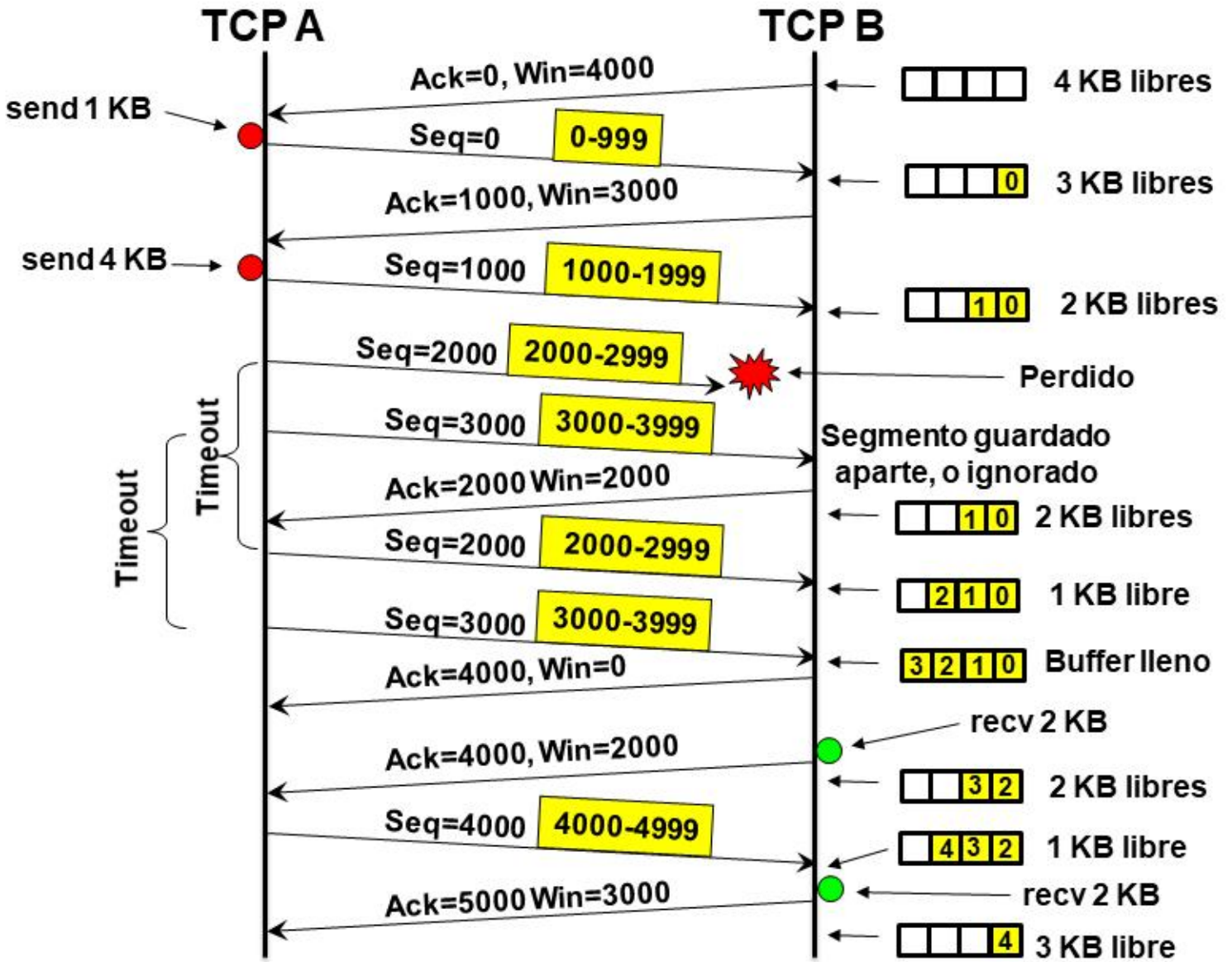



Flujo de datos
Ventana deslizante de TCP
Datos ya enviados y confirmados. El emisor los ha borrado de su buffer
Datos enviados pendientes de confirmar. Pueden tener que reenviarse en caso de pérdida de segmentos
Datos que pueden ser transmitidos inmediatamente
Datos que no pueden ser enviados hasta que el receptor lo permita
La recepción de ACKs hace avanzar el extremo izquierdo de la ventana
El TCP local avanza esta línea a medida que transmite datos
El extremo derecho de la ventana avanza a medida que el TCP receptor anuncia espacio en su buffer

# Pérdida y reenvío de segmentos

- Un paquete IP puede perderse por diversos motivos. En ese caso el segmento TCP correspondiente no llegará a su destino
- Cuando TCP envía un segmento espera recibir el ACK dentro de un tiempo razonable (timeout); si no lo recibe reenvía el segmento.
- Si un segmento llega al TCP de destino pero su checksum no es correcto se descarta y se actúa como si se hubiera perdido, es decir no se envía ningún mensaje.

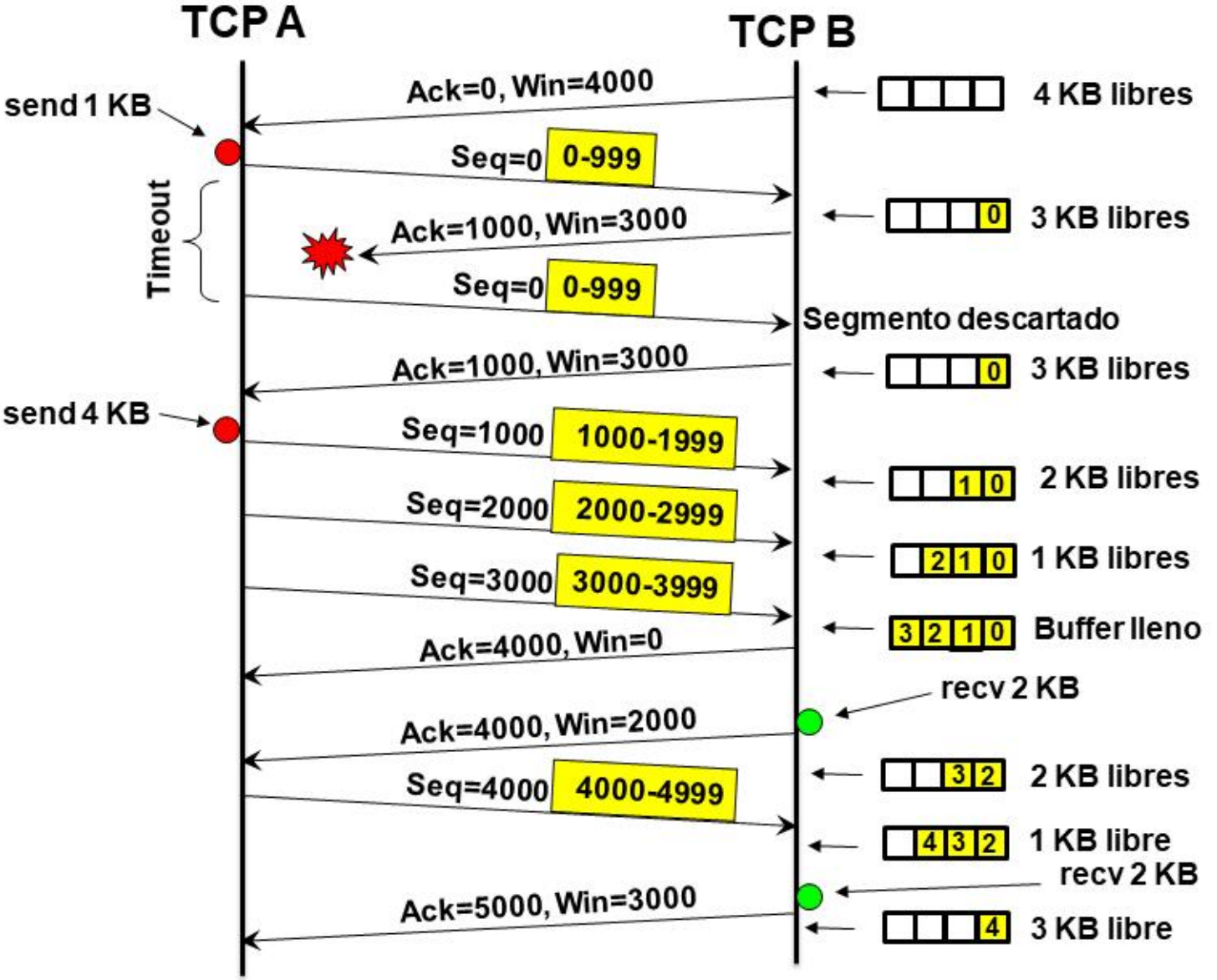
# Pérdida de un segmento de datos



# Pérdida del ACK

- También es posible que no se pierda el segmento de datos sino el ACK correspondiente
- Desde el punto de vista del TCP emisor ambas cosas son equivalentes, pues no sabe cual de los dos se ha perdido. Por tanto reenvía el segmento de datos como en el caso anterior
- El TCP que recibe un segmento duplicado debe:
  - Descartarlo y no ponerlo en el buffer de lectura (ya que si lo hiciera la aplicación recibiría datos duplicados)
  - Reiterar el ACK al emisor, de lo contrario este no parará de insistir

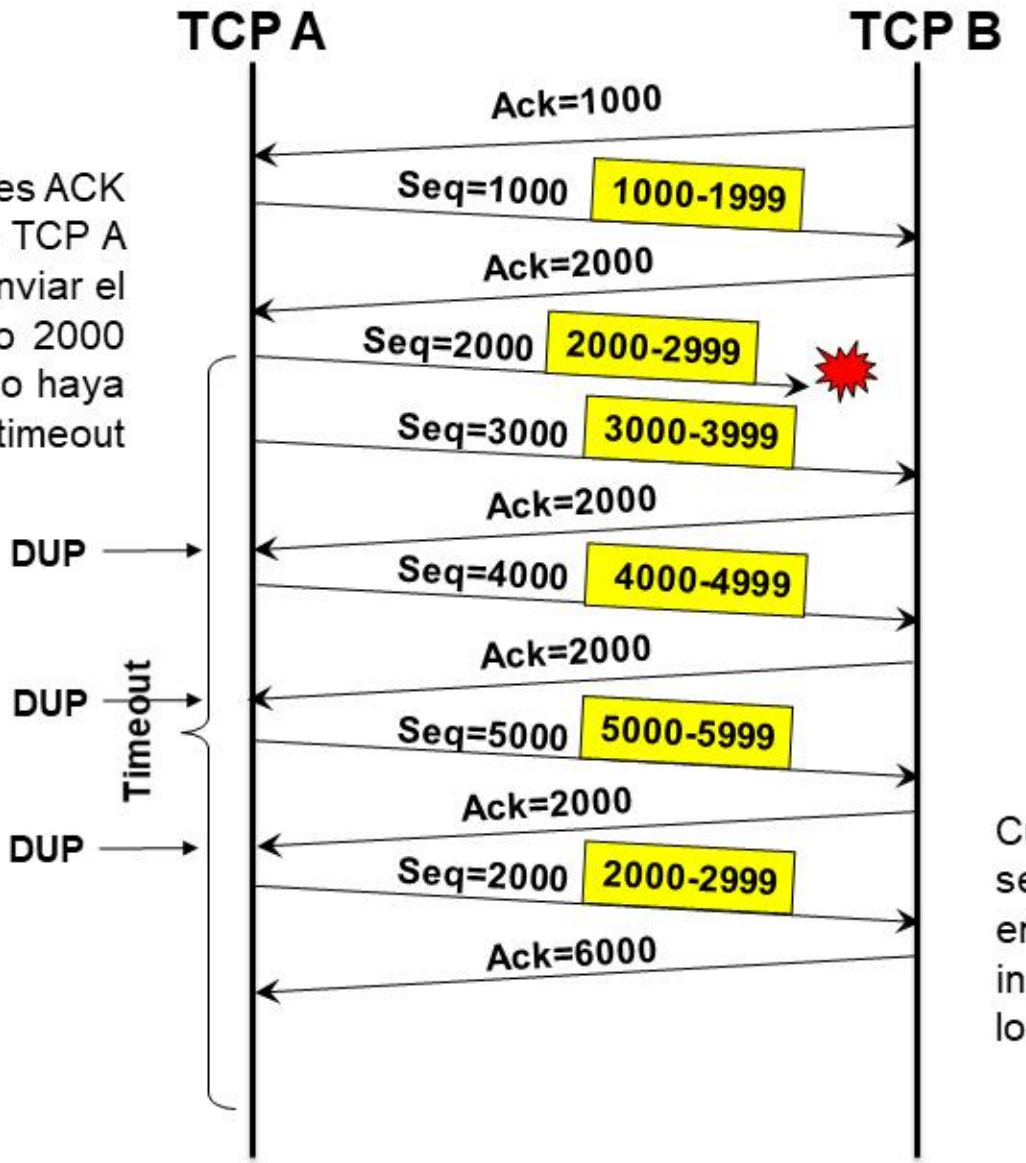
# Pérdida de ACK



# Fast retransmit

- Cuando TCP recibe un segmento cuyo número de secuencia no es el siguiente en la secuencia, sino uno posterior, envía un ACK duplicado del último segmento recibido en secuencia.
- El TCP emisor interpreta el ACK duplicado como un aviso de que el segmento faltante no ha llegado, lo cual puede ser debido a que:
  - El segmento faltante se ha perdido
  - El segmento faltante va por una ruta más larga y llegará más tarde
- Cuando el TCP emisor recibe tres ACK duplicados de un mismo segmento (4 en total) considera que el segmento faltante se ha perdido, y lo reenvía sin esperar a agotar su timeout

# Fast retransmit en acción




Al recibir tres ACK duplicados TCP A decide reenviar el segmento 2000 aunque aun no haya agotado su timeout

Cuando recibe el segmento faltante TCP envía un ACK que incluye de golpe ese y los tres siguientes

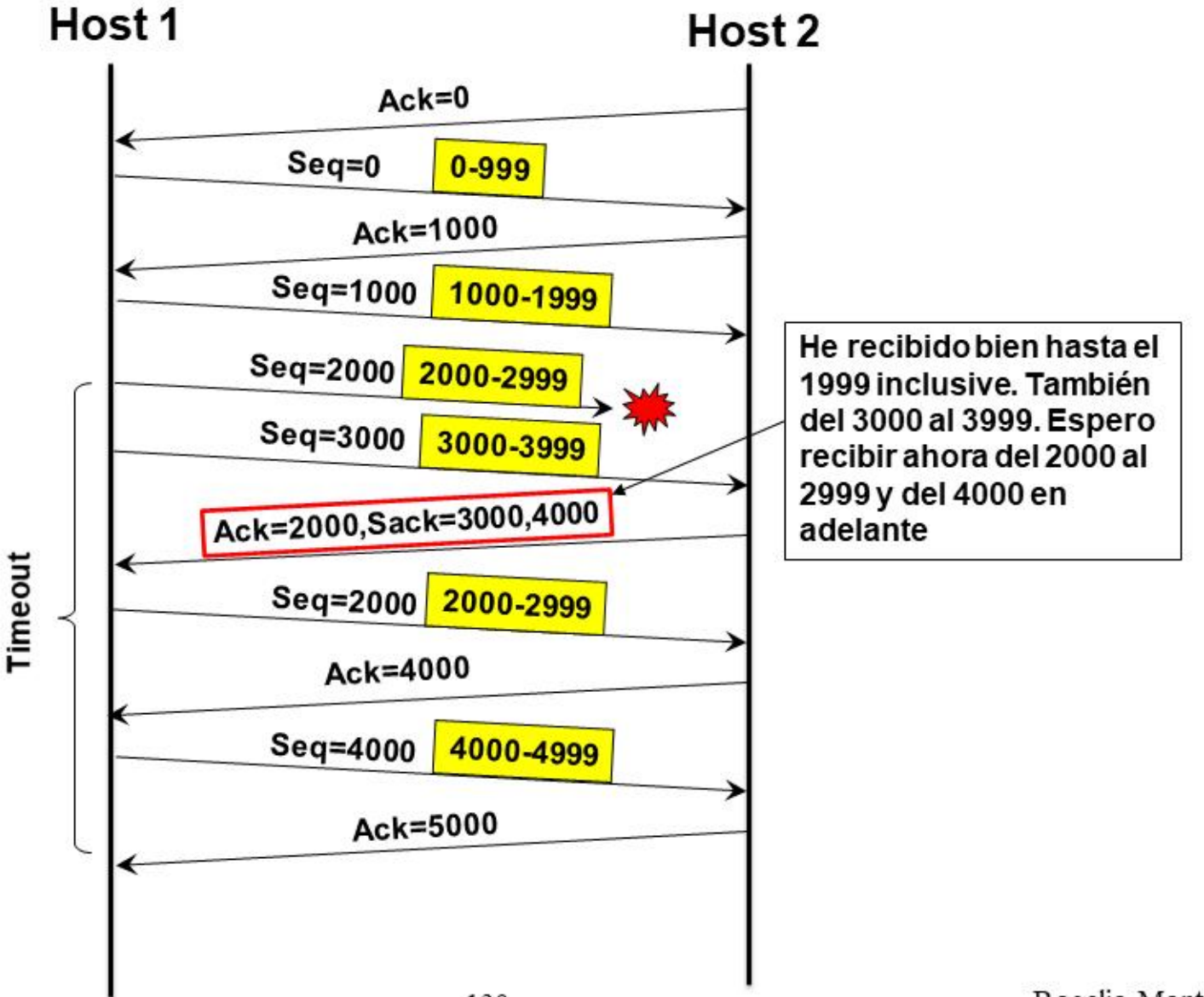
# Acuse de recibo selectivo (SACK)

- Al principio TCP requería que los ACK fueran acumulativos, de forma que si se perdía un segmento no se podía enviar ACK de los siguientes, aunque hubieran llegado bien.
- En la práctica esto suponía que cuando se perdía un segmento a veces se tenía que reenviar todo a partir de ese punto, aunque algunos segmentos posteriores ya hubieran llegado bien
- El RFC 2018 (1996) estableció la opción SACK (Selective ACK) que permite acusar el recibo de 'retales' (segmentos no consecutivos)

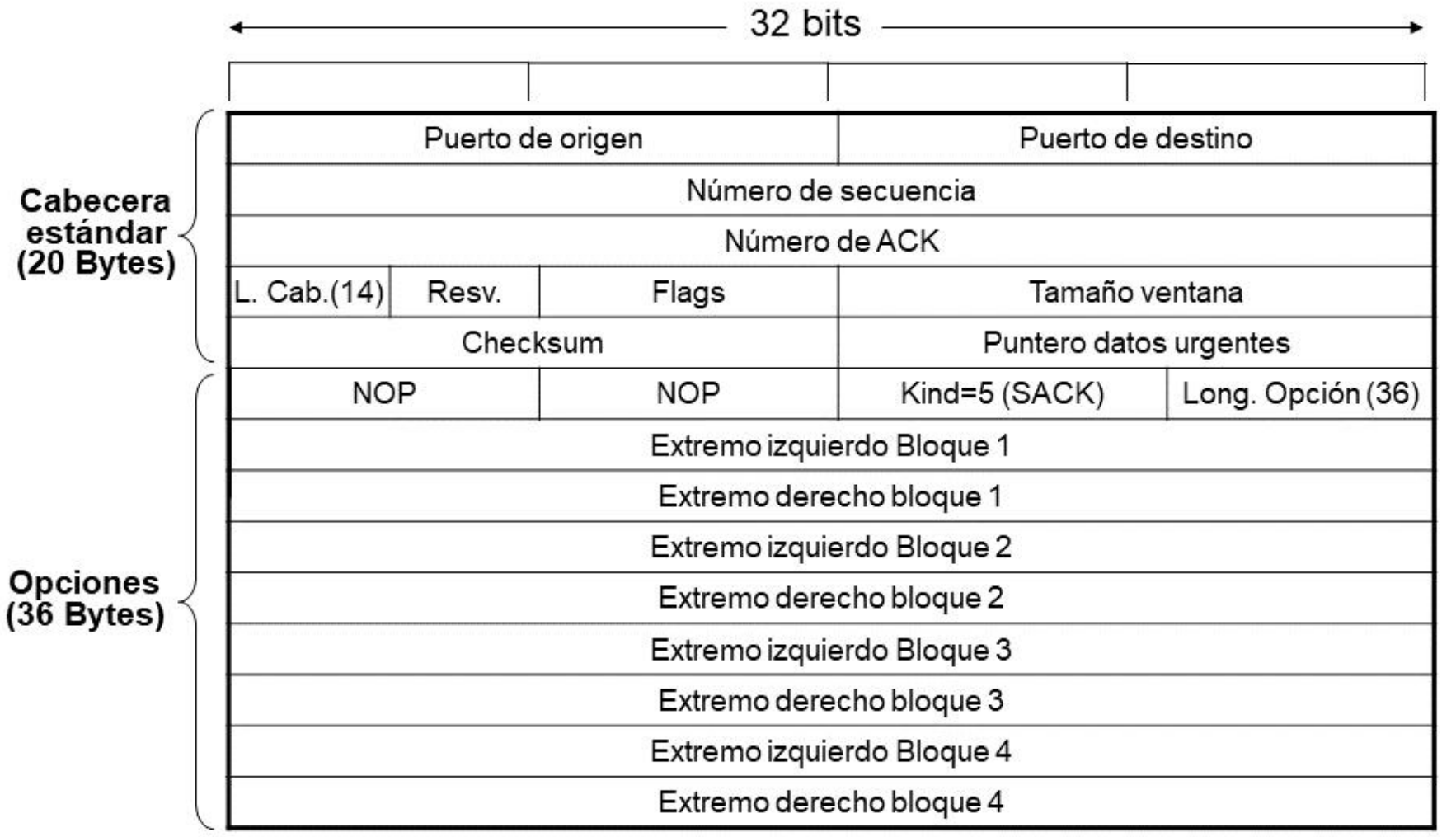
# Opciones SACK (Selective ACK)

- Para el uso del SACK TCP utiliza dos opciones:
  - La opción 'SACK-permitted' se utiliza en el segmento SYN para indicarle al otro TCP que soportamos la opción SACK
  - La opción 'SACK' se utiliza para indicar la recepción de segmentos no consecutivos, e indicar que rango del flujo de bytes comprenden esos segmentos. SACK sólo puede usarse si previamente ambos TCP han anunciado el 'SACK-permitted'.
- Con SACK se pueden especificar hasta cuatro rangos de números de secuencia (principio, fin) que indican datos recibidos por encima del valor indicado en el campo ACK

# SACK en acción



# Cabecera TCP con opción SACK y cuatro retales



# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - **Mejoras de rendimiento**
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

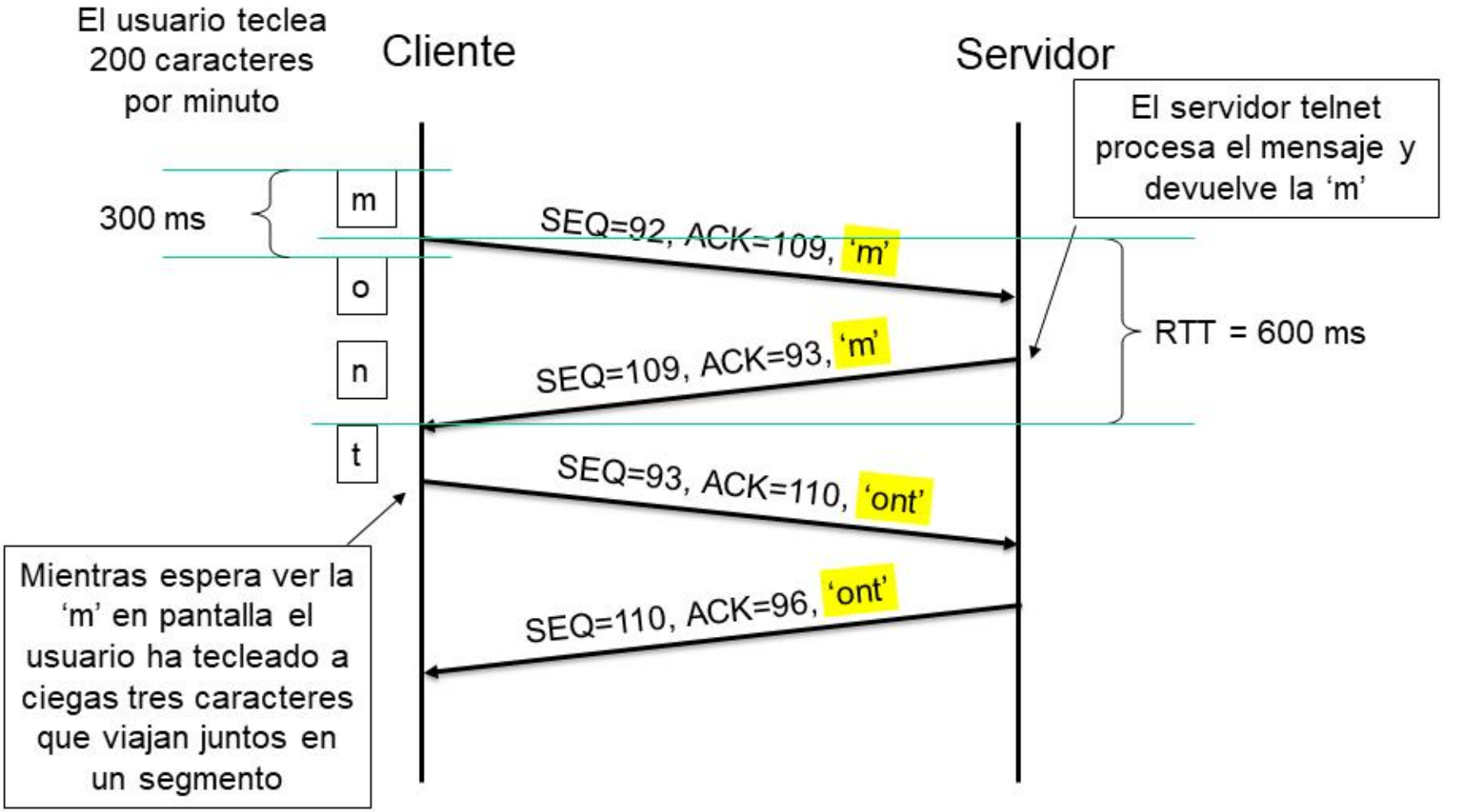
# Rendimiento de TCP

- En transferencias masivas de datos (Ej. FTP) no suele haber problemas de rendimiento, ya que se manejan segmentos grandes.
- Los problemas surgen sobre todo con aplicaciones, como telnet, que generan paquetes pequeños ('tinygrams'). Para evitarlo se utilizan varios mecanismos:
  - El **algoritmo de Nagle**: Mejora la eficiencia cuando el emisor escribe datos en el buffer poco a poco
  - La **solución de Clark**: Ayuda cuando el receptor lee datos del buffer en pequeñas dosis (síndrome de la ventana tonta)
- También se utilizan los **mensajes de Keepalive** para evitar que se queden conexiones medio abiertas

# Algoritmo de Nagle

- El algoritmo de Nagle se especificó en 1984 (RFC 896)
- Consiste en evitar que TCP envíe datos en cuanto tenga algo en el buffer, si ve que se van a generar segmentos pequeños. Antes de mandar un segmento espera a recibir el ACK del segmento anterior. Así agrupa los datos y los segmentos son más grandes.
- Puede aplicarse a datos 'pushed' en caso necesario.
- En algunos casos se deshabilita, por ejemplo si se utiliza el algoritmo de Nagle en X Windows o en conexiones de escritorio remoto se pierde interactividad y el movimiento del ratón resulta errático.

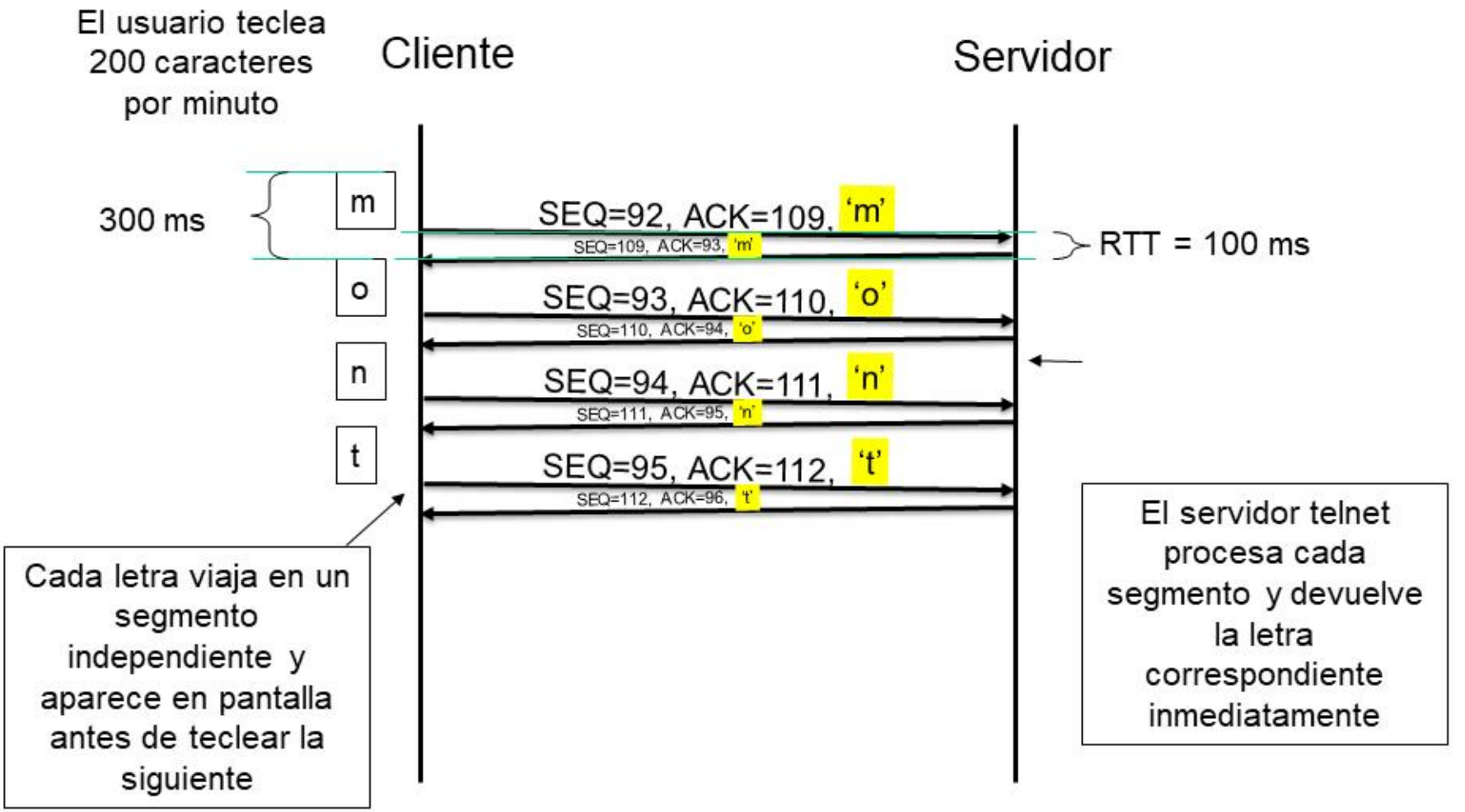
# Telnet con algoritmo de Nagle



# Adaptabilidad del algoritmo de Nagle

- El algoritmo de Nagle es autoadaptativo: con un RTT grande los ACK tardan en llegar y los segmentos son más grandes, dando lugar a un uso eficiente de la red
- Si el RTT es pequeño los ACK llegan enseguida y se crean segmentos pequeños. Con RTT pequeño la red está poco cargada, en este caso prima la agilidad y rápida respuesta, no tiene sentido hacer segmentos grandes.

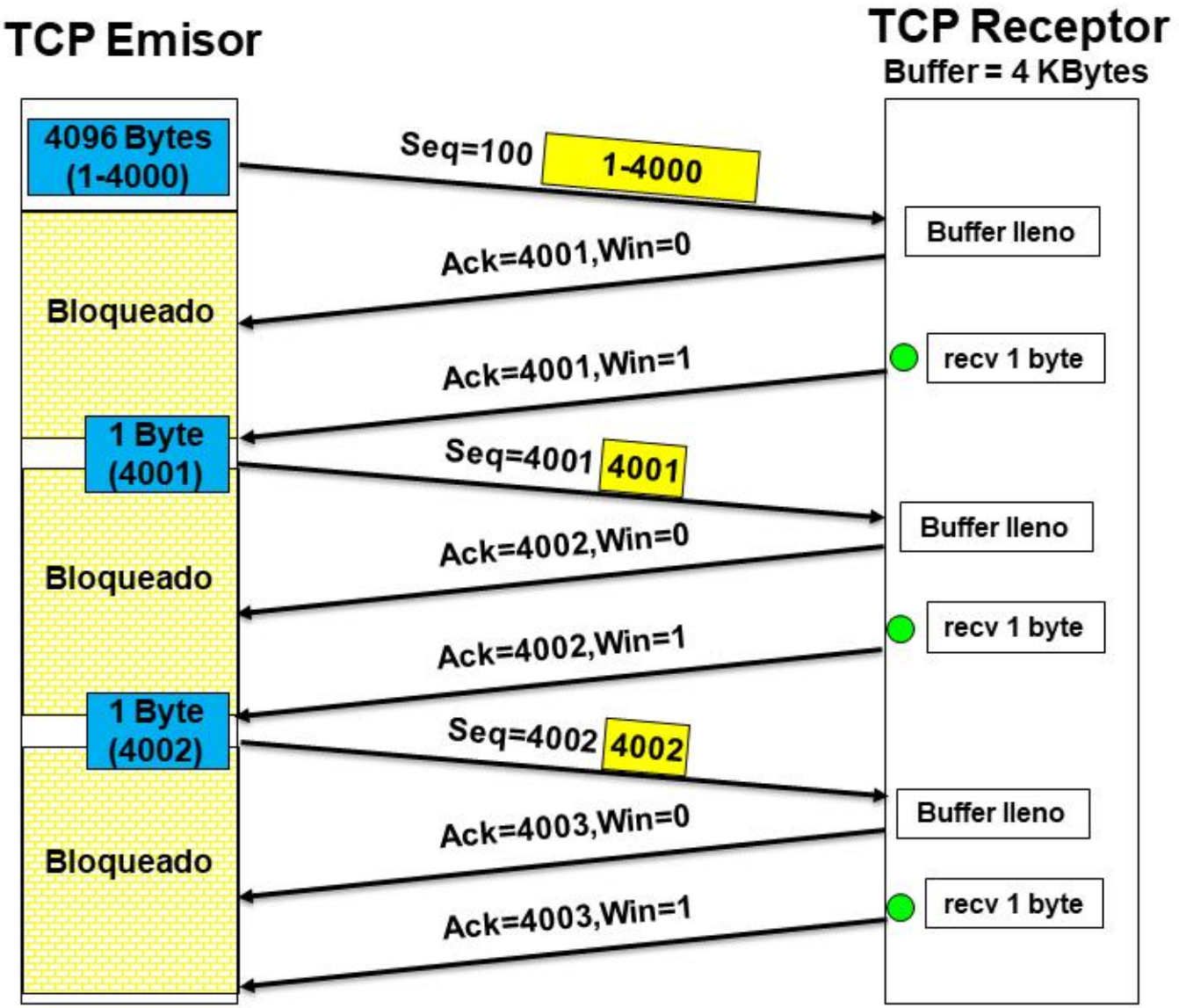
# Telnet con algoritmo de Nagle, RTT bajo



# Síndrome de la ventana tonta

- Ocurre cuando, estando lleno el buffer del TCP receptor, la aplicación lee los datos en pequeñas dosis. TCP anunciará cada vez que se produzca un hueco en el buffer y el emisor se verá obligado a enviar segmentos pequeños.
- La solución a este problema la planteó Clark en 1982 (RFC 813): el TCP receptor solo debe notificar una nueva ventana cuando tenga una cantidad razonable de espacio libre. Razonable significa que tiene al menos:
  - Un MSS (Maximum Segment Size), o
  - La mitad del espacio asignado en el buffer a esa conexión

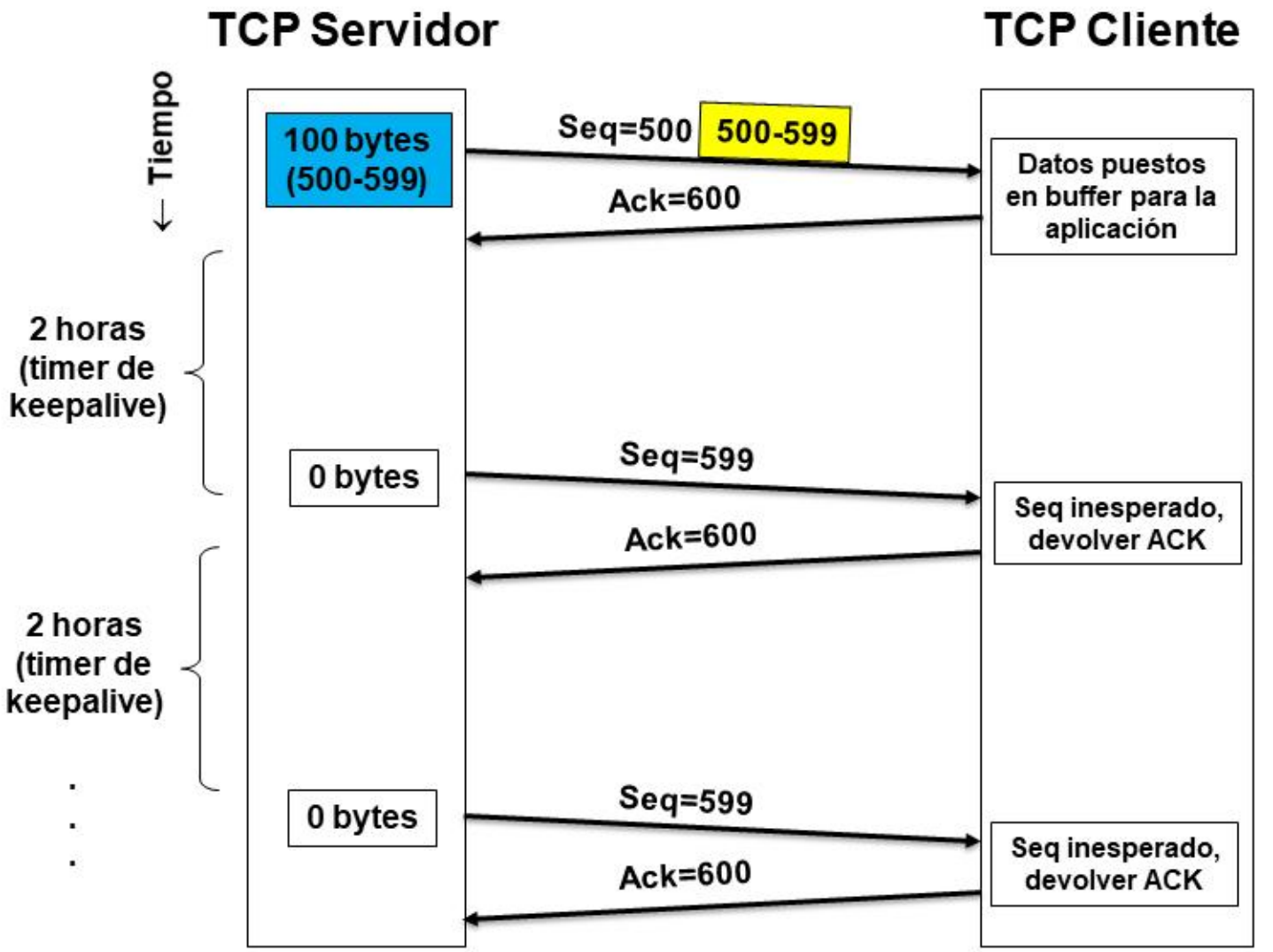
# Síndrome de la ventana tonta



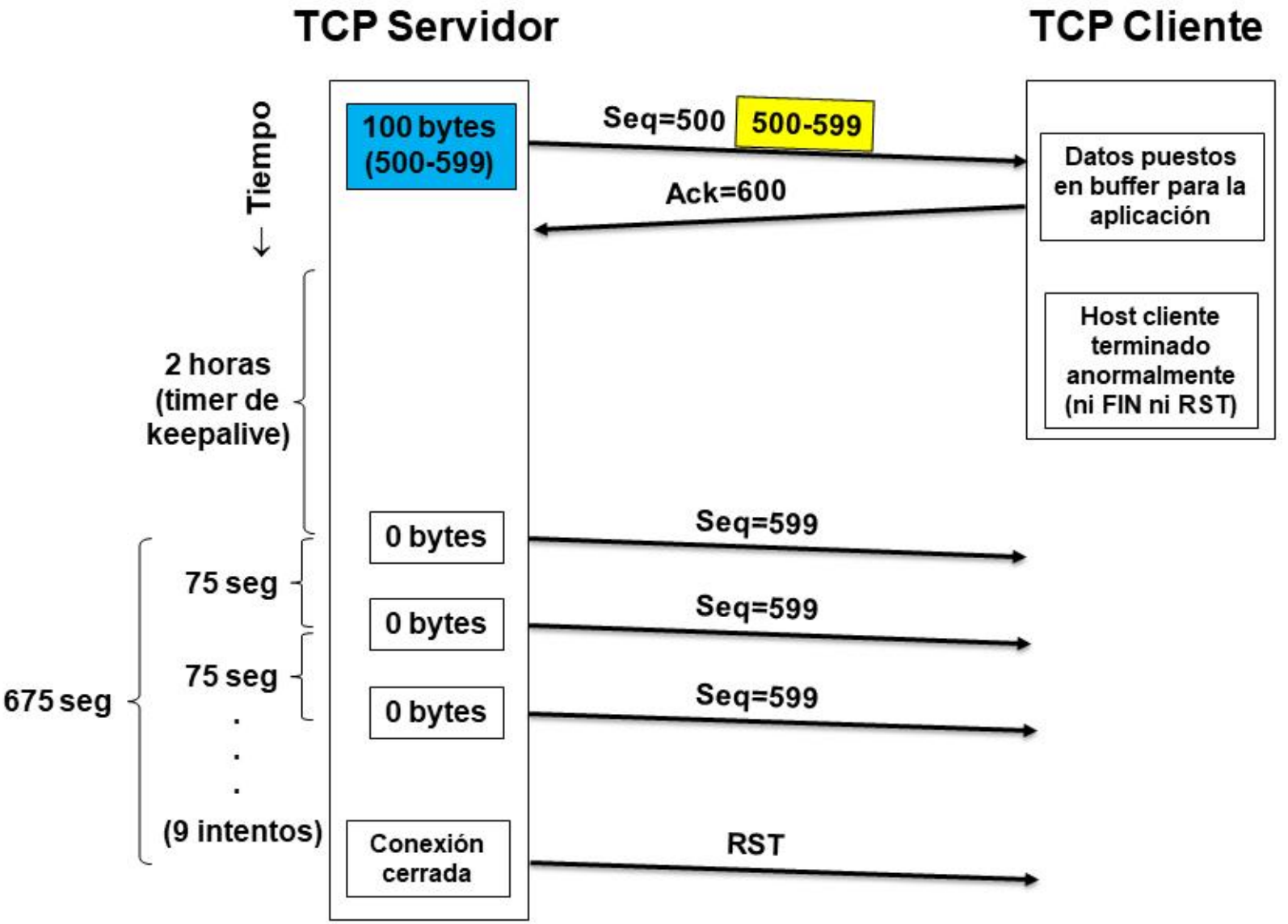
# Conexiones medio abiertas

- Cuando un TCP se marcha sin enviar un FIN ni un RST la conexión en el otro TCP se queda establecida. Esto es lo que se conoce como una conexión **medio abierta**.
- Las conexiones medio abiertas consumen recursos y conviene evitarlas. El problema es especialmente importante en servidores.
- Una conexión medio abierta puede pasar desapercibida si el turno de palabra depende del TCP que se ha ido sin despedirse. En TCP no hay un tráfico mínimo requerido, no se puede cortar una conexión por inactividad.
- TCP prevé un mecanismo para detectar las conexiones medio abiertas, que son los 'mensajes de keepalive'

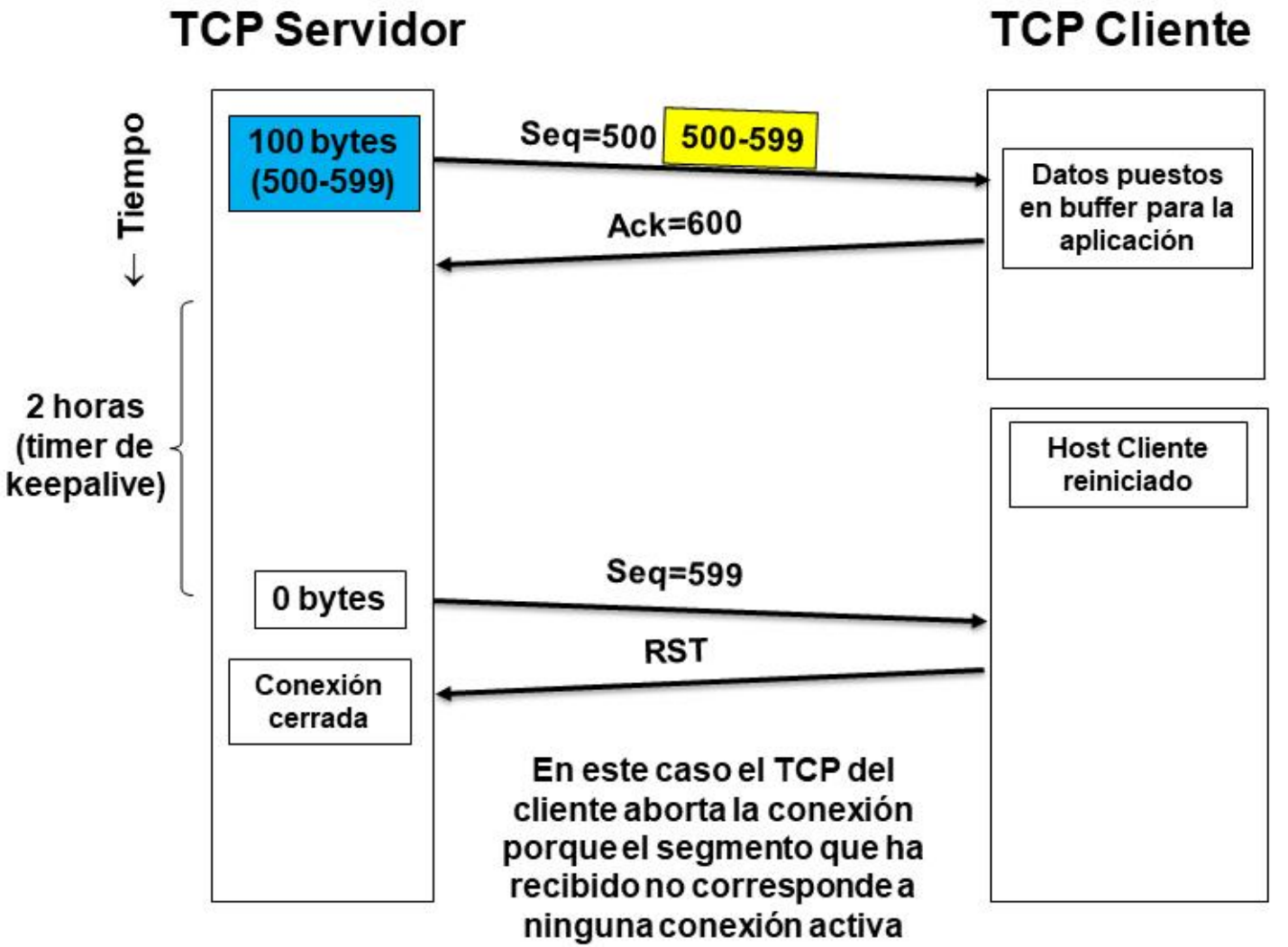
# Mensajes de keepalive, cliente vivo



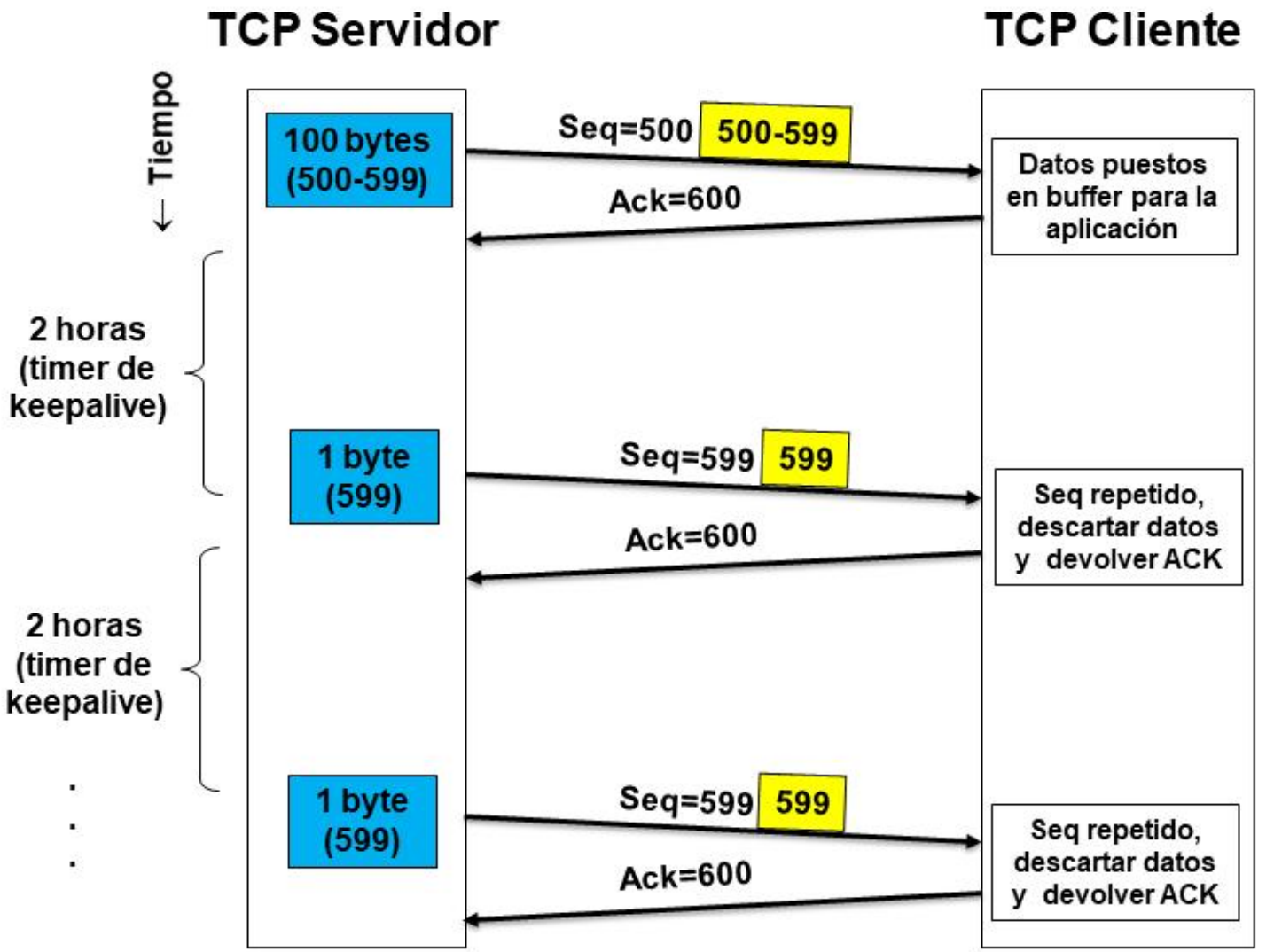
# Mensajes de keepalive, cliente muerto



# Mensajes de keepalive, cliente reiniciado



# Keepalive modo alternativo, cliente vivo



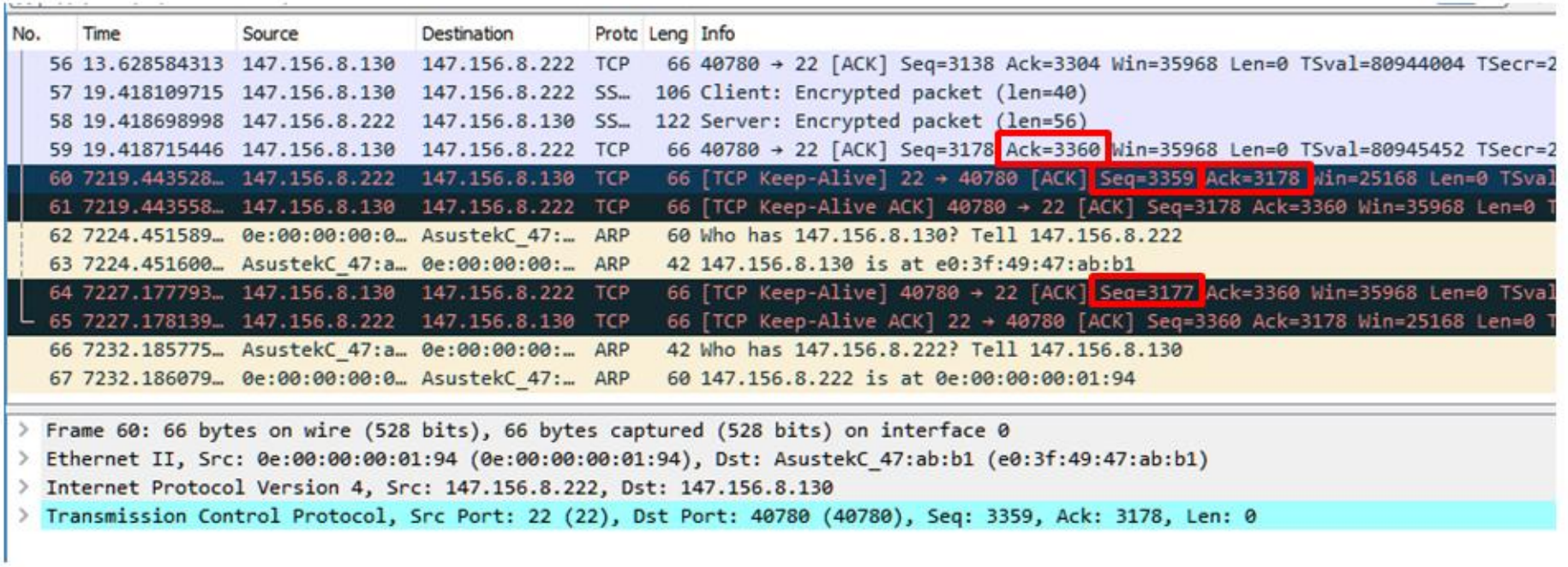
# Conexión cerrada por keepalive, Windows 10

| No. | Time | Source | Destination | Protc | Leng | Info |
|---|---|---|---|---|---|---|
| 64 | 3.670478 | 192.168.1.130 | 147.156.8.162 | TCP | 54 | 1697 → 21 [ACK] Seq=117 Ack=744 Win=15872 Len=0 |
| 65 | 3.742994 | 147.156.8.162 | 192.168.1.130 | TCP | 54 | 49231 → 1699 [ACK] Seq=224 Ack=2 Win=66560 Len=0 |
| 66 | 903.671334 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 67 | 903.733532 | 147.156.8.162 | 192.168.1.130 | TCP | 66 | [TCP Keep-Alive ACK] 21 → 1697 [ACK] Seq=744 Ack=117 Win=66560 Len=0 |
| 68 | 1803.745536 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 69 | 1804.745748 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 70 | 1805.754770 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 71 | 1806.761708 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 72 | 1807.773670 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 73 | 1808.780959 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 74 | 1809.792469 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 75 | 1810.795727 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 76 | 1811.811504 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 77 | 1812.823637 | 192.168.1.130 | 147.156.8.162 | TCP | 55 | [TCP Keep-Alive] 1697 → 21 [ACK] Seq=116 Ack=744 Win=15872 Len=1 |
| 78 | 1813.823596 | 192.168.1.130 | 147.156.8.162 | TCP | 54 | 1697 → 21 [RST, ACK] Seq=117 Ack=744 Win=0 Len=0 |

```
> Frame 71: 55 bytes on wire (440 bits), 55 bytes captured (440 bits) on interface 0
> Ethernet II, Src: IntelCor_a9:a5:e2 (d0:7e:35:a9:a5:e2), Dst: Comtrend_b3:c9:07 (38:72:c0:b3:c9:07)
> Internet Protocol Version 4, Src: 192.168.1.130, Dst: 147.156.8.162
> Transmission Control Protocol, Src Port: 1697 (1697), Dst Port: 21 (21), Seq: 116, Ack: 744, Len: 1
v Data (1 byte)
    Data: 00
    [Length: 1]
```

Paquete 66: Sondeo exitoso (incluye un byte de datos)
Paquete 67: respuesta del otro TCP
Paquete 68: Sondeo fallido (sin respuesta). 9 reintentos (tramas 69 a 77)
Paquete 78: Cierre de la conexión (flag RST)

# Mensajes de keepalive, Linux



| No. | Time | Source | Destination | Protc | Leng | Info |
|---|---|---|---|---|---|---|
| 56 | 13.628584313 | 147.156.8.130 | 147.156.8.222 | TCP | 66 | 40780 → 22 [ACK] Seq=3138 Ack=3304 Win=35968 Len=0 TSval=80944004 TSecr=2 |
| 57 | 19.418109715 | 147.156.8.130 | 147.156.8.222 | SS… | 106 | Client: Encrypted packet (len=40) |
| 58 | 19.418698998 | 147.156.8.222 | 147.156.8.130 | SS… | 122 | Server: Encrypted packet (len=56) |
| 59 | 19.418715446 | 147.156.8.130 | 147.156.8.222 | TCP | 66 | 40780 → 22 [ACK] Seq=3178 Ack=3360 Win=35968 Len=0 TSval=80945452 TSecr=2 |
| 60 | 7219.443528… | 147.156.8.222 | 147.156.8.130 | TCP | 66 | [TCP Keep-Alive] 22 → 40780 [ACK] Seq=3359 Ack=3178 Win=25168 Len=0 TSval |
| 61 | 7219.443558… | 147.156.8.130 | 147.156.8.222 | TCP | 66 | [TCP Keep-Alive ACK] 40780 → 22 [ACK] Seq=3178 Ack=3360 Win=35968 Len=0 T |
| 62 | 7224.451589… | 0e:00:00:00:0… | AsustekC_47:… | ARP | 60 | Who has 147.156.8.130? Tell 147.156.8.222 |
| 63 | 7224.451600… | AsustekC_47:a… | 0e:00:00:00:… | ARP | 42 | 147.156.8.130 is at e0:3f:49:47:ab:b1 |
| 64 | 7227.177793… | 147.156.8.130 | 147.156.8.222 | TCP | 66 | [TCP Keep-Alive] 40780 → 22 [ACK] Seq=3177 Ack=3360 Win=35968 Len=0 TSval |
| 65 | 7227.178139… | 147.156.8.222 | 147.156.8.130 | TCP | 66 | [TCP Keep-Alive ACK] 22 → 40780 [ACK] Seq=3360 Ack=3178 Win=25168 Len=0 T |
| 66 | 7232.185775… | AsustekC_47:a… | 0e:00:00:00:… | ARP | 42 | Who has 147.156.8.222? Tell 147.156.8.130 |
| 67 | 7232.186079… | 0e:00:00:00:0… | AsustekC_47:… | ARP | 60 | 147.156.8.222 is at 0e:00:00:00:01:94 |

```
> Frame 60: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
> Ethernet II, Src: 0e:00:00:00:01:94 (0e:00:00:00:01:94), Dst: AsustekC_47:ab:b1 (e0:3f:49:47:ab:b1)
> Internet Protocol Version 4, Src: 147.156.8.222, Dst: 147.156.8.130
> Transmission Control Protocol, Src Port: 22 (22), Dst Port: 40780 (40780), Seq: 3359, Ack: 3178, Len: 0
```

Trama 60: sondeo hecho por 147.156.8.222. No contiene datos
Trama 61: respuesta de 147.156.8.130
Tramas 64 y 65: sondeo de 147.156.8.130 y respuesta de 147.156.8.222

# Mensaje y timer de keepalive

- Se implementa enviando un segmento vacío con el número de secuencia del último byte enviado. Siempre que ve un número de secuencia inesperado TCP debe devolver un ACK, aunque el segmento no lleve datos.
- Si el keepalive no es respondido se envían varios intentos en un período corto y si no hay respuesta se cierra la conexión.
- El keepalive no requiere modificaciones en el TCP receptor. Solo suele utilizarse en servidores
- El valor típico del timer de keepalive son 2 horas
- El uso del keepalive es polémico pues tiene el riesgo de interrumpir conexiones legítimas por un fallo momentáneo de la comunicación

# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - **Cálculo del timer de retransmisión**
  - Control de congestión
  - Redes LFN y factor de escala
  - Opciones de TCP

# Timer de retransmisión

- De todos los timers que maneja TCP el de retransmisión es el más importante, ya que resulta crucial para su correcto funcionamiento:
  - Si es excesivo el emisor espera inútilmente, el rendimiento baja
  - Si es demasiado corto se hacen reenvíos innecesarios, el rendimiento baja
- TCP ha de funcionar sobre enlaces de Kb/s a Gb/s, por tanto no hay un valor adecuado para todos los casos. Por ello se utilizan mecanismos autoadaptativos.
- Los ACK recibidos se utilizan como 'sonda' para estimar el RTT de la conexión, y a partir de él se calcula el timeout.

# Timer de retransmisión

- A partir de los RTT medidos en una conexión TCP podemos calcular su valor medio o MRTT:

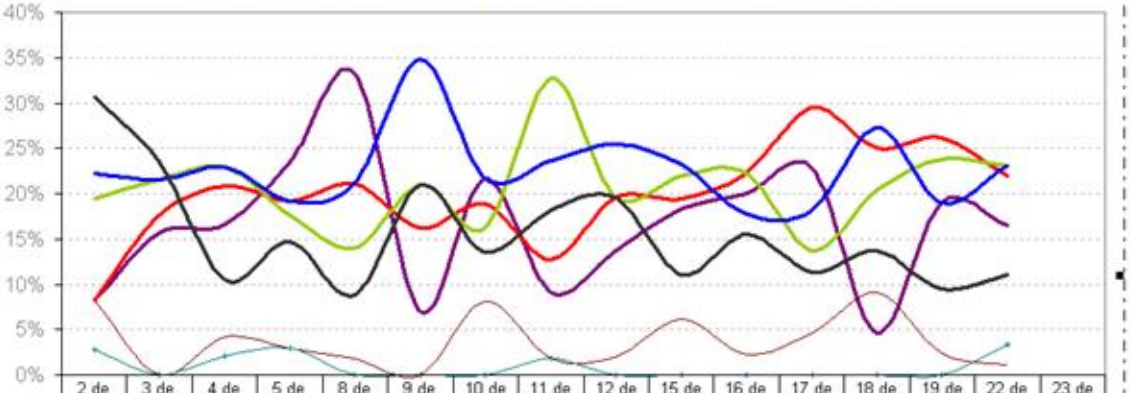
$$MRTT = \frac{\sum_{i=1}^{n} RTT_i}{n}$$

# MRTT y SRTT

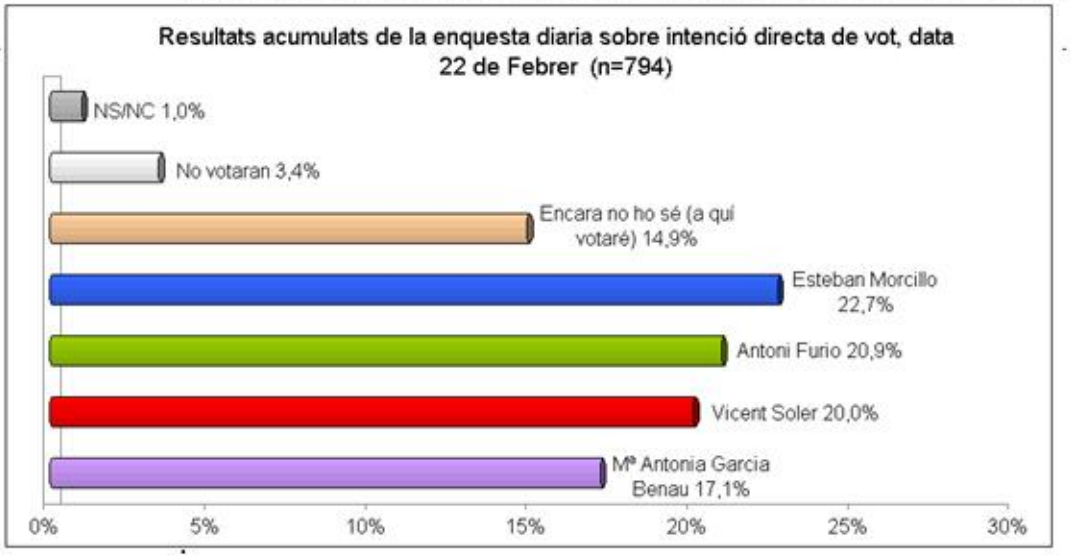
- Dado que una conexión TCP puede durar horas y días, los valores recientes de RTT representan mejor la situación actual, que puede variar cada minuto.
- En esta situación hacer la media aritmética de todos los RTT no resulta lo más adecuado.
- Por ello en vez de eso se calcula una media ponderada, en la que el peso de cada valor disminuye a medida que aumenta su antigüedad
- Esto se conoce como una **Media móvil exponencial ponderada**, que denominaremos SRTT (Smoothed RTT, o RTT 'suavizado')

## Resultats enquesta diaria sobre intenció de vot a les eleccions a Rector de la UV de 2 de Març.

(Data 22 de febrer: Errada máxima d´estimació ±9,36% amb confiança del 95%)




| | 2 de Febrer | 3 de Febrer | 4 de Febrer | 5 de Febrer | 8 de Febrer | 9 de Febrer | 10 de Febrer | 11 de Febrer | 12 de Febrer | 15 de Febrer | 16 de Febrer | 17 de Febrer | 18 de Febrer | 19 de Febrer | 22 de Febrer | 23 de Febrer |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Mª Antonia Garcia Benau | 8,3% | 15,7% | 16,7% | 23,5% | 33,3% | 7,0% | 21,6% | 9,1% | 13,7% | 18,3% | 20,0% | 22,7% | 4,6% | 19,1% | 16,5% | |
| Vicent Soler | 8,3% | 17,7% | 20,8% | 19,1% | 21,1% | 16,3% | 18,9% | 12,7% | 19,6% | 19,5% | 22,2% | 29,6% | 25,0% | 26,2% | 22,0% | |
| Antoni Furio | 19,4% | 21,6% | 22,9% | 17,7% | 14,0% | 20,9% | 16,2% | 32,7% | 19,6% | 22,0% | 22,2% | 13,6% | 20,5% | 23,8% | 23,1% | |
| Esteban Morcillo | 22,2% | 21,6% | 22,9% | 19,1% | 21,1% | 34,9% | 21,6% | 23,6% | 25,5% | 23,2% | 17,8% | 18,2% | 27,3% | 19,1% | 23,1% | |
| Encara no ho sé (a quí votaré) | 30,6% | 23,5% | 10,4% | 14,7% | 8,8% | 20,9% | 13,5% | 18,2% | 19,6% | 11,0% | 15,6% | 11,4% | 13,6% | 9,5% | 11,0% | |
| No votaran | 8,3% | 0,0% | 4,2% | 2,9% | 1,8% | 0,0% | 8,1% | 1,8% | 2,0% | 6,1% | 2,2% | 4,6% | 9,1% | 2,4% | 1,1% | |
| NS/NC | 2,8% | 0,0% | 2,1% | 2,9% | 0,0% | 0,0% | 0,0% | 1,8% | 0,0% | 0,0% | 0,0% | 0,0% | 0,0% | 0,0% | 3,3% | |



# Cálculo del SRTT

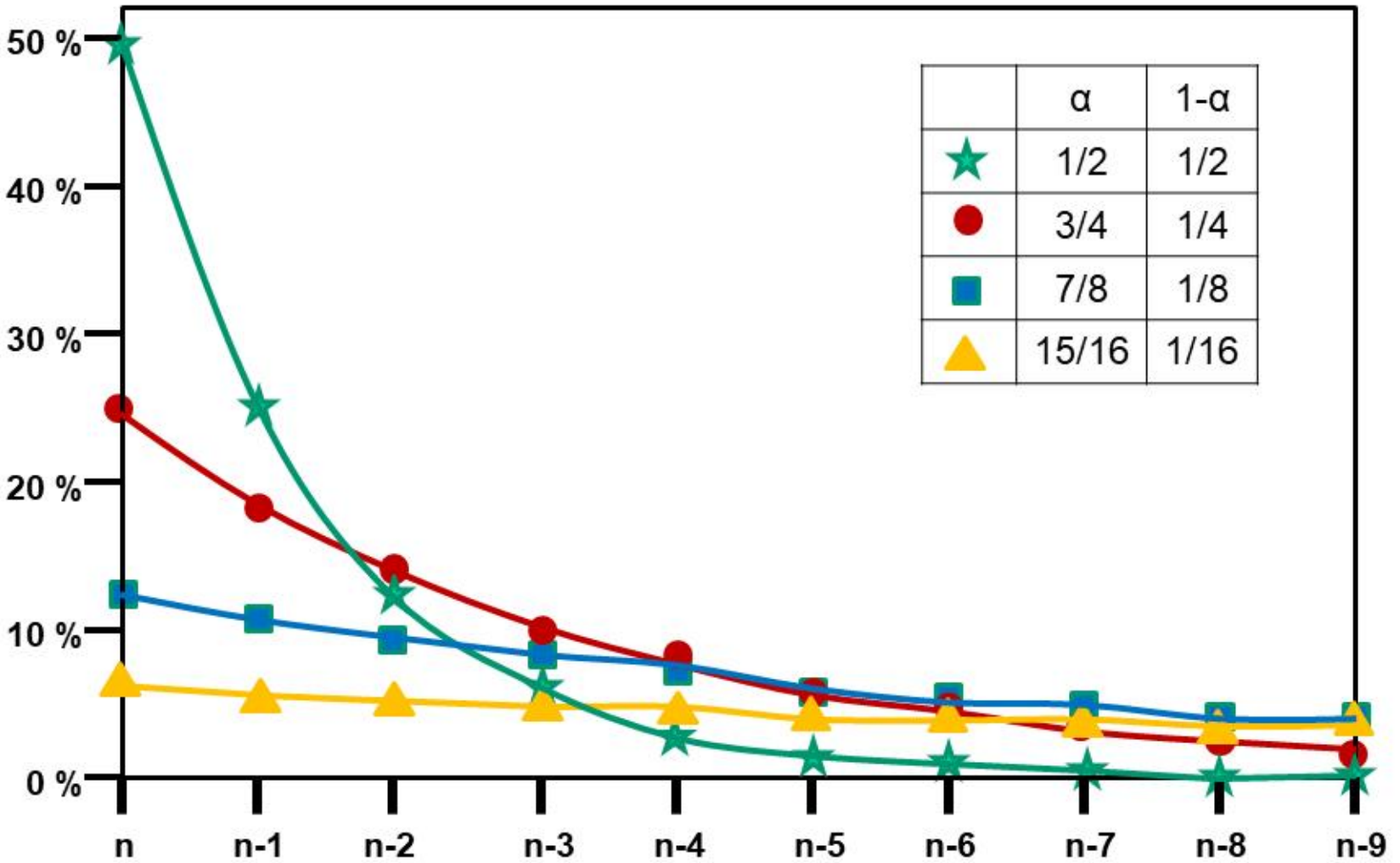
- Para el cálculo del SRTT se utiliza la fórmula:

$$SRTT_n = \alpha\ SRTT_{n-1} + (1 - \alpha)\ RTT$$

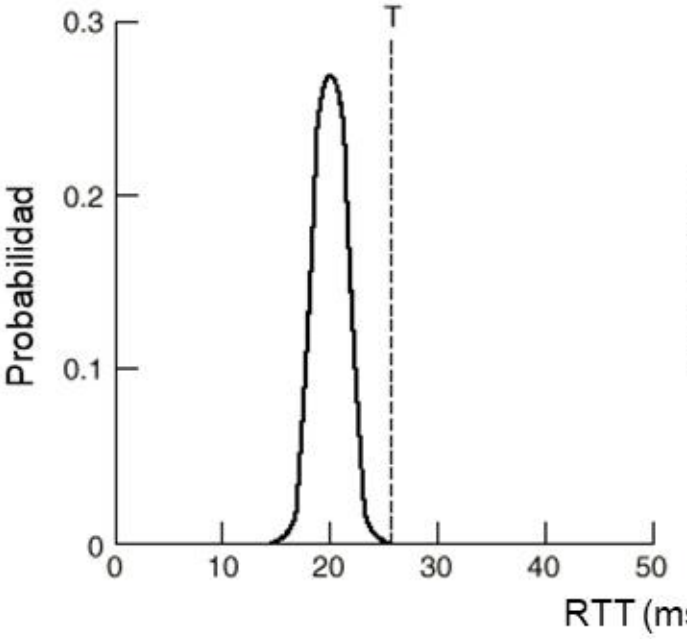
donde RTT es el último valor medido de RTT.

- $\alpha$ es un factor de amortiguación entre 0 y 1. Cuanto mayor es $\alpha$ más peso tienen los valores antiguos. Un valor de $\alpha$ demasiado pequeño nos impedirá responder con rapidez a situaciones cambiantes, un valor excesivo nos hará demasiado reactivos, pudiendo dar lugar a situaciones oscilantes.
- Por eficiencia del cálculo se busca que $1-\alpha = 1/(2)^n$. De ese modo la multiplicación se reduce a un desplazamiento de bits.

# Peso de $RTT_n$ en el cálculo de SRTT según $\alpha$

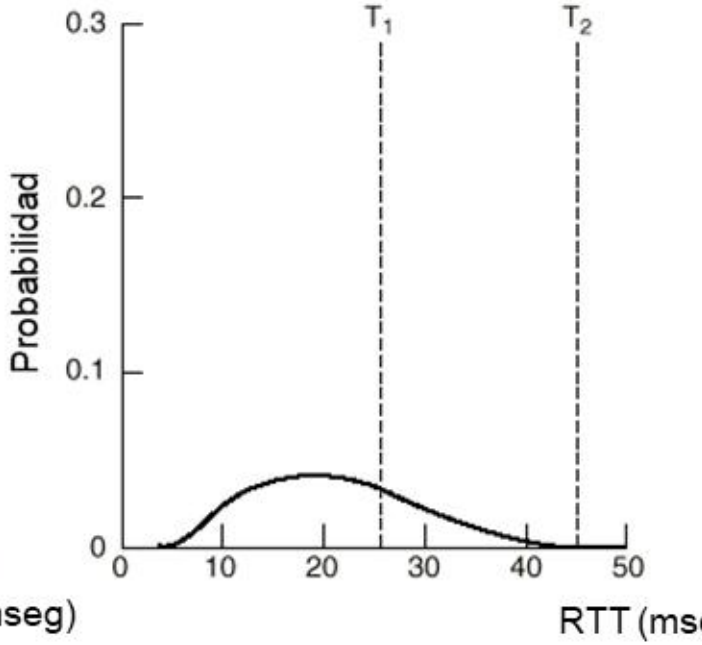


# Dispersión del RTT




RTT en una red sin tráfico

**Con poca dispersión el timeout puede ser solo ligeramente mayor que el valor medio**



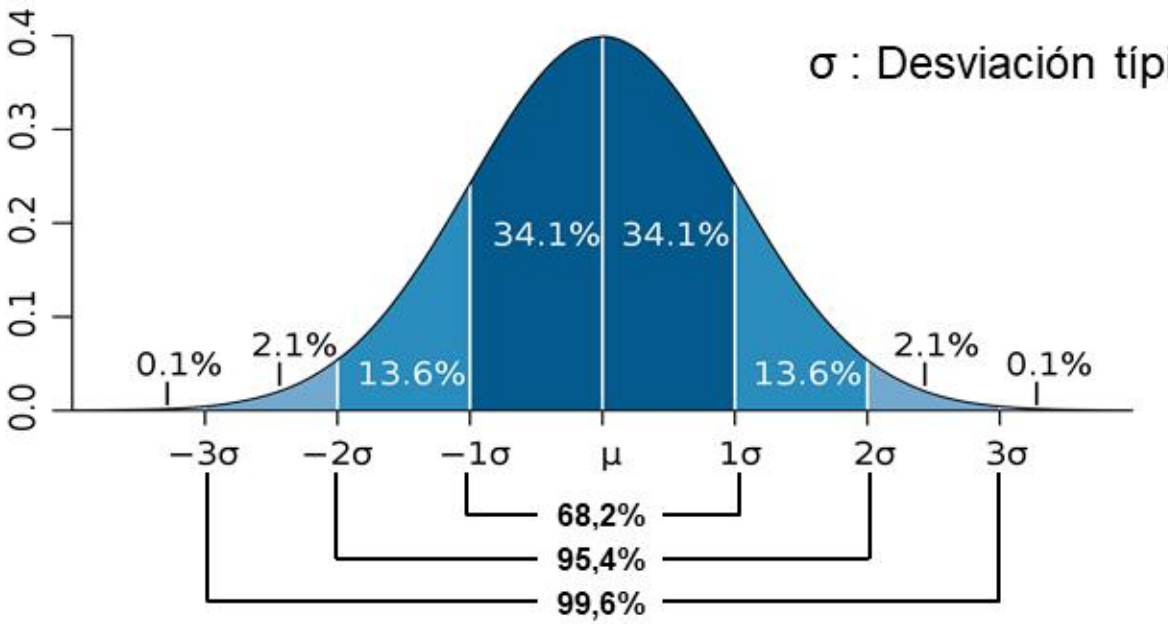

RTT en una red congestionada

**Con mucha dispersión el timeout ha de ser bastante superior al valor medio**

**Saber el rango de variación de los RTT es tan importante como saber su valor medio. Esto se estima con la**

# Distribución normal

- Si los valores de RTT siguieran una distribución normal el rango MRTT ± σ abarcaría el 68% de los valores medidos, MRTT ± 2σ comprendería el 95% y MRTT ± 3σ el 99,6%:




Esto es sólo una aproximación, ya que los valores de RTT no siguen una distribución normal (la distribución no es simétrica, por ejemplo)

# Variación del RTT

- A partir de los valores de RTT podemos calcular la **desviación típica (σ)** de una conexión; esto nos da una medida de la dispersión de los valores:

$$\sigma = \sqrt{\frac{\sum_{i=1}^{n} (RTT_i - MRTT)^2}{n}}$$

- En TCP se utiliza la **desviación media ($D_m$)**, más fácil de calcular que σ :

$$D_m = \frac{\sum_{i=1}^{n} | RTT_i - MRTT |}{n}$$

# Cálculo de la variación de RTT

- Tan importante como tener una buena estimación del RTT es tener una buena estimación de su desviación media (el rango de variación de fluctuación)
- En TCP la desviación media $D_m$ se estima a partir de la RTTVAR (RTT Variation) que se calcula como una media móvil exponencial ponderada de las diferencias (en valor absoluto) entre RTT y el SRTT (no el MRTT):

$$RTTVAR_n = \beta\, RTTVAR_{n-1} + (1 - \beta)\,|\,SRTT_{n-1} - RTT|$$

En este caso el factor de amortiguación $\beta$ suele valer 3/4

# Cálculo del timeout

- A partir de $SRTT_n$ y $RTTVAR_n$ se calcula el RTO (Retransmission TimeOut) como:

$$RTO = SRTT_n + 4 * RTTVAR_n$$

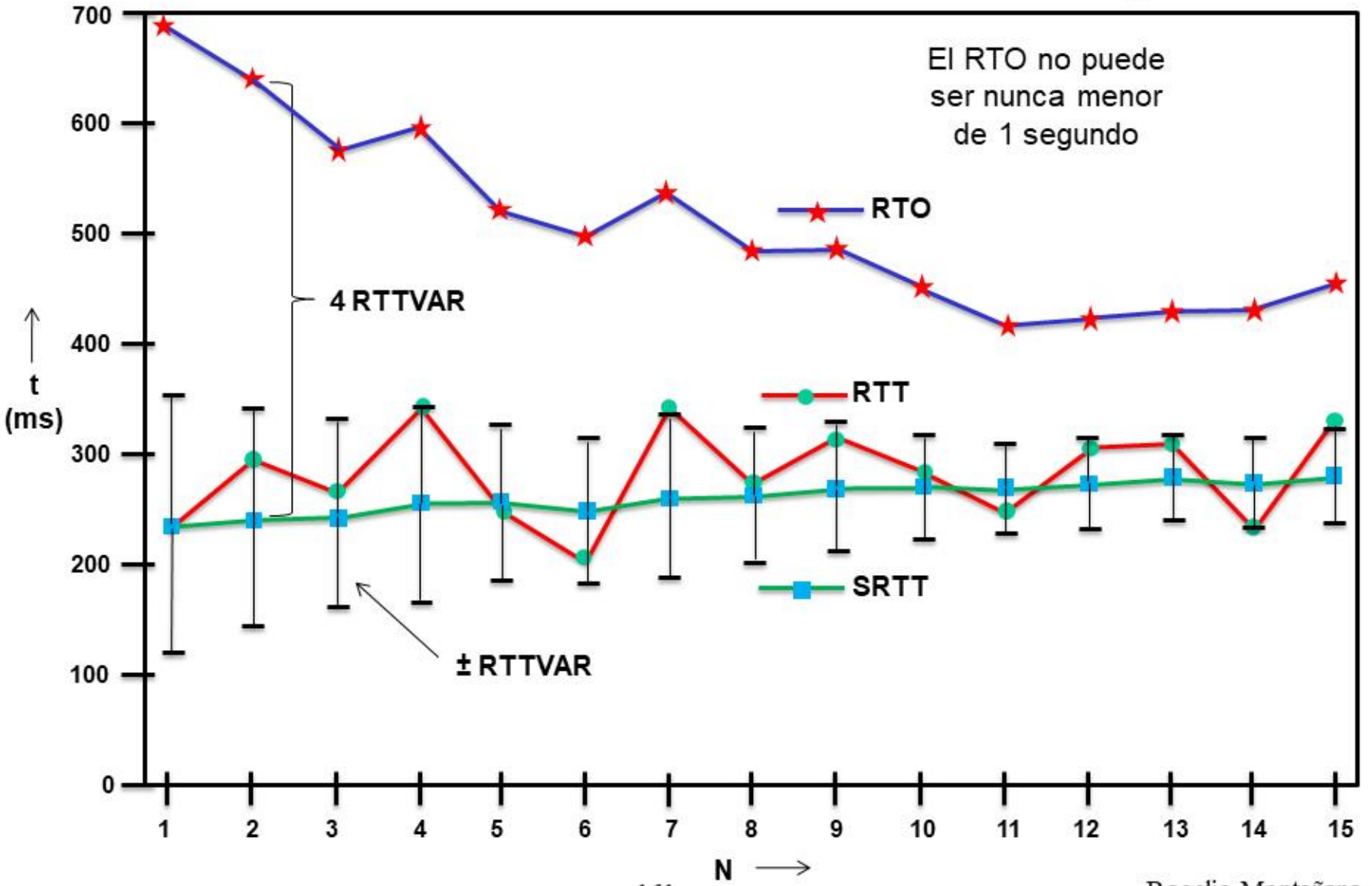
- Con este valor la media de fallos (paquetes dados por perdidos que luego aparecen) es menor del 1%.
- Todos los cálculos que tienen que ver con TCP se basan en potencias enteras de dos:
  - 1/8, en SRTT
  - 1/4 en RTTVAR
  - 4 en RTO

a fin de acelerar el cálculo (las multiplicaciones o divisiones por potencias de 2 se hacen por desplazamiento de bits).

# Evolución de SRTT, RTTVAR y RTO según RTT

| N | $RTT_n$ | $SRTT_n$ | $RTTVAR_n$ | RTO |
|---|---|---|---|---|
| 1 | 230 | 230,00 | 115 | 690,00 |
| 2 | 294 | 238,00 | 102,25 | 647,00 |
| 3 | 264 | 241,25 | 83,19 | 574,00 |
| 4 | 340 | 253,59 | 87,08 | 601,91 |
| 5 | 246 | 252,64 | 67,21 | 521,47 |
| 6 | 201 | 246,19 | 63,32 | 499,45 |
| 7 | 340 | 257,92 | 70,94 | 541,68 |
| 8 | 272 | 259,68 | 56,73 | 486,58 |
| 9 | 311 | 266,09 | 55,38 | 487,59 |
| 10 | 282 | 268,08 | 45,51 | 450,12 |
| 11 | 246 | 265,32 | 39,65 | 423,93 |
| 12 | 304 | 270,16 | 39,41 | 427,79 |
| 13 | 308 | 274,89 | 39,02 | 430,96 |
| 14 | 230 | 269,27 | 40,48 | 431,21 |
| 15 | 328 | 276,62 | 45,04 | 456,79 |

# Evolución de RTT, SRTT, RTTVAR y RTO



# RTT de segmentos perdidos

- Cuando TCP reenvía un segmento y luego recibe el ACK no puede saber si corresponde al primer envío o al segundo. Por eso cuando hay reenvíos no se utiliza el RTT correspondiente. Esto es lo que se conoce como el '**algoritmo de Karn**' (1987)
- Además, cada vez que se reenvía un segmento TCP duplica el RTO, para asegurar que si se produce un aumento repentino del RTT pronto se adaptará el RTO a la nueva situación

# Timers de TCP

| Timer | Cuando se agota provoca que … | Valor en BSD UNIX |
|---|---|---|
| Establecimiento de conexión | Se abandone un intento de establecimiento de conexión pendiente (SYN sin respuesta) | 75 seg. |
| Retransmisión | Se retransmita un segmento para el que no se recibió el correspondiente ACK. Se calcula dinámicamente a partir del RTT y del número de veces que se ha retransmitido ese segmento. | 1-64 seg. |
| ACK retrasado | Se envíe un ACK vacío que estaba retenido a la espera de tener datos para enviar | 200 mseg. |
| Persistencia | Se envíe un byte de datos para que el otro TCP confirme si la ventana sigue cerrada. Se calcula dinámicamente a partir del RTT | 5-60 seg. |
| Keepalive | Se le envíe un segmento vacío con seq anómalo a un TCP que no transmite datos para forzarle a enviar un ACK y confirmar que sigue conectado | 2 horas |
| FIN-WAIT-2 | Se termine una conexión que estaba en FIN-WAIT-2, a pesar de no haber recibido el FIN del otro TCP | 10 min. |
| TIME-WAIT | Se cierre una conexión que estaba en TIME-WAIT (2*MSL) | 1 min. |

# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - **Control de congestión**
  - Redes LFN y factor de escala
  - Opciones de TCP

# Control de congestión: planteamiento del problema

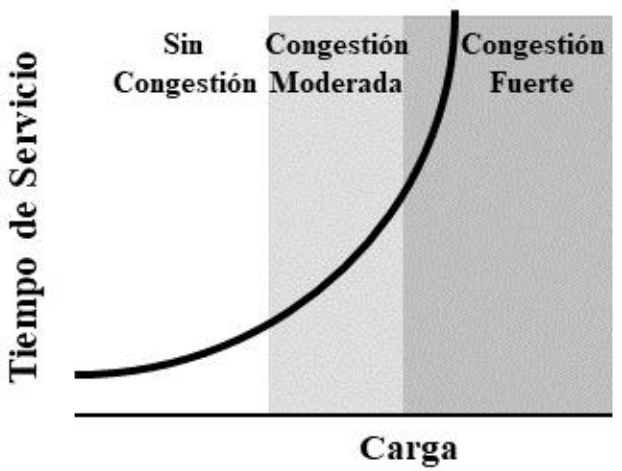


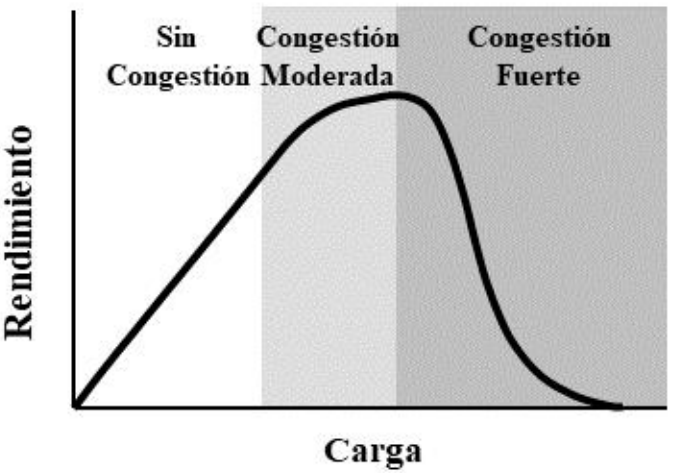
Van Jacobson
(1950 - )

- Al principio TCP transmitía cuanto quería; si la red se congestionaba y perdía segmentos TCP los retransmitía
- En 1986 se produjo en Internet un colapso por congestión. Los segmentos se empezaron a perder y reenviar de forma que el rendimiento de la red se redujo a la milésima parte.
- Gracias al trabajo de Van Jacobson se desarrolló una versión de TCP que incorporaba mecanismos de control de congestión. Este TCP se incluyó en 1988 en el S.O Unix 4.3BSD-Tahoe, por lo que se conoce como TCP-Tahoe.

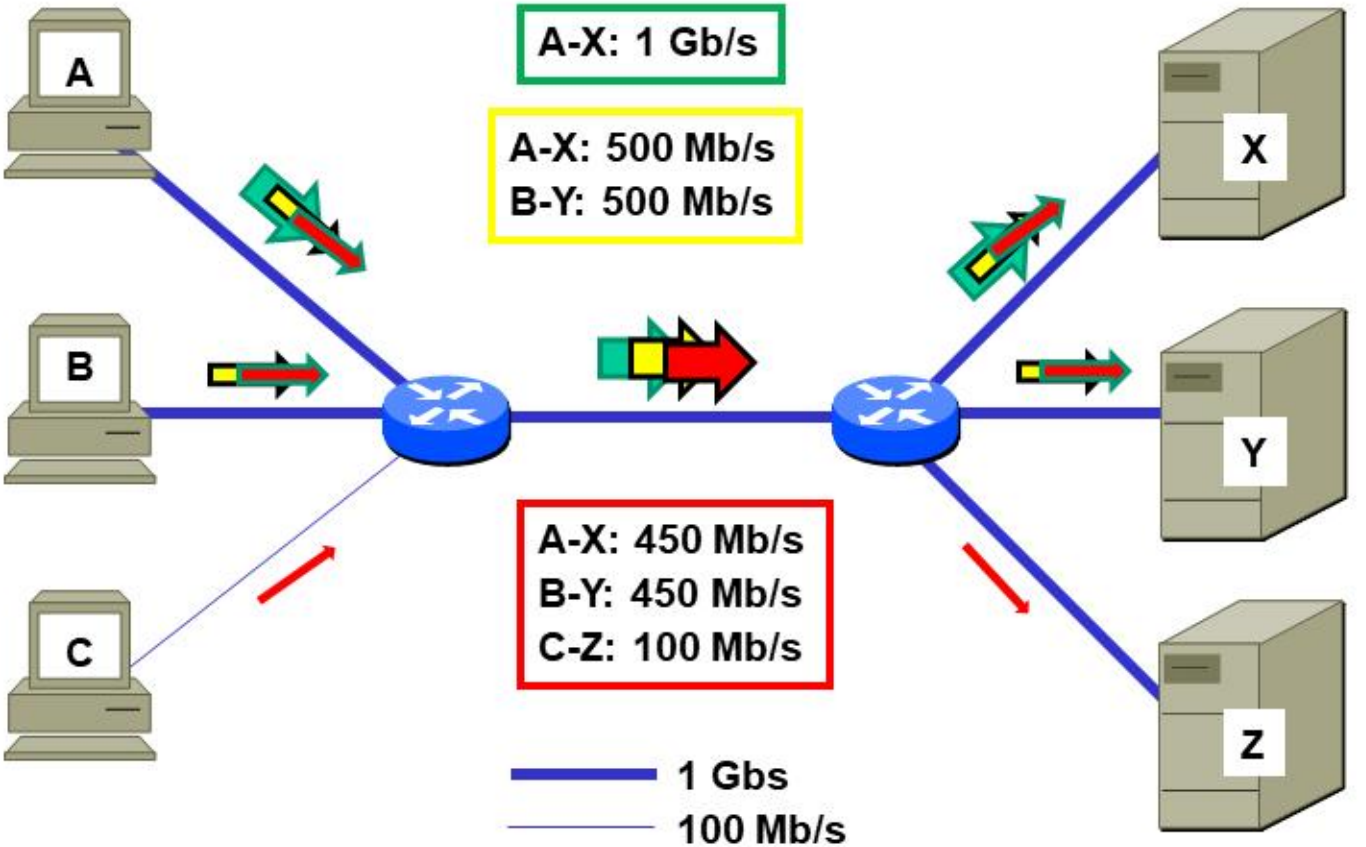
# Objetivos del control de congestión

1. Evitar el 'congestion collapse'
2. Repartir los recursos de manera equitativa entre las conexiones establecidas
3. Aprovechar lo mejor posible los recursos disponibles (acercarse tanto como sea posible al 100% de carga sin incurrir en el colapso por congestion)



# Ejemplo de reparto equitativo



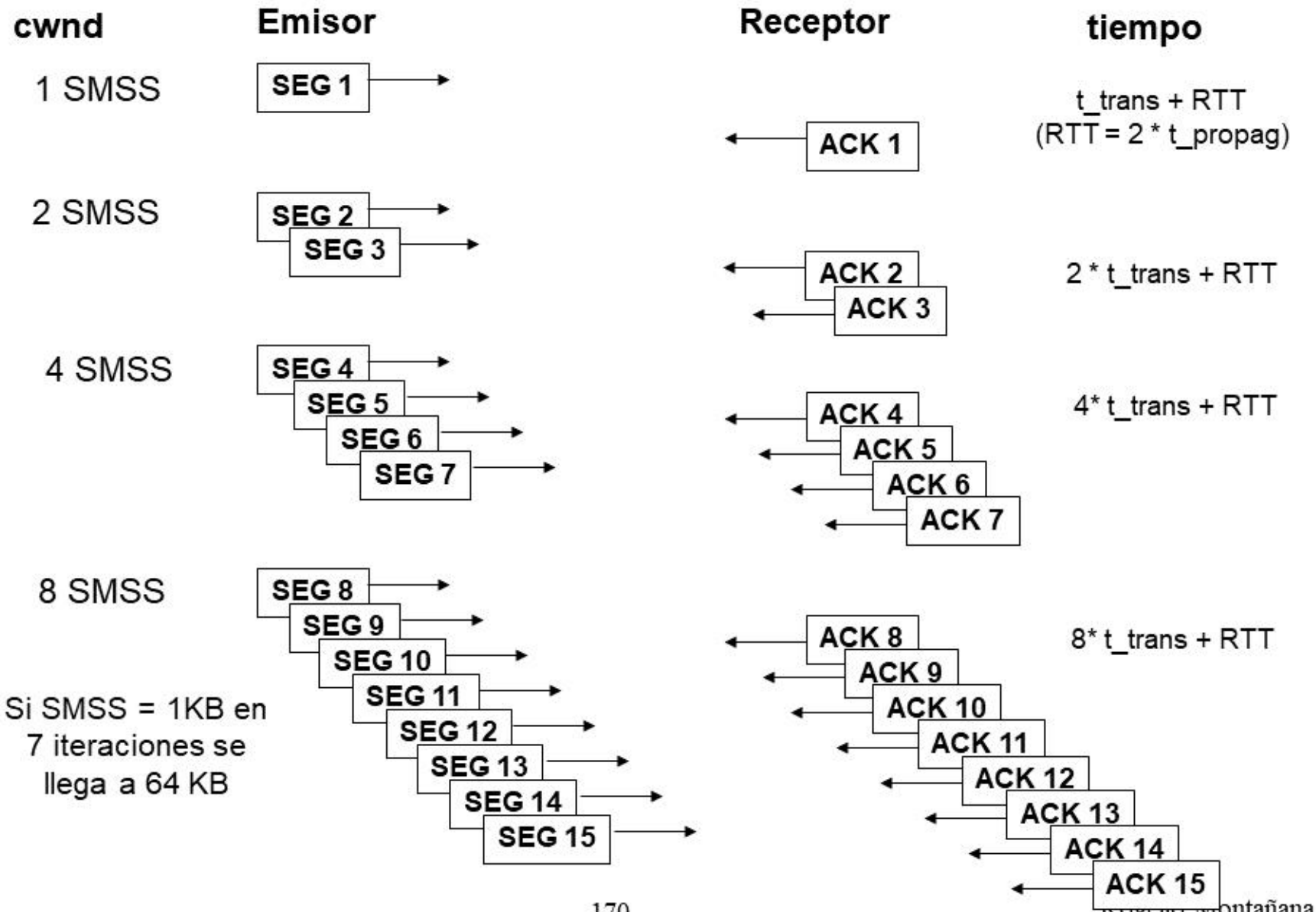
# Control de congestión en TCP

- TCP detecta la congestión de forma implícita: si se pierden segmentos supone que hay congestión y baja el ritmo.
- Utiliza un algoritmo en dos fases conocidas como *slow-start* (arranque lento) y *congestion avoidance* (evitación de la congestión)
- El TCP emisor mantiene en todo momento una ventana de congestión (congestion window, cwnd) que establece cuantos datos puede enviar en cada momento.
- La cwnd no se anuncia a nadie, es interna del TCP e independiente de la ventana de control de flujo que va en la cabecera TCP. Se usa siempre la más pequeña de ambas.

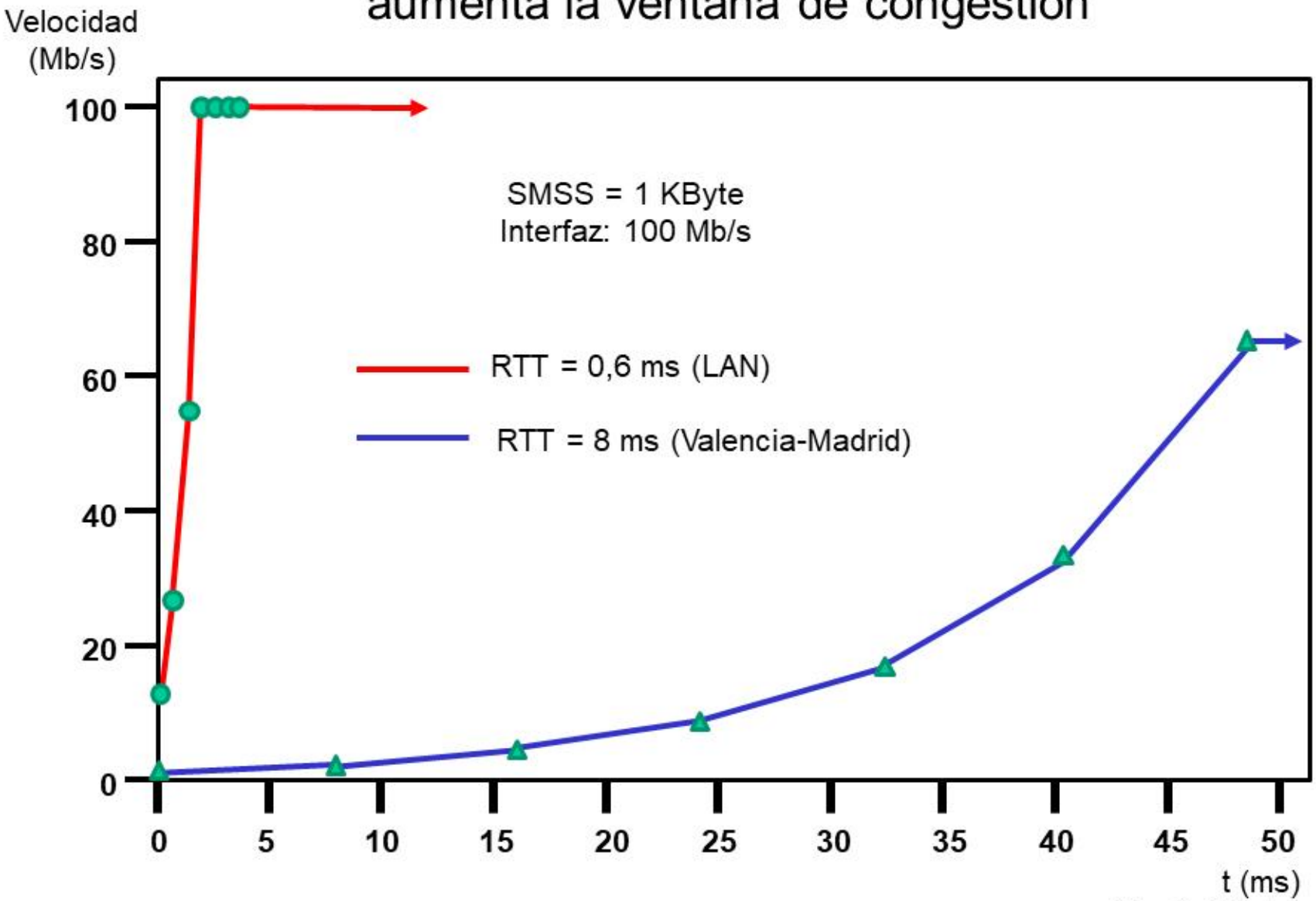
# Arranque lento (Slow Start, fase 1)

- Inicialmente la cwnd (ventana de congestión) tiene un tamaño de entre uno y cuatro SMSS (Sender Maximum Segment Size). El valor exacto depende de la implementación
- Por cada segmento enviado con éxito la cwnd se amplía en un SMSS. El éxito se conoce al recibir el ACK
- En la práctica esto supone un crecimiento exponencial de la cwnd (en potencias de dos)
- Mientras no se pierda ningún segmento la cwnd crece ilimitadamente.

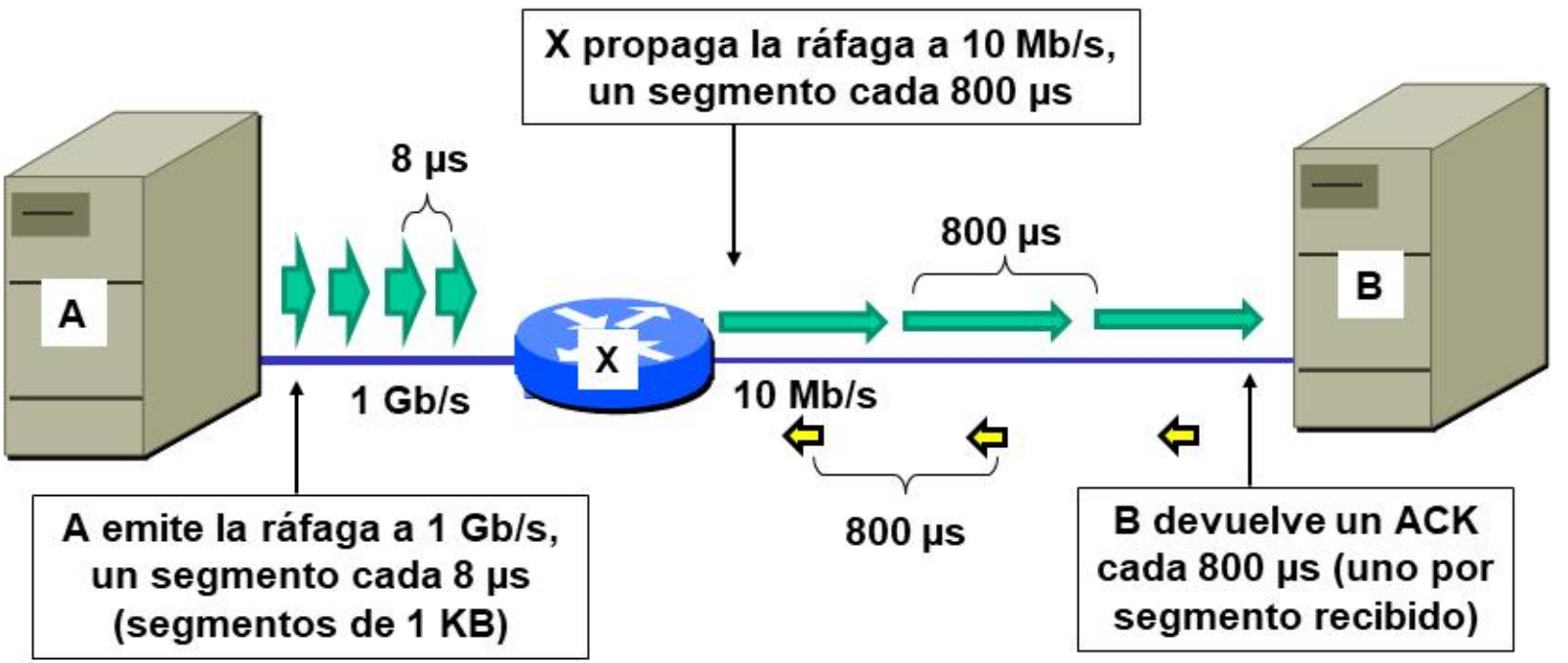
# Control de congestión, fase 1 (slow start)



# Evolución del rendimiento según aumenta la ventana de congestión



# El 'reloj ACK'

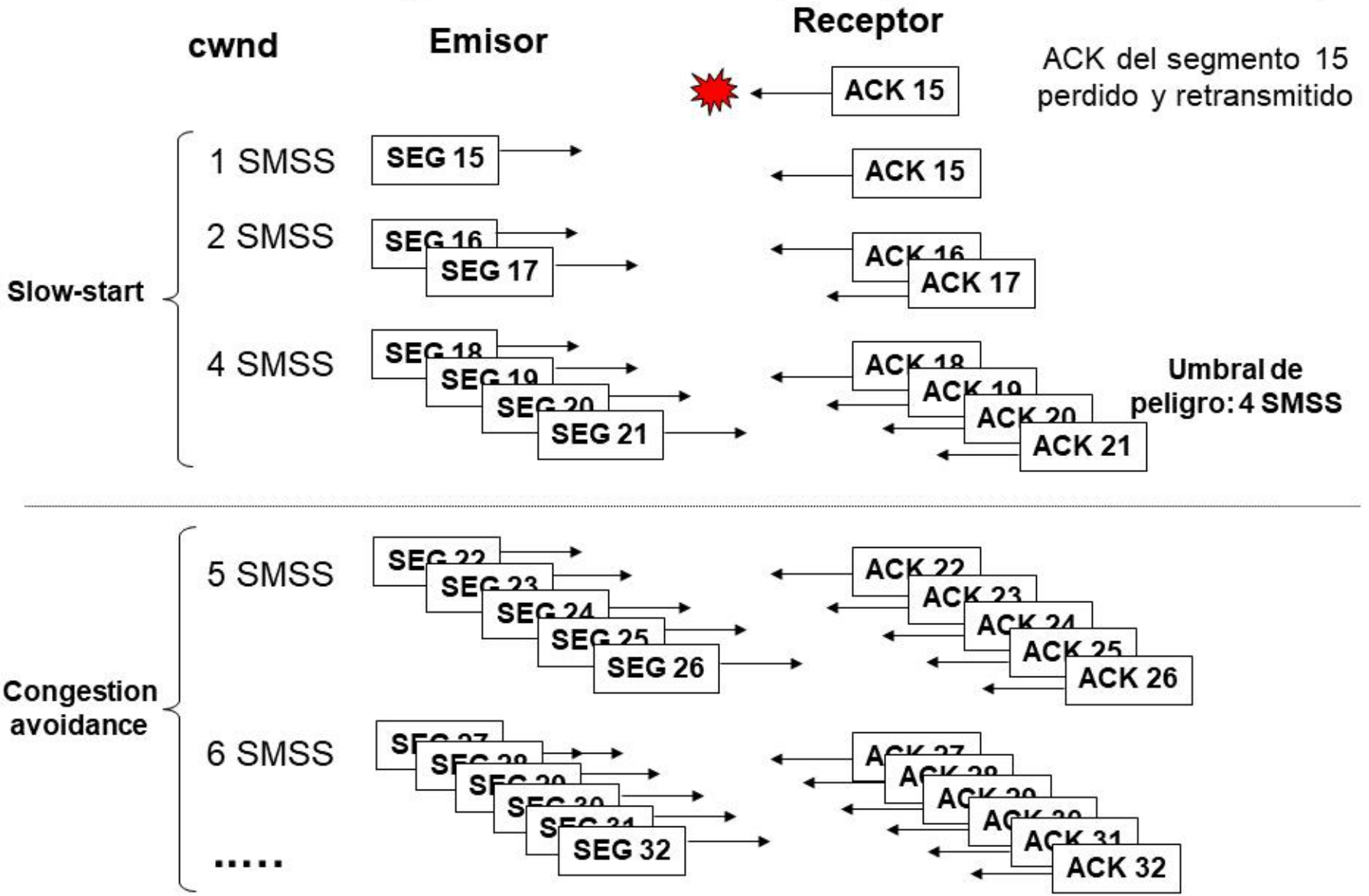



Como A espera el ACK de B antes de enviar la siguiente tanda el caudal se ajusta automáticamente al enlace de menor capacidad en la ruta
Esto es lo que se conoce como el reloj ACK o 'ACK clock'

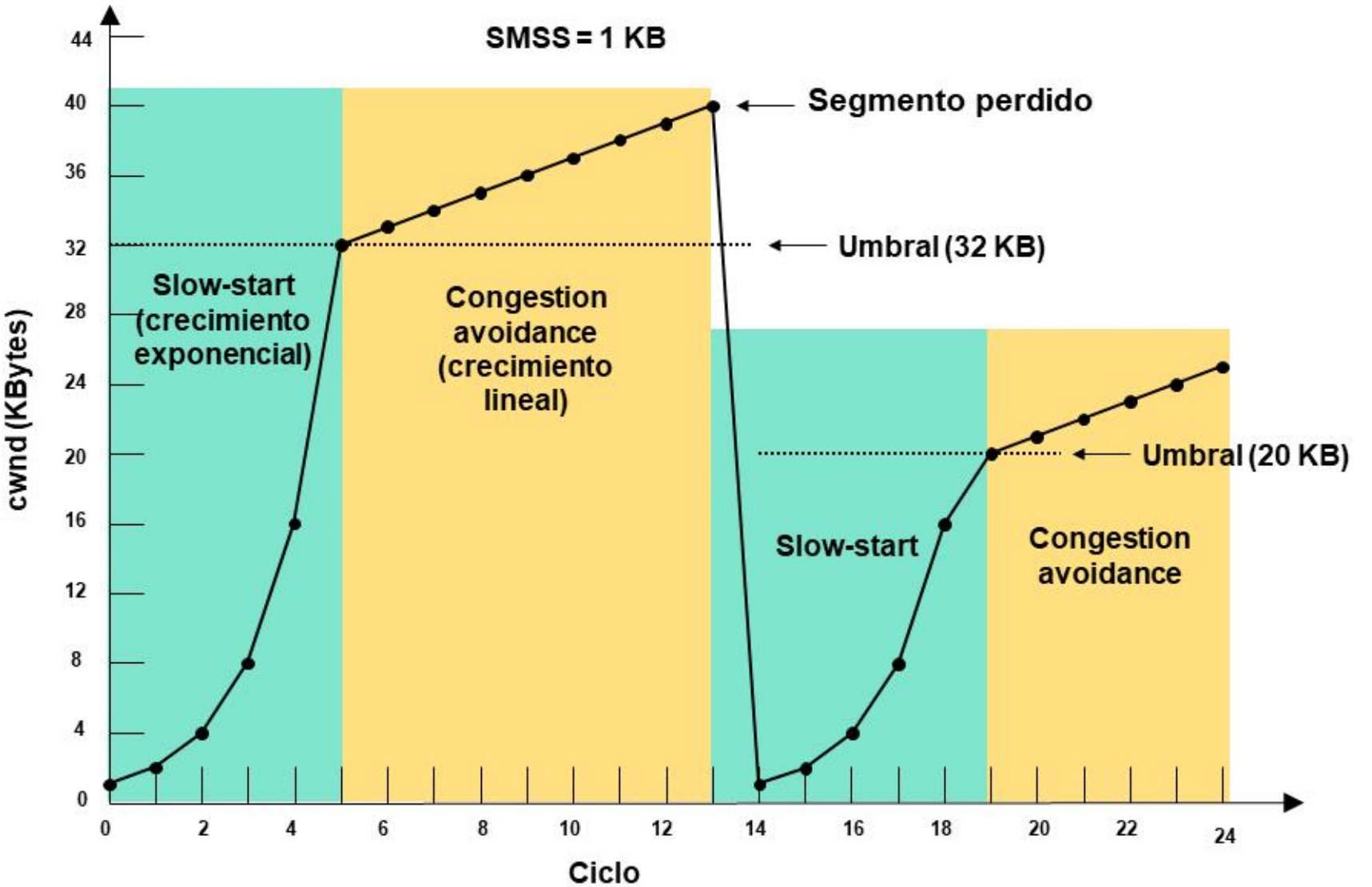
# Evitación de la congestión (Congestion avoidance, fase 2)

- Cuando se pierde un segmento:
  1. La cwnd vuelve a su valor inicial (de uno a cuatro SMSS)
  2. Se fija un 'umbral de peligro' en un valor igual a la mitad de la cwnd que había cuando se produjo la pérdida.
  3. La cwnd crece exponencialmente como antes (slow-start) hasta el umbral de peligro; a partir de ahí crece linealmente, sólo un SMSS en cada ciclo

# Control de congestión, fase 2 (congestion avoidance)



# Algoritmo TCP-Tahoe (1988)



# Ejercicio: control de congestión

- Una conexión TCP sobre un medio físico poco fiable pierde exactamente una trama de cada 10. El nivel de enlace descarta las tramas erróneas, sin pedir reenvío (CRC):
  - Detalle como se producirá el envío de los segmentos y como evolucionará la ventana de congestión en el TCP emisor (se supone que se ha negociado utilizar SACK)
  - Estime cualitativaente la merma de rendimiento que cabe esperar debido a la tasa de error:
    A. Menor del 10%
    B. Alrededor del 10%
    C. Mayor del 10%
  - Diga como influiría el RTT en el rendimiento

# Ejercicio: Evolución ventana de congestión

| | cwnd (KB) | Trama | Segmento | Umbral peligro (KB) |
|---|---|---|---|---|
| S.S. | 1 | 1 | 1 | 64 |
| | 2 | 2,3 | 2,3 | 64 |
| | 4 | 4,5,6,7 | 4,5,6,7 | 64 |
| | 8 | 8,9,10,11,12,13,14,15 | 8,9,10,11,12,13,14,15 | 64 |
| S.S. | 1 | 16 | 10 | 4 |
| | 2 | 17,18 | 16,17 | 4 |
| | 4 | 19,20,21,22 | 18,19,20,21 | 4 |
| S.S. | 1 | 23 | 19 | 2 |
| | 2 | 24,25 | 22,23 | 2 |
| C.A. | 3 | 26,27,28 | 24,25,26 | 2 |
| | 4 | 29,30,31,32 | 27,28,29,30 | 2 |
| S.S. | 1 | 33 | 28 | 2 |
| | 2 | 34,35 | 31,32 | 2 |
| C.A. | 3 | 36,37,38 | 33,34,35 | 2 |
| | 4 | 39,40,41,42 | 36,37,38,39 | 2 |
| S.S. | 1 | 43 | 37 | 2 |
| | 2 | 44,45 | 40,41 | 2 |
| C.A. | 3 | 46,47,48 | 42,43,44 | 2 |
| | 4 | 49,50,51,52 | 45,46,47,48 | 2 |
| S.S. | 1 | 53 | 46 | 2 |
| | 2 | 54,55 | 49,50 | 2 |

# Ejercicio: solución

- La merma de rendimiento será siempre superior al 10%, ya que además de tener que reenviar el 10% de los paquetes tenemos que estar continuamente reiniciando la ventana de congestión.
- Cuanto mayor sea el RTT mayor será la pérdida. Con un RTT cero la pérdida sería del 10%

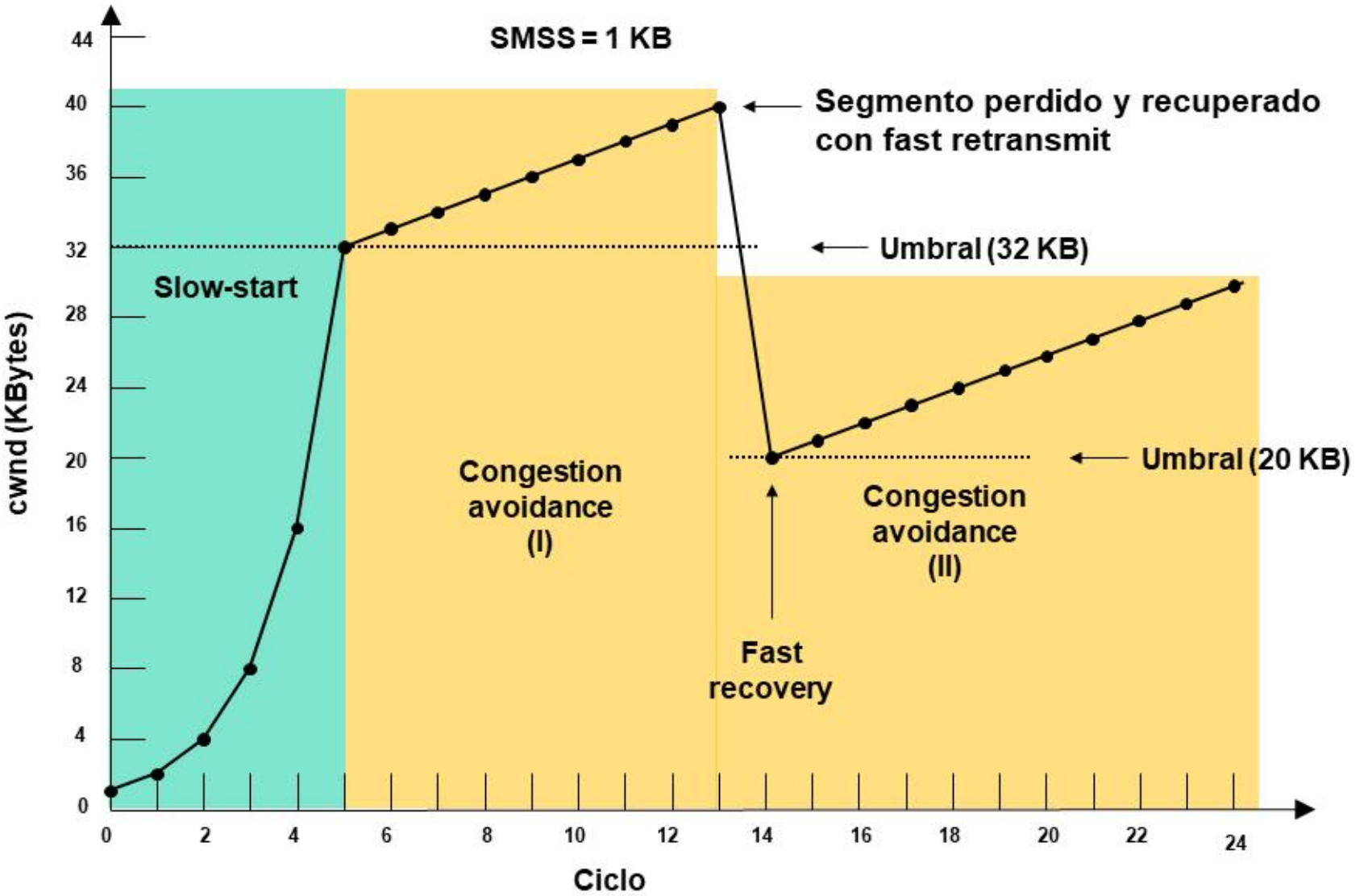
# Fast recovery

- Van Jacobson introdujo una mejora en el algoritmo de congestión de TCP-Tahoe llamada fast recovery. Observó que la pérdida de un segmento podía detectarse de dos maneras:
  - Por agotamiento del timeout del ACK
  - Por la recepción de ACK duplicados (fast retransmit)
- En TCP Tahoe (1988) ambos casos se trataban del mismo modo: umbral de peligro, slow-start, congestion-avoidance
- Van Jacobson pensó que, en el caso del fast-retransmit, se podía aplicar un correctivo menos contundente, ya que después de todo el problema se había resuelto con el reenvío de un solo segmento.

# Fast Recovery

- Van Jacobson decidió saltarse, en el caso del fast retransmit, la fase del slow-start, es decir, cuando se produjera un fast retransmit:
  1. TCP fijaría el umbral de peligro en la mitad de la cwnd vigente en ese momento
  2. Fijaría la nueva cwnd en el umbral de peligro
  3. Creceria linealmente a partir de ahí (congestion avoidance), pero sin empezar desde el principio
- Este mecanismo lo denominó 'fast recovery' y lo incorporó en 1990 en el 4.3BSD-Reno, por lo que se conoce como TCP-Reno

# Algoritmo TCP-Reno (1990)



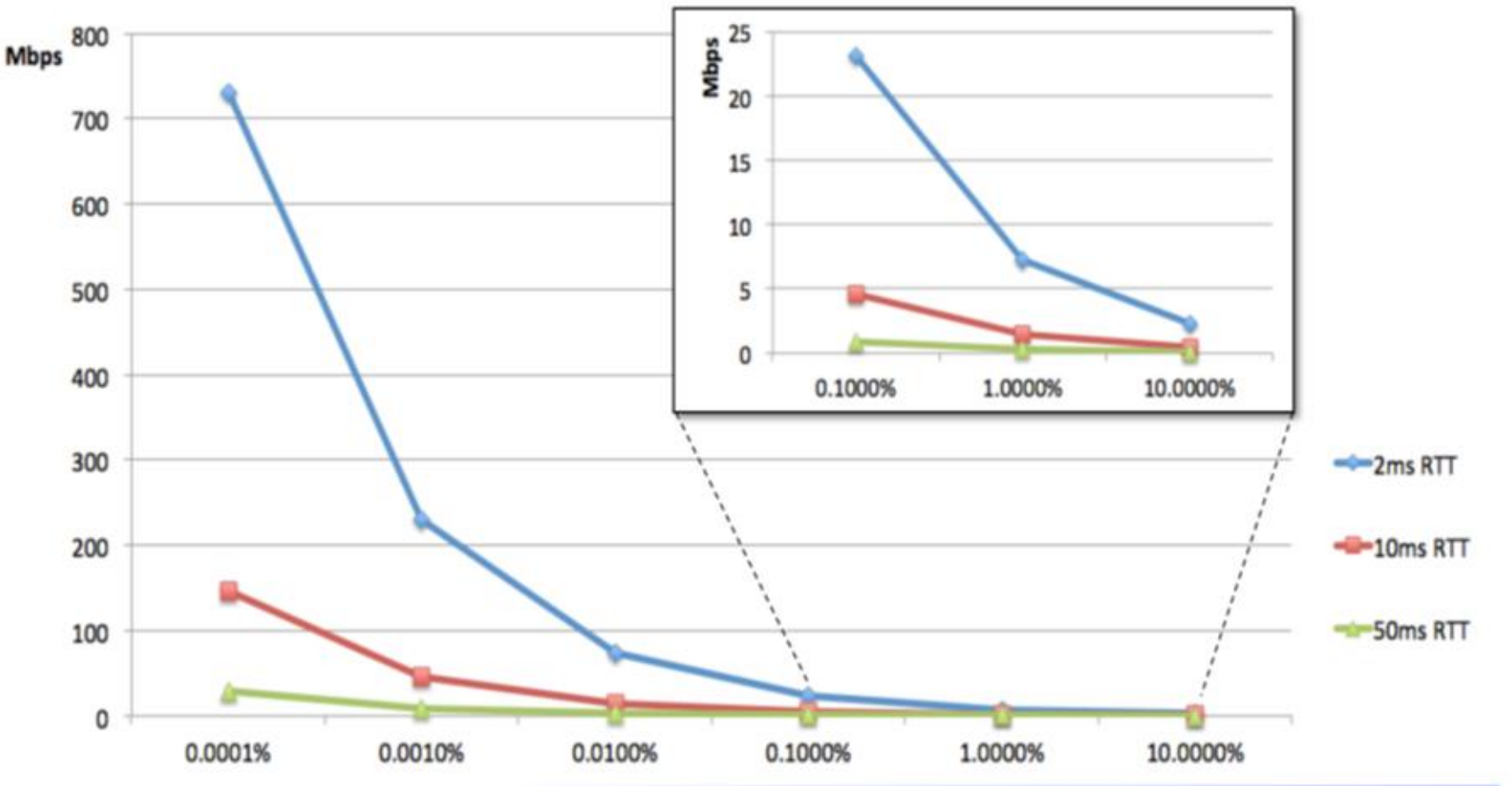
# Algoritmos de congestión de TCP

- El control de congestión de TCP es objeto de continuos estudios. Todos los algoritmos actuales tienen como base a TCP-Reno. Los más utilizados actualmente son:
  - TCP New-Reno: FreeBSD, Linux < 2.6.8
  - TCP BIC (Binary Increase Congestion): Linux 2.6.8-18
  - TCP CUBIC: Linux ≥ 2.6.19
  - TCP PRR (Proportional Rate Reduction): Linux ≥3.2
  - Compound TCP: Windows Vista y sigtes.
- El algoritmo utilizado no ha de negociarse, puede ser diferente en cada lado.
- Ver en Wikipedia 'TCP Congestion Control'

# Congestión y Errores de transmisión en TCP

- El mecanismo implícito de detección de congestión que utiliza TCP supone que las pérdidas de segmentos se deben a congestión.
- Si la pérdida de tramas se debe a errores será erróneamente interpretada como congestión por TCP
- Se iniciará entonces el slow-start y bajará el caudal de forma considerable, sobre todo en conexiones con RTT elevado. P. ej. un enlace de 1 Gb/s con un RTT de 10ms y una pérdida del 1% da un rendimiento útil de 2 Mb/s
- Para un buen rendimiento TCP necesita que el canal de comunicación no tenga errores; si los hay el nivel de enlace debe habilitar los mecanismos adecuados para corregirlos (códigos FEC) o retransmitir las tramas erróneas (WiFi)

# Efecto de la pérdida de paquetes en el rendimiento de TCP para diversos valores de RTT (enlace de 1 Gbps)

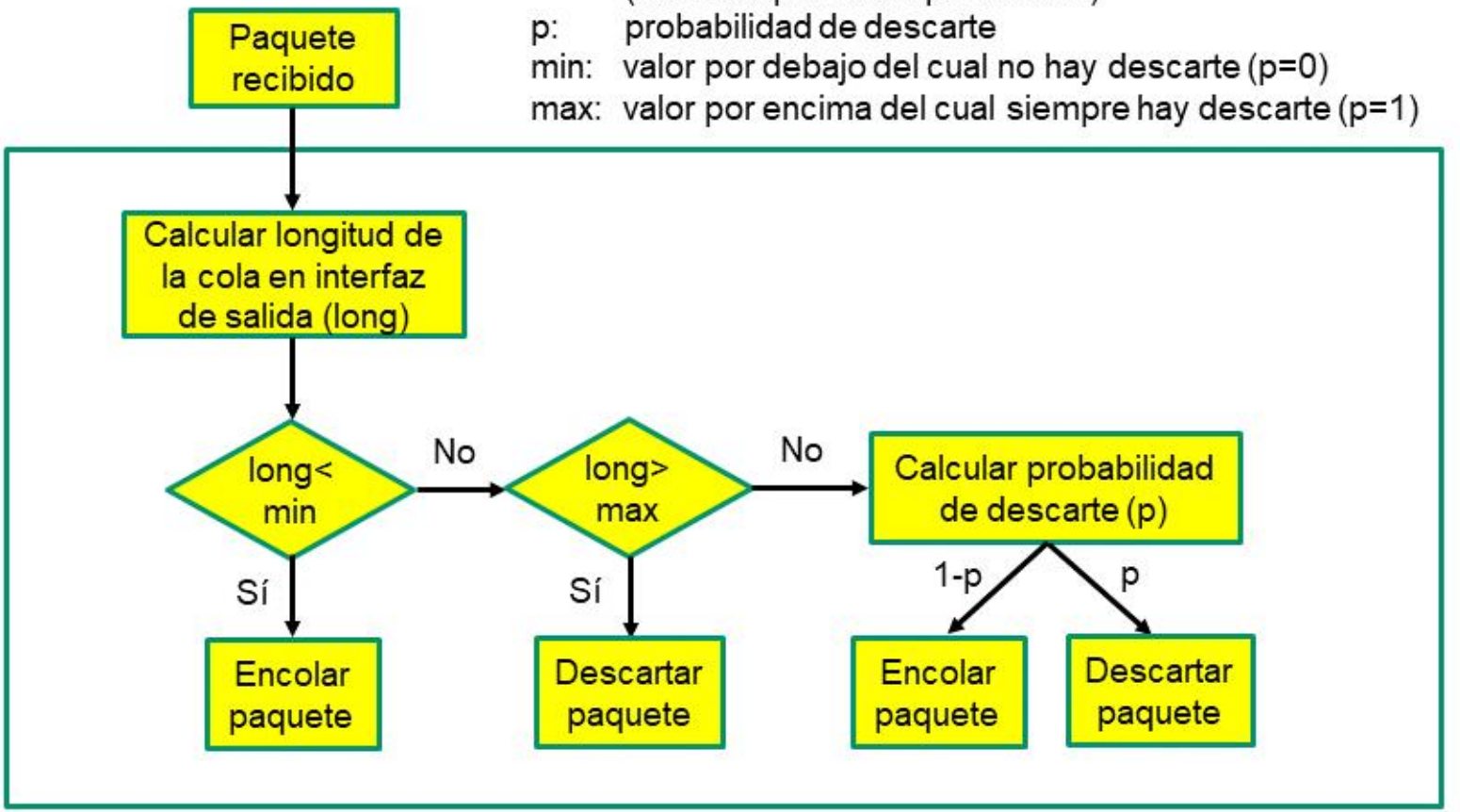


# RED (Random Early Detection)

- A principios de los 90s se detectó que el slow-start tenía un efecto negativo imprevisto. Cuando se llenan las colas los routers descartan paquetes de forma masiva, provocando el reinicio de cwnd en muchas conexiones TCP a la vez. Esto daba lugar a situaciones oscilantes y a un bajo rendimiento.
- Para evitarlo en 1993 Van Jacobson diseñó un algoritmo llamado RED que se ejecuta en los routers (RFC 7567).
- RED consiste en fijar dos umbrales, *min* y *max,* al tamaño de la cola. Cuando la cola supera *min* se empieza a descartar paquetes con una probabilidad creciente hasta llegar a *max*, a partir de ese punto la probabilidad de descarte es 1 (certeza).

# Funcionamiento de RED

long: longitud de la cola en la interfaz de salida (media exponencial ponderada)
p: probabilidad de descarte
min: valor por debajo del cual no hay descarte (p=0)
max: valor por encima del cual siempre hay descarte (p=1)

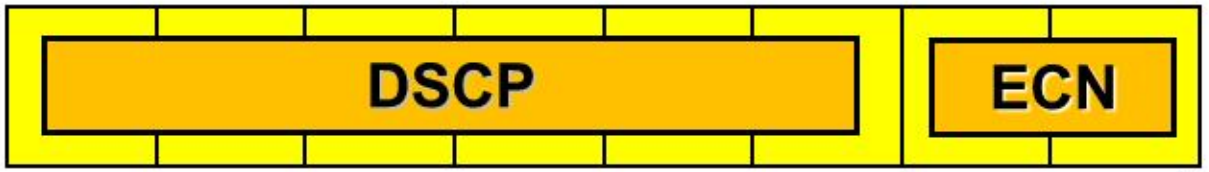


# Control explícito de congestión

- Con RED los routers descartan paquetes anticipadamente como forma de aviso. ¿No sería mejor que avisaran antes de descartar?
- En 1999 el IETF estableció un protocolo experimental para el control explícito de congestión llamado ECN (Explicit Congestion Notification).
- Con ECN los routers con RED, en vez de descartar marcan los paquetes para que los hosts sepan que hay congestión y que por tanto han de bajar el caudal
- En 2001 ECN se elevó a la categoría de 'standards track' (RFC 3168). ECN supone cambios en IP y en TCP

# Campo ECN en la cabecera IP

| Vers. | L.Cab. | DS (DiffServ) | Longitud Total | | | |
|---|---|---|---|---|---|---|
| Identificación | | | Res. | DF | MF | Desplaz Fragm. |
| T. vida (TTL) | Protocolo | | Checksum | | | |
| Dirección de origen | | | | | | |
| Dirección de destino | | | | | | |

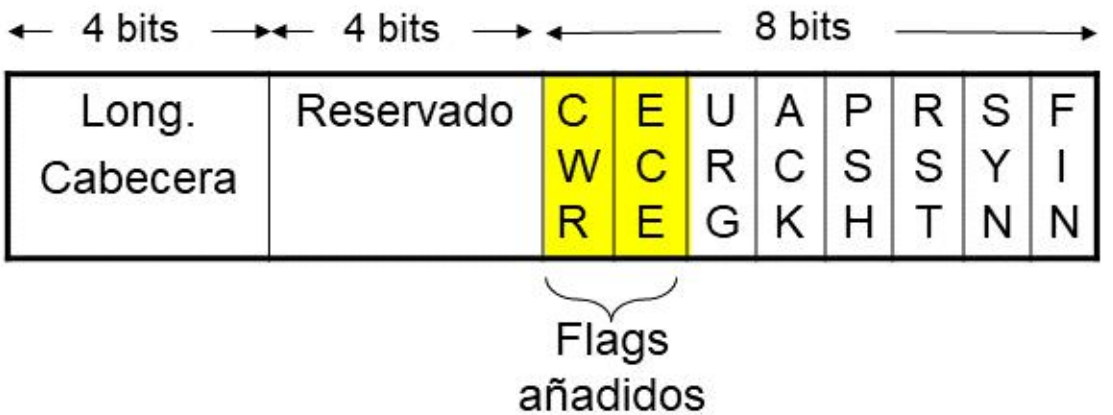



| ECN | Significado |
|---|---|
| 00 | El Host emisor no soporta ECN. |
| 01 | El Host emisor soporta ECN (modo alternativo). |
| 10 | El Host emisor soporta ECN (modo normal). |
| 11 | CE (Congestión Encontrada) |

# Cambios en la cabecera TCP

| Puerto de Origen | | | Puerto de destino |
|---|---|---|---|
| Número de Secuencia | | | |
| Número de ACK | | | |
| L. Cab | Res. | Flags | Tamaño Ventana |
| Checksum | | | Punt. datos urgentes |

Se han añadido dos flags nuevos, aprovechando la parte reservada de los octetos 13 y 14 en la cabecera:



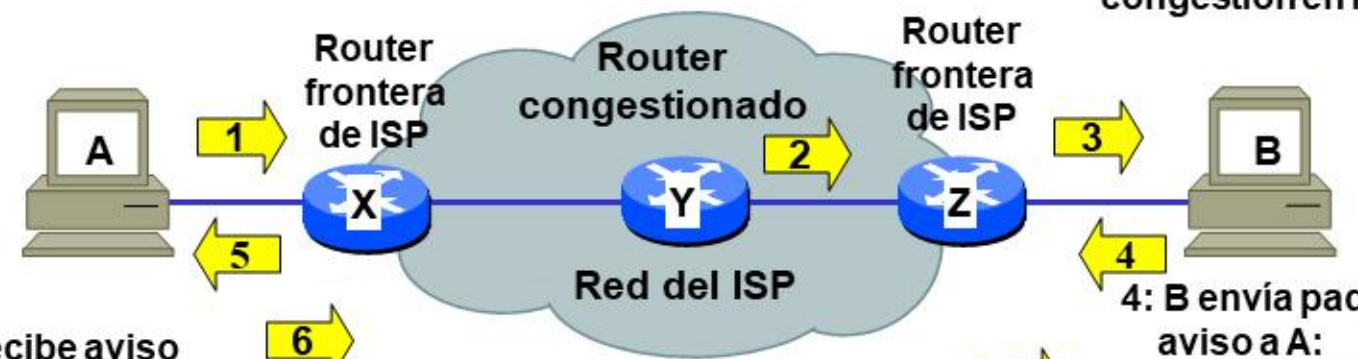

**CWR: Congestion Window Reduced**
**ECE: ECN Echo**

# Funcionamiento de IP y TCP con ECN

1: A envía un paquete a B:
IP: ECN = '10'
TCP: CWR = 0, ECE = 0

2: Router Y recibe el paquete, detecta congestión y cambia ECN a '11'

3: B recibe el paquete y al ver ECN='11' sabe que hay congestión en la red




4: B envía paquete de aviso a A:
IP: ECN = '10'
TCP: CWR = 0, ECE = 1

5: A recibe aviso de B (ECE = 1)

6: A actúa igual que si hubiera ocurrido un 'fast recovery' (TCP-Reno) y envía confirmación a B indicando que ha recibido el aviso:
IP: ECN = '10'
TCP: CWR = 1, ECE = 0

7: B recibe confirmación (CWR = 1) y se queda tranquilo, sabe que no ha de insistir con ECE = 1

# Efecto de ECN

- Cuando TCP recibe un segmento con ECE=1 debe actuar igual que cuando recibe tres ACK duplicados (fast recovery): reduce *cwnd* a la mitad del valor que tenía en ese momento y crece de manera lineal a partir de ahí.
- Obviamente no se retransmite nada porque no se ha perdido nada. Esta es la gran ventaja de ECN, que no se pierden recursos en el reenvío de segmentos.
- ECN está diseñado para poder coexistir con equipos que no soportan ECN

# Soporte de ECN

- Hosts (a partir de):
  - Linux 2.4.20 (2003)
  - Windows Vista (2007)
  - NetBSD (2007)
  - FreeBSD (2009)
  - OpenBSD
  - Mac OS X 10.5 (2007)
  - iOS 9 (2015)
  - Solaris 11 (2015)
- Routers:
  - Cisco IOS 12.2(8)T (2006)
  - Linux
  - FeeBSD, NetBSD, OpenBSD
- Aunque hoy en día está disponible en muchos S.O., ECN suele estar deshabilitado por defecto y su uso es todavía muy limitado

# Control de Congestión en Internet

| **Mecanismo** | **Consiste en:** | **Aplicado a nivel de:** |
|---|---|---|
| Slow Start, congestion-avoidance. Fast-retransmit y fast-recovery | Cuando un host detecta ACK perdidos o duplicados reduce el ritmo y se autocontrola. | Transporte (TCP) |
| RED (Random Early Detect) | Cuando los routers detectan congestión descartan paquetes al azar. Los hosts reducen el ritmo. | Red (IP) |
| ECN (Explicit Congestion Notification) | Cuando los routers detectan congestión notifican a los hosts para que reduzcan el ritmo. | Red (IP) y Transporte (TCP) |

# Sumario

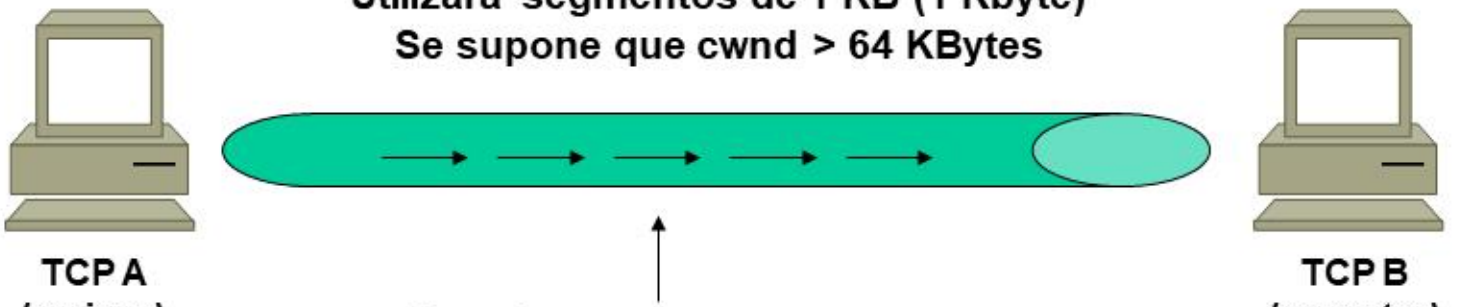
- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - **Redes LFN y factor de escala**
  - Opciones de TCP

# ¿Qué son las redes LFN?

- Las redes LFN (Long, Fat pipe Networks) o redes 'elephant' son aquellas en las que el producto del ancho de banda por el RTT es elevado. Dicho producto da la medida de lo 'LFN' que es la red. Ej.:
  - Enlace vía satélite (RTT 500 ms) de 2.048 Kb/s: BW*RTT = 1,024 Mbit
  - Enlace WAN (40 ms) de alta capacidad (1 Gb/s): BW*RTT = 40 Mbits

# TCP en redes LFN

**TCP A quiere enviar datos a TCP B de forma masiva**
**Utilizará segmentos de 1 KB (1 Kbyte)**
**Se supone que cwnd > 64 KBytes**




**0 ms: TCP A empieza a enviar segmentos a 2 Mb/s, sin parar**
**250 ms: TCP A ha enviado 64 segmentos, tiene que parar pues aun no ha recibido ningún ACK**
**500 ms: TCP A empieza a recibir los 64 ACK y transmite los siguientes 64 segmentos**
**750 ms: TCP A ha enviado el segundo grupo de 64 segmentos y tiene que parar**
**1000 ms: TCP A empieza a recibir los ACK del segundo grupo y transmite el tercero**
**1250 ms: TCP A tiene que parar**
**…**

**Eficiencia: 250/500 = 50 % = 1,048 Mb/s**
**Ventana / RTT = 0,512 Mb / 0,5 s = 1,048 Mb/s**

# Problema de TCP con las redes LFN

- La ventana de control de flujo en la cabecera TCP tiene 16 bits. Su valor máximo es 65.535.
- Por tanto en TCP no es posible enviar más de 65.535 bytes seguidos sin recibir un ACK
- Si en una red BW*RTT > 64 Kbytes el rendimiento está limitado por el tamaño de ventana. La limitación es tanto mayor cuanto mayor es el BW*RTT de la red
- La limitación se aplica a cada conexión TCP por separado, si hay varias en paralelo pueden usar más capacidad

# Solución al problema de TCP con las redes LFN

- Puesto que el campo tamaño de ventana no se puede ampliar los dos TCPs acuerdan no contar los bytes de uno en uno, sino de dos en dos, de cuatro en cuatro, etc. Siempre en potencias enteras de dos (es como añadir ceros por la derecha).
- Esto es lo que se conoce como factor de escala y para poder usarlo se ha de negociar en el momento de conectarse. Se negocia en una opción del segmento SYN, ha de estar soportado por los dos TCP y si se acuerda se ha de mantener durante toda la vida de la conexión

# Ejemplo de uso del factor de escala

- El IFIC (Instituto de Física Corpuscular) realiza transferencias de datos masivas entre Valencia y el CERN en Ginebra, Suiza
- En pruebas hechas con un FTP entre máquinas conectadas a 100 Mb/s se obtenía un caudal máximo de 9 Mb/s
- Se observó que lanzando varias transferencias en paralelo se obtenían caudales superiores, pudiendo acercase a los 100 Mb/s
- Esto hizo sospechar que la limitación no se debía al ancho de banda de la conexión, sino al BW*RTT

```
C:\Documents and Settings\montanan>ping www.cern.ch

Haciendo ping a webr2.cern.ch [137.138.28.230] con 32 bytes de datos:

Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114
Respuesta desde 137.138.28.230: bytes=32 tiempo=43ms TTL=114

Estadísticas de ping para 137.138.28.230:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0 (0% perdidos),
Tiempos aproximados de ida y vuelta en milisegundos:
    Mínimo = 43ms, Máximo = 43ms, Media = 43ms


C:\Documents and Settings\montanan >tracert www.cern.ch

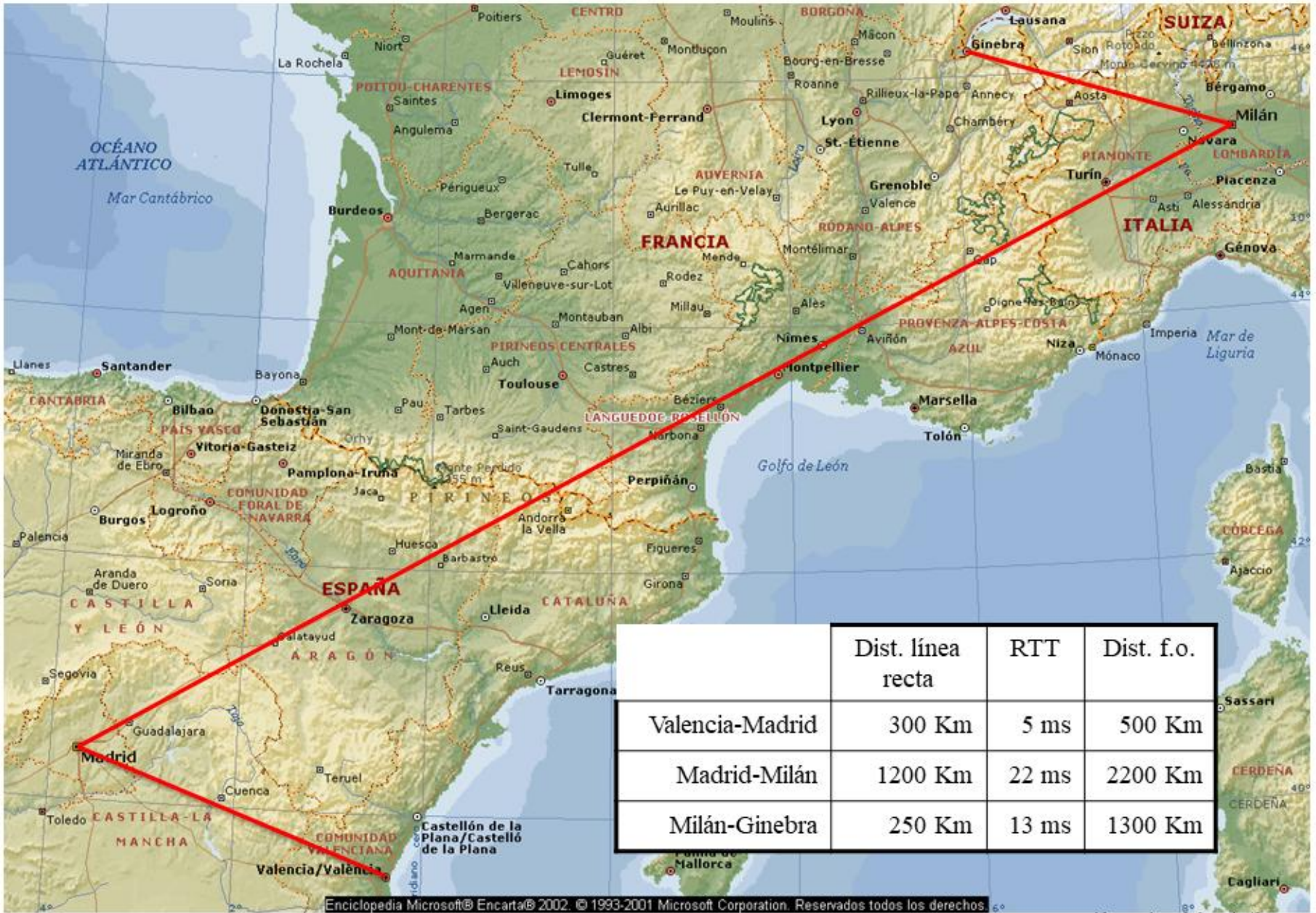
Traza a la dirección webr2.cern.ch [137.138.28.230] sobre un máximo de 30 saltos:

  1    <1 ms    <1 ms    <1 ms  burl3ci.red.uv.es [147.156.2.50]
  2    <1 ms    <1 ms    <1 ms  burladerol3.red.uv.es [147.156.200.184]
  3    <1 ms    <1 ms    <1 ms  neveraout.red.uv.es [147.156.7.1]
  4    <1 ms    <1 ms    <1 ms  AT0-2-0-0.EB-Valencia0.red.rediris.es [130.206.211.181]
  5     6 ms     6 ms     6 ms  VAL.SO3-0-0.EB-IRIS4.red.rediris.es [130.206.240.13]
  6     7 ms     7 ms     7 ms  rediris.es1.es.geant.net [62.40.103.61]
  7    29 ms    29 ms    29 ms  es.it1.it.geant.net [62.40.96.186]
  8    42 ms    42 ms    42 ms  it.ch1.ch.geant.net [62.40.96.33]
  9    43 ms    42 ms    42 ms  swiCE2-P6-1.switch.ch [62.40.103.18]
 10    43 ms    43 ms    42 ms  cernh7-gb1-1.cern.ch [192.65.184.222]
 11    42 ms    43 ms    42 ms  cernh2-vlan2.cern.ch [192.65.192.2]
C:\Documents and Settings\montanan>
```

5 ms { 4 5

22 ms { 6 7

13 ms { 8




| | Dist. línea recta | RTT | Dist. f.o. |
|---|---|---|---|
| Valencia-Madrid | 300 Km | 5 ms | 500 Km |
| Madrid-Milán | 1200 Km | 22 ms | 2200 Km |
| Milán-Ginebra | 250 Km | 13 ms | 1300 Km |

# Rendimiento de la conexión TCP

- Caudal máximo = ventana / RTT
- Con RTT = 43 ms y ventana 524.288 bits:
  Caudal máximo = 524.288 / 0,043 = **12,2 Mb/s**
- Para mejorar el rendimiento se configuró el uso de factor de escala en TCP, obteniendo así un mayor tamaño de ventana. Con un factor 16x ya se conseguían casi los 100 Mb/s (velocidad de la interfaz):

| Factor de escala | Tam. Ventana (bits) | Caudal max. (Mb/s) |
|---|---|---|
| 0 (1x) | 524.288 | 12,2 |
| 1 (2x) | 1.048.576 | 24,4 |
| 2 (4x) | 2.097.152 | 48,8 |
| 3 (8x) | 4.194.304 | 97,5 |
| 4 (16x) | 8.388.608 | 195,1 |

# Factor de escala

- El factor de escala puede añadir como máximo 14 ceros, equivalente a una ventana de $2^{30} = 1$ GB.
- Con un RTT de 86 ms y un caudal de 100 Gb/s ese tamaño de ventana ya resulta insuficiente
- Hay que destacar no obstante que la limitación se aplica a cada conexión TCP individual e independientemente
- El factor de escala se ha de fijar a ciegas en la conexión, no existe un mecanismo de 'descubrimiento del factor de escala del trayecto'

# Ejercicio: redes LFN

P: Sesión TCP con 100 Mb/s de BW y 20 ms de RTT. Calcular caudal máximo aprovechable suponiendo que el factor de escala no está disponible.

R: De la fórmula: Ventana = BW * RTT

obtenemos: BW = Ventana / RTT

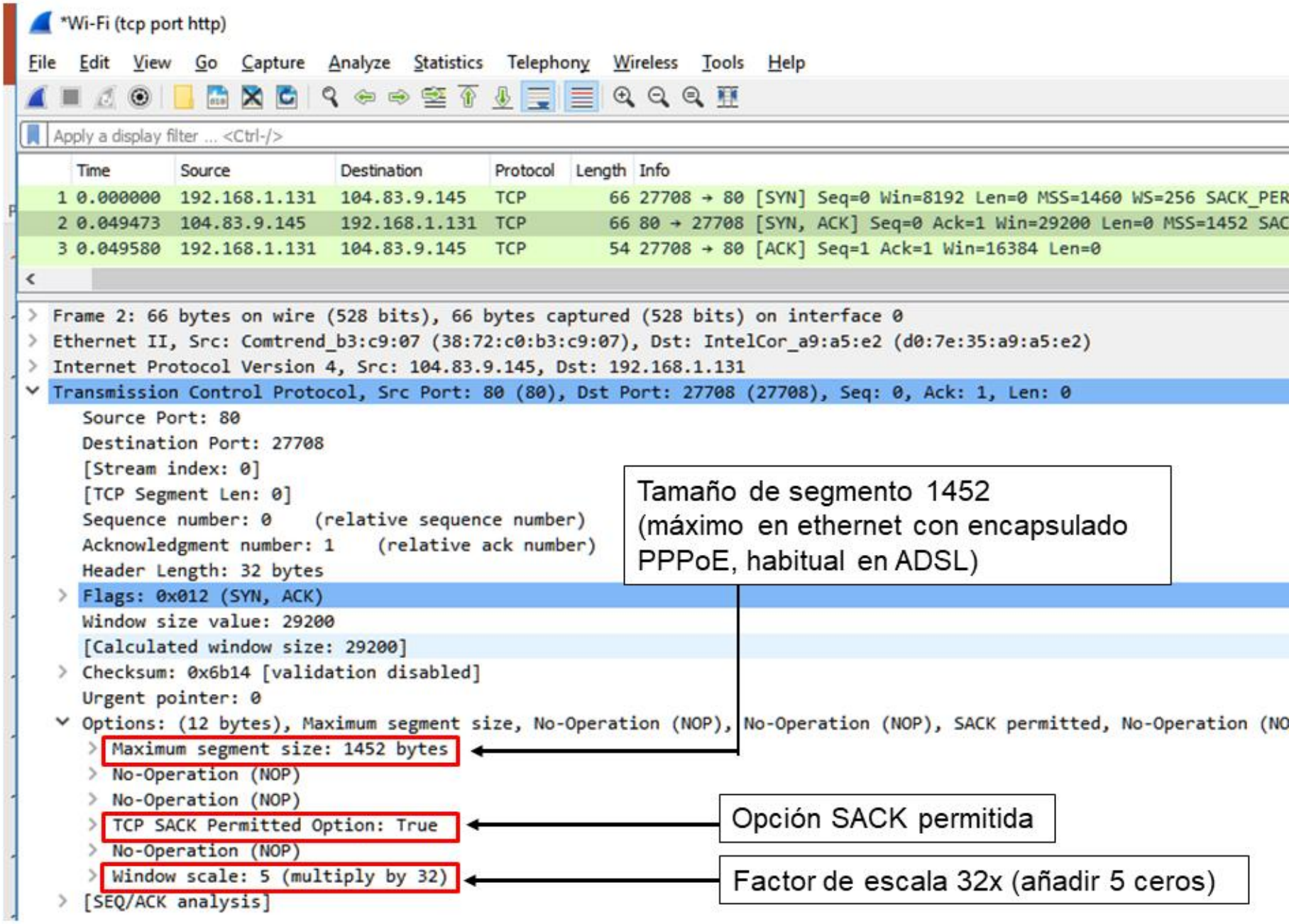
Sustituyendo: BW = 65.535*8 / 0,02 = 26,2 Mb/s

# Sumario

- Introducción
- Protocolo UDP
- Protocolo TCP
  - Multiplexación
  - Conexión
  - Desconexión
  - Intercambio de datos y control de flujo
  - Mejoras de rendimiento
  - Cálculo del timer de retransmisión
  - Control de congestión
  - Redes LFN y factor de escala
  - **Opciones de TCP**

# Opciones más habituales de TCP

- **Anuncio del MSS**: solo se admite en los segmentos SYN. Cada TCP informa al otro del MSS que aceptará. El tamaño puede ser diferente para cada sentido.
- **Factor de escala**: solo se admite en los segmentos SYN. Mejora la eficiencia en redes 'LFN'
- **SACK-permitted.** solo se admite en los segmentos SYN. Indica si se soporta la opción SACK
- **SACK (ACK selectivo):** permite que TCP confirme la recepción de segmentos no consecutivos. Puede aparecer en cualquier segmento.


*Wi-Fi (tcp port http)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
Time Source Destination Protocol Length Info
1 0.000000 192.168.1.131 104.83.9.145 TCP 66 27708 → 80 [SYN] Seq=0 Win=8192 Len=0 MSS=1460 WS=256 SACK_PER
2 0.049473 104.83.9.145 192.168.1.131 TCP 66 80 → 27708 [SYN, ACK] Seq=0 Ack=1 Win=29200 Len=0 MSS=1452 SAC
3 0.049580 192.168.1.131 104.83.9.145 TCP 54 27708 → 80 [ACK] Seq=1 Ack=1 Win=16384 Len=0
Frame 2: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
Ethernet II, Src: Comtrend_b3:c9:07 (38:72:c0:b3:c9:07), Dst: IntelCor_a9:a5:e2 (d0:7e:35:a9:a5:e2)
Internet Protocol Version 4, Src: 104.83.9.145, Dst: 192.168.1.131
Transmission Control Protocol, Src Port: 80 (80), Dst Port: 27708 (27708), Seq: 0, Ack: 1, Len: 0
Source Port: 80
Destination Port: 27708
[Stream index: 0]
[TCP Segment Len: 0]
Sequence number: 0 (relative sequence number)
Acknowledgment number: 1 (relative ack number)
Header Length: 32 bytes
Flags: 0x012 (SYN, ACK)
Window size value: 29200
[Calculated window size: 29200]
Checksum: 0x6b14 [validation disabled]
Urgent pointer: 0
Options: (12 bytes), Maximum segment size, No-Operation (NOP), No-Operation (NOP), SACK permitted, No-Operation (NO
Maximum segment size: 1452 bytes
No-Operation (NOP)
No-Operation (NOP)
TCP SACK Permitted Option: True
No-Operation (NOP)
Window scale: 5 (multiply by 32)
[SEQ/ACK analysis]
Tamaño de segmento 1452 (máximo en ethernet con encapsulado PPPoE, habitual en ADSL)
Opción SACK permitida
Factor de escala 32x (añadir 5 ceros)

**Los contadores de netstat son de todo el sistema desde el último arranque**

```
Netstat -p tcp
tcp:
  978740 packets sent
    949215 data packets (1306073886 bytes)
    544 data packets (329353 bytes) retransmitted
    10186 ack-only packets (8786 delayed)
    0 URG only packets
    188 window probe packets
    17669 window update packets
    938 control packets
  432947 packets received
    251266 acks (for 863756680 bytes)
    1294 duplicate acks
    0 acks for unsent data
    76150 packets (68148251 bytes) received in-sequence)
    174 completely duplicated packets (57347 bytes)
    15 packets with some dup. data (34 bytes duped)
    341 out-of-order packets (4224 bytes)
    23 packets (0 bytes) of data after window
    0 window probes
    23158 window update packets
    1 packet received after close
    0 discarded for bad checksums
    0 discarded for bad header offset fields
    0 discarded because packet too short
  397 connection requests
  414 connection accepts
  629 connections established (including accepts)
  848 connections closed (including 38 drops)
  179 embryonic connections dropped
  313267 segments updated rtt (of 314002 attempts)
  347 retransmit timeouts
    2 connections dropped by rexmit timeout
  190 persist timeouts
  54 keepalive timeouts
    53 keepalive probes sent
    1 connection dropped by keepalive
```

0,06 % retransmitidos

188 Paquetes enviados por el timer de persistencia

0,5 % ACK duplicados

0,04 % duplicados (indica ACK perdidos)

0,1 % recibidos fuera de orden

1 recibido después de cerrar

0 erróneos o malformados

28% Conex. embrionarias abortadas

72% actualizaron el MRTT

0,04 % agotaron RTO

# Cronología TCP

- 05/1974: "A protocol for packet networks Intercom." (Cerf y Kahn)
- 12/1974: TCP ( P = Program) (Cerf, RFC675)
- 1975: Saludo a 3 vías (Tomlinson, ACM SIGCOMM)
- 1978: TCPv3, primera versión independiente de IP
- 1982: TCPv4, versión actual (Postel, RFC 793)
- 1982: Solución de Clark (síndrome de la ventana tonta)
- 01/01/1983: ARPANET migra a TCP/IP
- 08/1983: TCP/IP se incluye en 4.2BSD
- 1984: Algoritmo de Nagle (Nagle, RFC 896)
- 1986: Se produce el colapso de congestión en Internet
- 1987: Algoritmo de Karn (Karn, Partridge, ACM SIGCOMM)
- 1988: TCP-Tahoe (Van Jacobson, 4.3BSD)
- 1988: Factor de escala (Van Jacobson RFC 1072)
- 1988: SACK (Van Jacobson, RFC 1072)
- 1990: Path MTU Discovery (Mogul RFC 1191)
- 1990: TCP-Reno (Van Jacobson, 4.3BSD)
- 1993: RED (Van Jacobson, RFC 7567)
- 1999: ECN (Floyd, Ramakrishnan, RFC 3168)

# Comparación TCP - UDP

| **Función** | **TCP** | **UDP** |
|---|---|---|
| Transporte | Sí | Sí |
| Multiplexación | Sí | Sí |
| Detección de errores | Sí | Opcional(*) |
| Corrección de errores | Sí | No |
| Control de flujo | Sí | No |
| Control de congestión | Sí | No |
| Establecimiento/ terminación de conexión | Sí | No |

(*) Obligatoria en IPv6

# Proceso de una trama Ethernet TCP/IP recibida por un host

